

B. TECH. PROJECT REPORT
ON
ML VISION STUDIO AND ITS
APPLICATION TO PREDICT
SLIPPER WEAR IN AXIAL PISTON
PUMPS

BY
DARRYL ARUN DAVID (220003022)



DEPARTMENT OF MECHANICAL ENGINEERING
INDIAN INSTITUTE OF TECHNOLOGY INDORE
DECEMBER 2025

ML VISION STUDIO AND ITS APPLICATION TO PREDICT SLIPPER WEAR IN AXIAL PISTON PUMPS

A PROJECT REPORT

*Submitted in partial fulfillment of the
requirements for the award of the degree*

of
BACHELOR OF TECHNOLOGY
in

MECHANICAL ENGINEERING

Submitted by:
DARRYL ARUN DAVID (220003022)

Guided by:
DR. ANKUR MIGLANI AND PROF. PAVAN KUMAR KANKAR



**INDIAN INSTITUTE OF TECHNOLOGY INDORE
DECEMBER 2025**

CANDIDATE’S DECLARATION

We hereby declare that the project entitled “**ML Vision Studio and its Application to Predict Slipper Wear in Axial Piston Pumps**” submitted in partial fulfillment for the award of the degree of Bachelor of Technology in ‘Mechanical Engineering’ completed under the supervision of **Dr. Ankur Miglani, Associate Professor, Mechanical Engineering, IIT Indore** is an authentic work.

Further, I/we declare that I/we have not submitted this work for the award of any other degree elsewhere.



Signature and name of the student(s) with date

CERTIFICATE by BTP Guide(s)

It is certified that the above statement made by the students is correct to the best of my/our knowledge.



Associate Professor

04-12-2025

Signature of BTP Guide(s) with dates and their designation

Preface

This report on “**ML Vision Studio and its Application to Predict Slipper Wear in Axial Piston Pumps**” is prepared under the guidance of Dr. Ankur Miglani and Prof. Pavan Kumar Kankar.

Through this report I have given a detailed description of the working and internal functioning of a novel graphical desktop program which facilitates codeless setup and training of popular image-based CNN models like ResNet, MobileNet, Inception, and EfficientNet, paired with a separate, robust backend system that can run on a powerful sever, to asynchronously train and store these models.

Additionally, I have demonstrated the use of this tool by training a custom CNN model that predicts the wear levels of slippers in axial piston pumps, among five classes, achieving an exceptional accuracy, and proving the absence of overfit by further testing the model with varying levels of Gaussian noise.

Darryl Arun David

B.Tech. IV Year

Department of Mechanical Engineering

IIT Indore

Acknowledgements

I wish to thank Dr. Ankur Miglani, Prof. Pavan Kumar Kankar, Anoop K R, Niteen Sapkal, and Neeraj Sonkar for their kind support and valuable guidance.

It is their help and support, due to which I was able to complete the design and technical report.

Without their support this report would not have been possible.

Darryl Arun David

B.Tech. IV Year

Department of Mechanical Engineering

IIT Indore

Abstract

The traditional methods of creating a CNN based image classification model involve several tedious steps, involving acquiring the images, processing them, tons of coding, transferring the code and the training set to a powerful computer, executing the training, carrying out refinements, and transferring the metrics, results, and the outputs back to your main computer. This process was tiresome for our lab, especially for those who aren't proficient coders, where much of the methodology for training remains same across multiple projects such as damage classifications for rice grains, soyabeans, and other crops.

This project aims to largely simplify and generalize most steps discussed above, while eliminating the need to code, without compromising too much on customizability. It does so by providing a sleek and simple graphical user interface, systematically revealing various configurations often used in our lab for several pre-trained models like MobileNet, EfficientNet, ResNet, and Inception. The software also provides a neat graphical interface for creating labeled datasets, and in doing so, also optionally processes the images (such as cropping and rotation, as the cameras capture in resolutions different from the desired specifications of the model). The software also has a storage system that keeps the training sets and the trained results organized. Processing images can be time-taking on weaker computers, and so the software caches the processed images, which are utilized when the same images are used again, skipping the processing.

Additionally, ML Vision Studio was used to train a custom CNN model to non-invasively predict slipper wear in axial piston pumps using vibration data from an accelerometer run through continuous wavelet transformation, among five classes of Healthy, 10 μm , 45 μm , 75 μm , and 100 μm , with a remarkable accuracy of 99.7%. To test for overfit, varying levels of Gaussian noise of SNRs 30 db, 15 db, and 10 db were applied to the training set, which resulted in a reduction in accuracy, proving the model to be well-fitted.

Table of Contents

<i>CANDIDATE'S DECLARATION</i>	<i>v</i>
<i>CERTIFICATE by BTP Guide(s)</i>	<i>v</i>
<i>Preface</i>	<i>vii</i>
<i>Acknowledgements</i>	<i>viii</i>
<i>Abstract</i>	<i>ix</i>
<i>Table of Contents</i>	<i>xi</i>
<i>List of Figures</i>	<i>xv</i>
<i>CHAPTER 1: INTRODUCTION</i>	<i>1</i>
<i>1.1 Motivation</i>	<i>1</i>
<i>1.2 Objective</i>	<i>2</i>
<i>CHAPTER 2: SOLUTION</i>	<i>4</i>
<i>2.1 Architecture</i>	<i>4</i>
<i>2.2 Setup Workflow</i>	<i>5</i>
2.2.1 Home Page.....	5
2.2.2 Create New Page	7
2.2.2.1 Training Set Builder	8
2.2.2.2 Layer Configuration Page.....	14
2.2.2.3 Parameters Configuration Page	17
2.2.2.4 Split Selection Page	25
2.2.2.5 Setup Complete Page	29
2.2.2.6 Error Checking Service.....	31
2.2.3 Export Page.....	34
2.2.4 Training Set Details Page	38
2.2.5 View Tasks Page	40
2.2.6 Completed Task Information Page	42
2.2.7 Settings Page.....	45
<i>2.3 Backend</i>	<i>51</i>
<i>CHAPTER 3: IMPLEMENTATION</i>	<i>54</i>
<i>3.1 MLVS 1</i>	<i>54</i>

3.1.1 Flutter.....	55
3.1.1.1 The Dart Programming Language and Compilation	55
3.1.1.2 Desktop and plugin ecosystem	56
3.1.1.3 Rationale for using Flutter in MLVS 1	56
3.1.2 Project Structure	57
3.1.2.1 Project Tree.....	57
3.1.2.2 Lib Tree	58
3.1.2.3 Relation to common Flutter layouts	59
3.1.3 Dependencies.....	60
cupertino_icons:.....	61
file_picker:.....	61
image:	61
shared_preferences:	62
path:	62
path_provider:.....	62
archive:	63
file_selector:	63
flutter_staggered_grid_view:.....	63
http:	63
3.1.4 Utilities and Data	64
3.1.4.1 Image Processing	64
ImgData: image representation and lifecycle	64
ImgProcessing: responsibilities and overall workflow	65
Native conditional processing with dart:ui	66
Decoding and setup via dart:ui.....	66
Rotation + center crop branch	68
Center crop + resize without rotation.....	70
Error handling and legacy fallback.....	71
Batch and multithreaded processing: from FilePickerResult to ImgData	71
Single-isolate batched processing	71
Multithreaded processing with isolates.....	72
Why dart:ui-based resizing is preferred over the legacy path	75
3.1.4.2 Archive Service	78
ArchiveService and the Archive package.....	78
Desktop exportDataset workflow	79
Web in-memory export and ExportResult.....	80
Archive folder structure and metadata contents	81
Integration with ExportPage and MLVS 2	81
3.1.5 Storage System	82
3.1.5.1 Permanent Storage	82
Application support directory and platform locations	82
PermanentData: structure and responsibilities.....	83
Integration with the Settings “Storage” section and deletion safeguards	85
3.1.5.2 Cache	86
Temporary directory and its platform mapping.....	87

CachedData.....	88
Time complexity of image processing with caching	89
Cache control, Settings integration, and automatic invalidation	91
3.1.6 Packaging.....	92
macOS.....	93
Windows	94
Other supported platforms and build commands	95
Rationale for the chosen packaging approach	96
3.2 MLVS 2	97
3.2.1 Docker	97
Role.....	97
Containerization of the API and worker	98
Benefits	99
3.2.2 FastAPI	100
Handling uploads and long-running jobs.....	100
Benefits	101
3.2.3 Redis	101
Deployment and connectivity	102
Redis as Celery broker for training jobs	102
Redis as job metadata and status store.....	103
Benefits	104
3.2.4 Celery.....	104
Role.....	105
Configuration and execution environment	106
CHAPTER 4: APPLICATION.....	107
4.1 Objective	107
4.2 Axial Piston Pumps	108
4.3 Slippers.....	110
4.4 Factors that indicate wear in slippers	111
4.4 Measuring Vibrations	113
4.5 Acquiring the Data	116
4.6 Preparing the Data	117
4.6.1 Merging the data.....	118
4.6.2 Plotting the complete merged data	120
4.6.3 Removing the transients	123
4.6.4 Splitting the Data	129
4.6.5 Bifurcating the Data into Train, Validation, and Test Sets	131
4.6.6 Running Continuous Wavelet Transformation (CWT)	132

4.7 Model Architecture	135
4.7.1 Convoluted Neural Networks (CNNs)	135
4.7.2 Architecture of our Custom CNN model.....	137
4.8 Model Performance.....	141
Confusion Matrix:.....	143
Training History:	144
4.9 Adding Gaussian Noise	146
4.8.1 Results for SNR 30 db.....	149
Confidence analysis:	149
Confusion Matrix:.....	150
4.8.2 Results for SNR 15 db	151
Confidence analysis:	151
Confusion Matrix:.....	152
4.8.2 Results for SNR 10 db.....	153
Confidence analysis:	153
Confusion Matrix:.....	154
<i>CHAPTER 5: RESULTS AND ANALYSIS.....</i>	<i>156</i>
5.1 ML Vision Studio	156
5.2 Slipper Wear Prediction in Axial Piston Pumps	158
<i>CHAPTER 6: CONCLUSION AND SCOPE FOR FUTURE WORK</i>	<i>161</i>
6.1 Conclusion.....	161
6.2 Scope for Future Work.....	162
<i>REFERENCES AND LINKS.....</i>	<i>166</i>

List of Figures

Fig 2.1 The Home Page	6
Fig 2.2 Step 1: The Training Set Builder of The Create New Page.....	9
Fig 2.3 List of empty classes in the Training Set Builder.....	10
Fig 2.4 New batch of images being processed and added to a class.....	11
Fig 2.5 All classes populated with images.....	12
Fig 2.6 Image options revealed on mouse hover	13
Fig 2.7 Class delete option revealed on mouse hover which temporarily hides the class size.....	13
Fig 2.8 Step 2: The Layers Configuration Page.....	15
Fig 2.9 Adding and deleting a layer.....	16
Fig 2.10 Drag and drop reordering of layers.....	17
Fig 2.11 The Parameters Configuration Page it its widest layout.....	20
Fig 2.12 The Parameters Configuration Page it its narrowest layout	21
Fig 2.13 The Parameters Configuration Page with modified unsaved elements	22
Fig 2.14 Tool Tip for Unsaved Changes that appears on mouse hover.....	23
Fig 2.15 A slider at maximum value.....	23
Fig 2.16 A value typed into the slider element that exceeds its maximum limit.....	24
Fig 2.17 Slider knob turns red to indicate a non-recommended saved value	24
Fig 2.18 Import Utility	25
Fig 2.19 The Split Page.....	27
Fig 2.20 The Split Page with Text Input.....	28
Fig 2.21 Slider Mouse Cursor style while hovering on slider	28
Fig 2.22 The Setup Complete Page.....	30
Fig 2.23 The Setup Complete Page with a valid name	30
Fig 2.24 Error service dialog in Create New Page.....	33
Fig 2.25 Error service with warnings only and no serious errors	34
Fig 2.26 The Export Page	36
Fig 2.27 On exit, the app navigates back to the Home Page, which now has the displays the newly created training set at the top below the Refresh button	37
Fig 2.28 The Training Set Detail Page.....	40

Fig 2.29 The View Tasks Page	42
Fig 2.30 The Completed Task Info Page	45
Fig 2.31 The Settings Page (part 1)	48
Fig 2.32 The Settings Page (part 2)	49
Fig 2.33 Clear Cache Confirmatory Dialog.....	49
Fig 2.34 Clear Permanent Storage Confirmatory Dialog with extra safety	50
Fig 2.35 Clear Cache Confirmatory Dialog with clear buttons activated.....	50
Fig 3.1 Popup shown while processing images	77
Fig 3.2 Processing images popup while adding images	78
Fig. 4.1 Vibration Sensor	115
Fig. 4.2 A snippet of the recorded vibration data	117
Fig. 4.3 Plot of time-series data of the complete dataset	122
Fig. 4.4 Frequency-domain plot of data of the complete dataset.....	123
Fig. 4.5 Graphical interface of Remover, which shows the plots with down sampling options for performance. Plots can be dragged and selected	125
Fig. 4.6 Graphical interface of Remover, which shows one of the plots selected across a range with a simple drag of with the mouse.	125
Fig. 4.7 Remover auto selects the same range in the other graphs to synchronize them.....	126
Fig. 4.8 The selected portions of the graphs can be removed with a click, and the gaps are auto filled by shifting the time.	126
Fig. 4.9 Remover has a built-in FFT view to compare them after reduction. This is the original FFT plot	127
Fig. 4.10 After removal FFT plot.....	127
Fig. 4.11 Time-domain plot after removal of transients	128
Fig. 4.12 Frequency-domain plot after removal of transients.....	128
Fig. 4.13 Plot of a chunk of data lasting ~ 0.2 s containing exactly 340 points each.....	130
Fig. 4.14 One of the resulting images after running CWT	134
Fig. 4.15 Model Summary	140
Fig. 4.16 Training Log.....	145
Fig. 4.17 Frequency-domain plot for data with SNR 10 db Gaussian Noise.....	155

CHAPTER 1: INTRODUCTION

1.1 Motivation

Traditional workflows for training image-based convolutional neural networks typically require a long sequence of disjoint, manual steps carried out across multiple tools and environments. The raw image data must first be collected, inspected, cleaned, and organized into a directory structure that encodes labels, followed by manual resizing, cropping, rotation, and normalization through ad hoc scripts or command-line tools. Subsequently, suitable CNN architectures must be researched and selected, layer configurations and hyperparameters must be designed, and framework-specific training code must be written, debugged, and adapted to the available hardware and operating system. For many laboratory projects, especially where students and researchers do not specialize in machine learning or software engineering, this fragmented process becomes time-consuming, error-prone, and difficult to reproduce.

In typical laboratory practice, each new image-classification problem often leads to the same repetitive cycle: separate scripts are created for dataset preparation, model definition, training, and evaluation, and different experiments are tracked manually in notes, screenshots, or scattered log files. This reliance on loosely connected scripts and interfaces (such as text editors, terminals, Jupyter notebooks, and file browsers) increases cognitive load and makes it challenging to compare experiments or hand over a project to another student. In addition, setting up and securely using remote GPU servers—while essential for training modern CNNs efficiently—requires familiarity with SSH, key management, Python environments, and containerization, which can act as a barrier for domain experts in areas such as mechanical engineering.

Because of these factors, there is a clear need for a more integrated, user-friendly approach that abstracts low-level implementation details while still preserving the flexibility required for serious experimentation. A graphical system that brings together dataset construction, model configuration, and remote training into a coherent workflow can make CNN-based image classification more accessible to non-coders, reduce redundant effort across projects, and improve the consistency and reproducibility of results in the laboratory. In this context, ML Vision Studio has been conceived as a generalized, product-quality software platform that systematizes these

tasks for a wide range of image-classification applications, rather than being tied to a single experimental dataset or domain

1.2 Objective

The primary objective of this project is to design and develop a graphical desktop program that streamlines the complete workflow of training popular image-based CNN models. This system is intended to replace ad hoc scripts and fragmented tools with a single coherent environment in which users can construct labelled image datasets, configure model architectures and hyperparameters, and manage experiment variants without writing code.

A second objective is to encapsulate best practices for dataset preparation and experiment design into a guided, four-step workflow, so that non-expert users can create valid training sets consistently. The application is required to support tasks such as importing and organizing images into labels, performing optional but configurable preprocessing operations (resizing, cropping, centering, and rotation) with caching for performance, and defining train/validation/test splits using interactive controls, while providing immediate feedback on potential configuration errors.

A further objective is to provide a flexible yet abstracted configuration mechanism for popular CNN models and training parameters that is suitable for a broad range of applications. The interface must expose model selection (for example, EfficientNet, MobileNet, ResNet, and Inception), layer customization via a dedicated layers view, and a comprehensive parameter panel for learning rate schedules, optimizers, regularization, augmentation, and transfer learning, with all settings stored in a way that can be reused and modified.

In addition, the project aims to implement a robust local storage system that maintains a library of training sets, each consisting of a configuration file and the corresponding image dataset. This library is intended to allow users to revisit previous experiments, inspect their configurations, rename or delete them, and send selected training sets to a remote server for execution, all from the home screen of the application.

Finally, an essential objective is to integrate this graphical application with a scalable, containerized backend that executes training jobs on a powerful server while remaining transparent to the end user. The system must be capable of packaging training sets, transmitting them

wirelessly, training them asynchronously on GPU-enabled containers, and storing the resulting models and metrics so that they can be retrieved from within the app interface, thereby closing the loop between configuration, execution, and analysis.

CHAPTER 2: SOLUTION

2.1 Architecture

ML Vision Studio is organized as a two-part system that cleanly separates user interaction from heavy model training. The first part is a graphical cross-platform desktop application built with Flutter, designed to run on Windows, macOS, and Linux, with limited but functional support on web and mobile platforms. This application is responsible for all human–computer interaction, including dataset creation, image preprocessing, model and hyperparameter configuration, and management of previously created training sets, which are stored locally as JSON configuration files paired with zipped image datasets.

The second part of the system consists of a set of Docker containers running on a powerful server, typically a workstation computer. This backend is implemented around a FastAPI web service that exposes HTTP endpoints for creating and monitoring training jobs, a Celery worker pool that executes the model training tasks, and a Redis instance used for job tracking and inter-process communication. Each training job is handled in an isolated job directory, where the received configuration JSON and dataset zip file are stored, unpacked, and used by the training script that runs inside the containerized environment.

Communication between the graphical app and the training cluster is performed using standard HTTP requests and JSON payloads, which allows the two parts to run either on the same physical machine or on separate systems connected over a network. When a user completes the four-step configuration flow in the Flutter application and exports a training set, the app packages the configuration and dataset into separate files and sends them to the FastAPI server, where a new job identifier is created and a Celery task is queued. The app can subsequently query job status via a dedicated status endpoint, which returns information such as the job state, configuration name, model type, and basic training parameters, enabling the user to monitor progress from the same interface used to create the experiment.

Because of this design, the graphical app and the training service are loosely coupled and can run either on the same computer or on different machines connected over a network. The Flutter

application focuses entirely on user interaction and local organization of training sets, while the server focuses on reliable, repeatable execution of training workloads inside Docker containers. As a result, users experience ML Vision Studio as a single integrated system: training sets are created and named in the desktop interface, sent to the server with one action, and their status and results are viewed again from the same app, even though the heavy computation is handled remotely.

2.2 Setup Workflow

2.2.1 Home Page

The Home page serves as the main entry point of ML Vision Studio and provides an overview of all training sets created on the current machine. It presents a clean, centered layout with a prominent “Home” title and a fixed content width, so that the interface appears uncluttered and readable on different screen sizes. Below the title, the key actions are presented in a simple vertical list, allowing a new user to understand immediately how to start using the application.

The first action on the Home page allows a new training set to be created. A dedicated “Create New Training Set” item navigates to the multi-step workflow where images can be organized into labels, model architectures can be configured, and hyperparameters can be specified. A second item, “View Tasks”, directs the user to the tasks view, where previously submitted training jobs and their statuses are listed. Together, these two options expose the two core activities supported by the system: defining new experiments and inspecting those that have already been sent for training.

The Home page also functions as a library of saved training sets. A central section lists all locally stored configurations, each displayed as a list item showing the training set name (derived from the configuration name if available) and the date and time at which it was created. If no training sets are present, a message is displayed indicating that none have been saved yet, and prompting the user to create a new one. A “Refresh” control is provided to reload this list, so that newly created or modified training sets appear without restarting the application.

Each entry in the list can be selected to open a detailed view of the corresponding training set. When a user clicks on an item, the application navigates to a dedicated information page, where the configuration, dataset details, and actions such as exporting or sending to the server are available. After returning from this page, the Home view automatically refreshes its list, ensuring that any changes made to the training set are reflected immediately. This behavior supports an efficient workflow in which users can repeatedly inspect, adjust, and manage their experiments directly from the main screen.

Finally, the Home page provides a direct link to the global Settings page, which is accessible through a text button at the bottom of the layout. From this Settings view, users can configure application-wide options, such as server endpoints and storage paths, that influence how all training sets behave. By grouping navigation to creation, task monitoring, configuration management, and global settings in one place, the Home page offers a simple and intuitive starting point for both new and experienced users of ML Vision Studio.

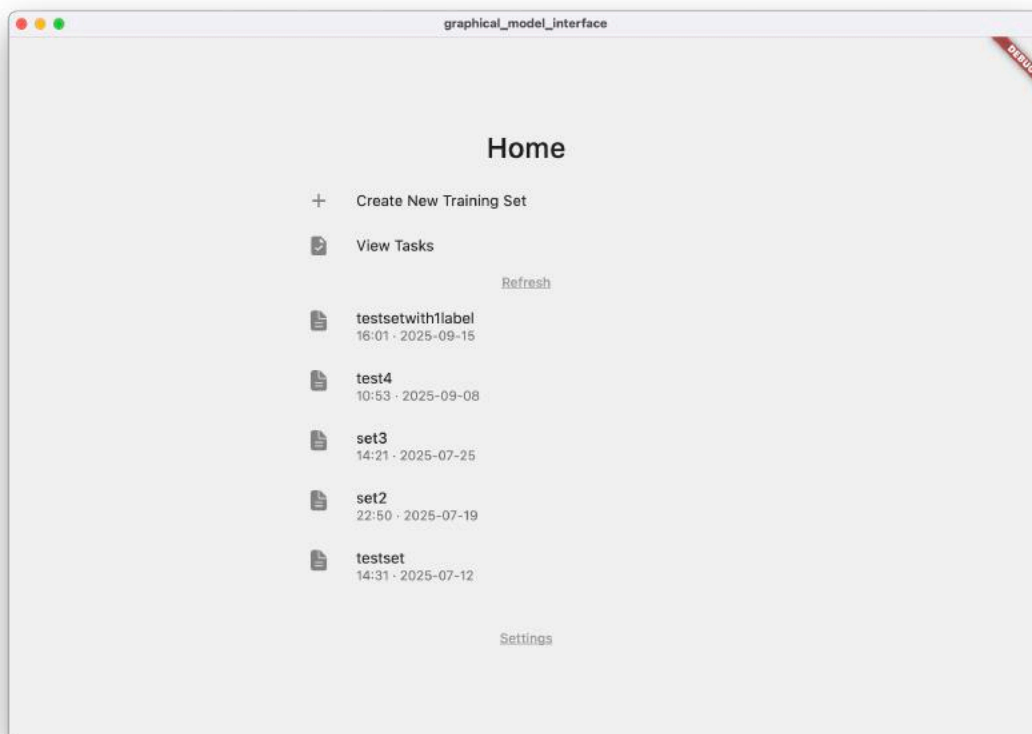


Fig 2.1 Home Page

2.2.2 Create New Page

The Create New page provides a guided environment for constructing a new training set, organized into four clearly indicated steps at the top of the screen. A horizontal stepper shows the current step number and allows movement forward or backward through the sequence, so that users can progress through the workflow in order while still being able to revisit earlier steps if adjustments are needed. All step content is displayed inside a single page view below this stepper, which keeps the layout visually consistent as the user moves through dataset input, model configuration, parameter selection, and data split definition.

A custom app bar at the top of the Create New page frames the entire workflow and exposes two important controls. On the left, a prominent “Discard” button allows the user to cancel the current setup at any time, clearing any labeled image data collected so far and returning directly to the first route of the application, which prevents partially completed configurations from cluttering the training set library. In the center, the title “Create New” is displayed, indicating that the user is currently working within the training-set creation flow. On the right, an error badge is shown, which summarizes any configuration issues detected by the validation service and reminds the user that certain fields may need attention before the training set can be finalized.

The body of the Create New page combines the stepper with a non-scrollable page view that holds all inner pages of the flow. Navigation between steps is controlled exclusively by the stepper’s next and previous actions, which encourages the user to follow the intended order of operations rather than jumping randomly between sections. After the main three configuration steps, the page presents a split configuration view and a final “Setup Complete” screen, where the user assigns a name to the training set and triggers generation of the training zip and configuration files. The “Generate Training Set” button remains disabled until a non-empty name has been provided, enforcing that each saved configuration is labeled meaningfully in the library.

Throughout this process, the Create New page performs background validation of the full configuration whenever the user moves between steps. This validation checks the combined settings for consistency and updates the error badge accordingly, so that potential problems are surfaced early rather than only at export time. By structuring the interface in this way, the Create New page offers a single, coherent surface where users can progress from raw images to a fully

specified and named training set, while being continuously guided by step indicators, clear labels, and high-level validation feedback.

2.2.2.1 Training Set Builder

The first step of the Create New workflow is the Training Set Builder, which allows labels to be defined and images to be organized under those labels in a single, split-screen view. The left side displays a vertical list of all existing labels, each showing its name and the number of images assigned to it and allows a label to be selected for editing or deleted if it is no longer needed. A text field at the bottom of this sidebar is used to add new labels: typing a name and pressing the Enter key immediately creates a label, and the adjacent “add” icon offers the same action as a clickable button. This design permits both keyboard-focused and mouse-focused users to build up the label set quickly.

The right side of the Training Set Builder displays the images belonging to the currently selected label. If no label has been chosen yet, a simple message prompts the user to add or select a label before proceeding. Once a label is selected, its images are shown in a responsive grid layout, where each item corresponds to a single image that has been added to that label. Each grid item includes controls to remove the image from the training set, so that mislabeled or unwanted images can be corrected directly within this view without leaving the workflow.

At the end of the grid, an additional tile is reserved for adding new images to the selected label. This tile appears visually distinct, with an icon and caption such as “Add Images”, and opens a file picker when clicked so that multiple images can be imported at once from the local file system. After selection, a progress dialog is shown in the center of the screen, displaying a text message (“Processing Images...”), a progress bar, and a dynamic status line that updates as images are processed. This dialog prevents accidental interaction during processing and provides feedback about the ongoing operation, which is important when large batches of images are being resized, cached, or otherwise prepared for training.

The Training Set Builder also incorporates memory-aware behavior that is transparent to the user but improves responsiveness on large datasets. When a different label is selected, images belonging to non-selected labels are unloaded from memory unless the user has explicitly chosen

to keep all images in memory in the settings. Similarly, individual images are loaded into memory only as they are needed for display and are released shortly after rendering, which makes the interface usable even when hundreds of images per label are present. Through this combination of keyboard shortcuts, clickable controls, progress feedback, and background optimizations, the Training Set Builder provides a practical, interactive environment for constructing clean, labelled image datasets as the foundation for later training steps.

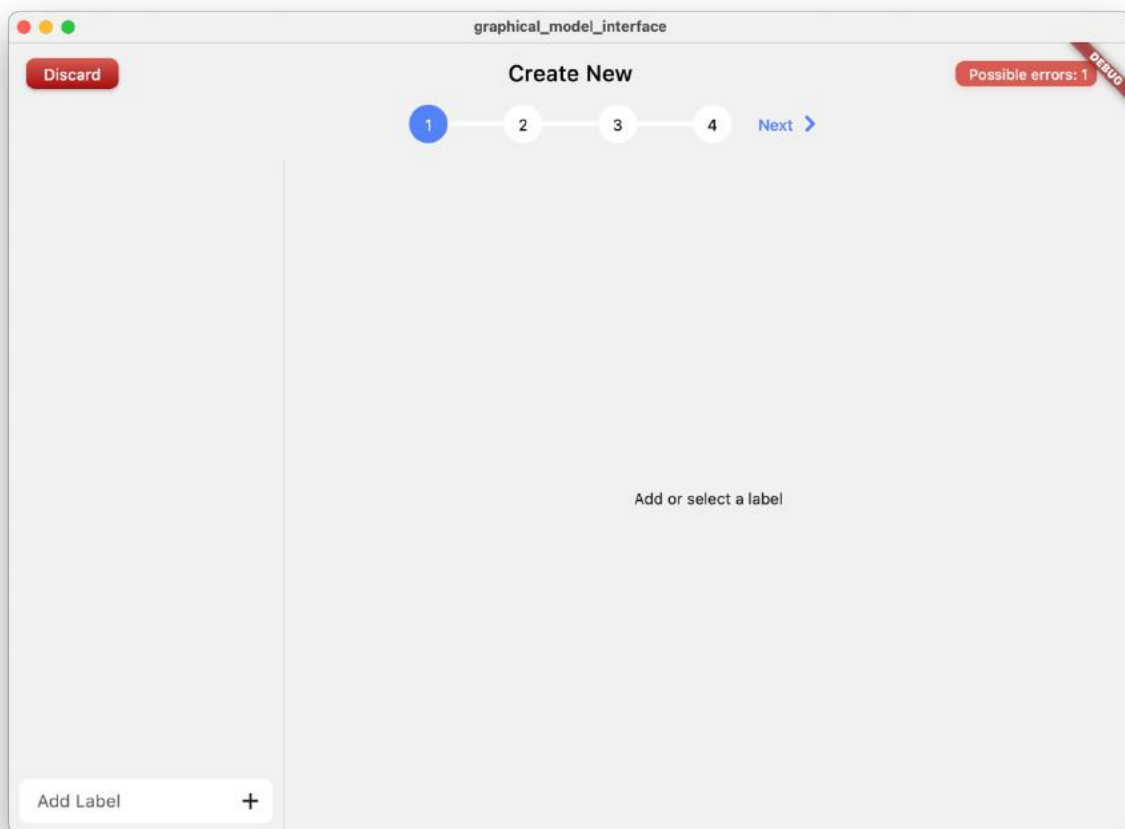


Fig 2.2 Step 1: Training Set Builder of The Create New Page

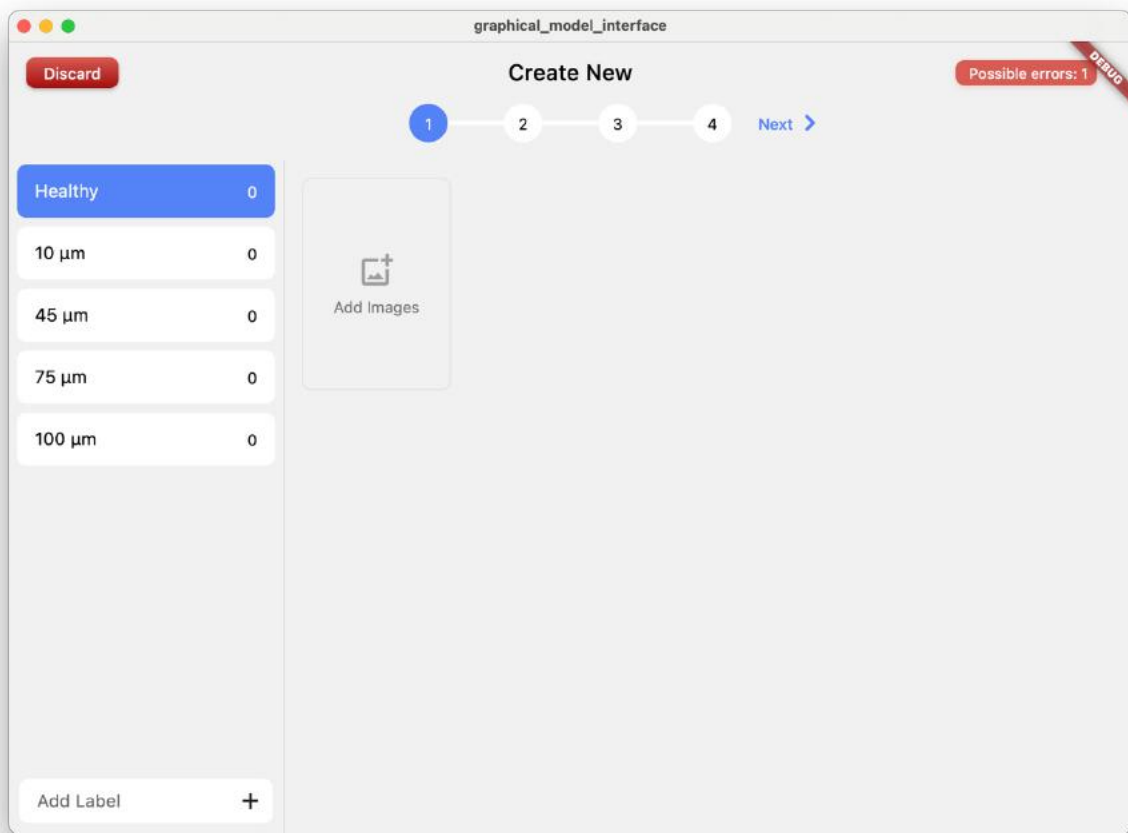


Fig 2.3 List of empty classes in the Training Set Builder

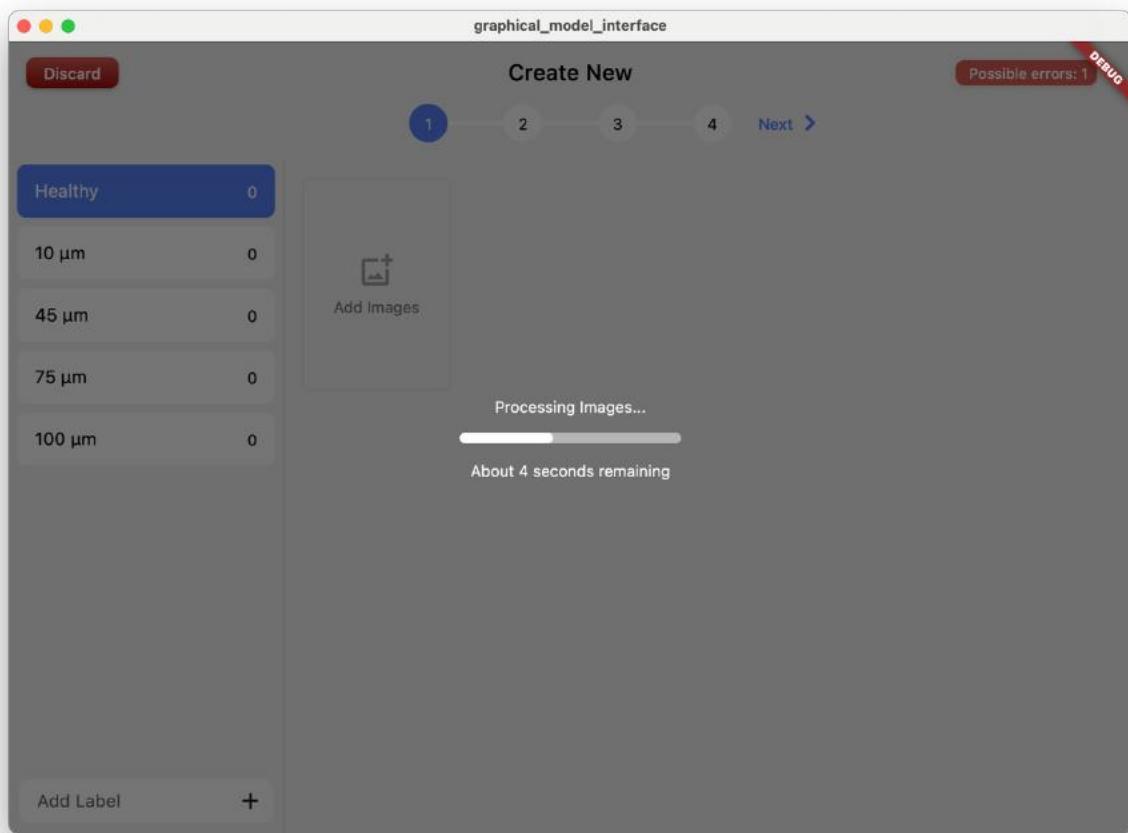


Fig 2.4 New batch of images being processed and added to a class

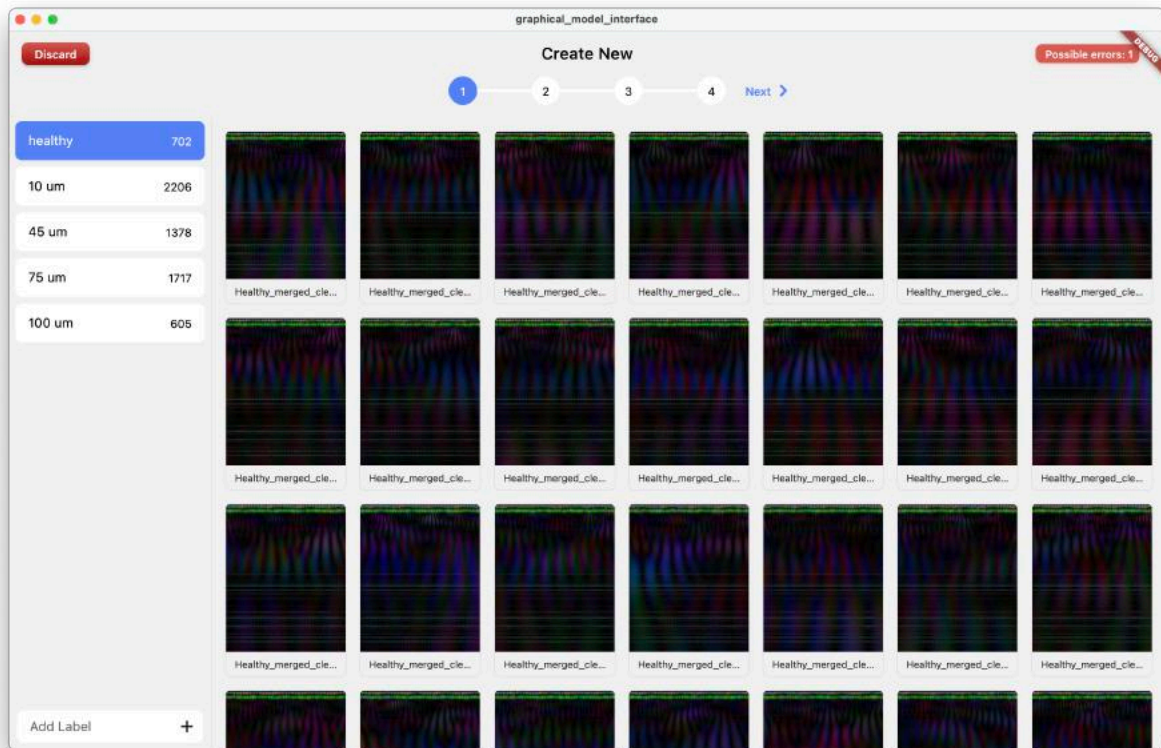


Fig 2.5 All classes populated with images

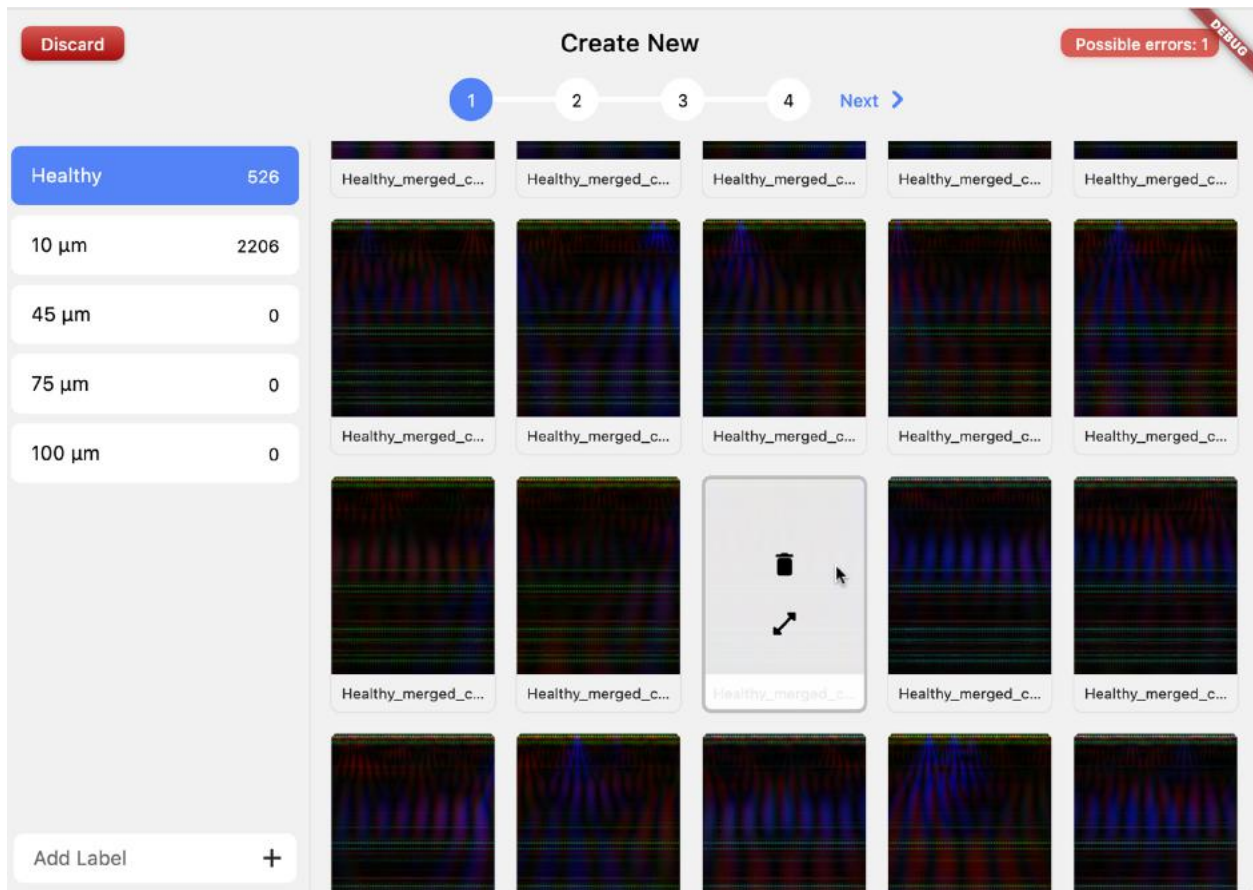


Fig 2.6 Image options revealed on mouse hover

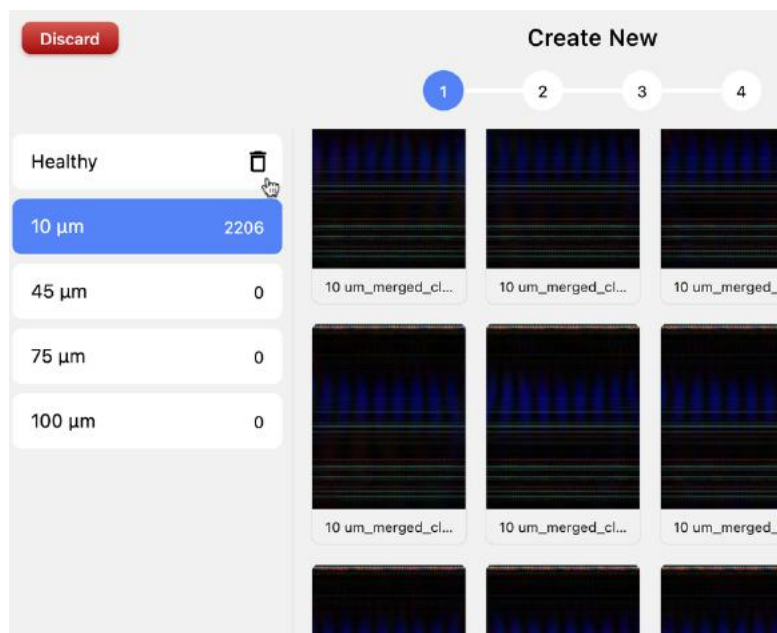


Fig 2.7 Class delete option revealed on mouse hover which temporarily hides the class size

2.2.2.2 Layer Configuration Page

The Layer Configuration Page provides an interactive, visual editor for the model head that will sit on top of the chosen base CNN (for example, EfficientNet or ResNet). It is laid out as a two-panel view: on the left, the overall layer sequence is shown as a vertical diagram, and on the right, the parameters of the currently selected layer are displayed in a form. A dotted background is drawn behind the content, giving the impression of a design canvas on which the layer blocks are placed. At the top of the left panel, the fixed input block represents the base model and cannot be reordered, which makes it clear where the custom head begins.

Below the input block, all intermediate (head) layers are displayed as individual boxes arranged vertically, connected by lines to suggest the flow of data from top to bottom. These boxes are placed inside a reorderable list, so that users can change the order of layers using drag-and-drop, via a drag handle shown next to each layer. The dragged item is rendered with a subtle shadow, which gives visual feedback about the reordering operation. An “Add Layer” button is placed at the bottom of this column, allowing new layers to be appended to the sequence; when clicked, it opens a small panel where the type of the new layer (such as Dense or Dropout) can be chosen and then added to the configuration.

The right-hand panel acts as a property inspector for the selected layer. When the input block is selected, the panel shows input-related settings such as the input image shape; when the final output layer is selected, the panel shows output-related settings such as the number of classes; and when any intermediate layer is selected, the panel displays the parameters relevant to that layer type, such as units, activation, or dropout rate. All fields are presented as simple text inputs, toggles, or dropdowns, with validation of numeric entries and tooltips or hints where appropriate. If no layer is selected, the panel displays a neutral message prompting the user to select a layer to view its parameters, which keeps the interface self-explanatory.

This page therefore allows users to design the model head visually by combining three actions: selecting the base model (on a different step), arranging a sequence of head layers in the desired order using drag-and-drop, and editing each layer’s parameters in a structured form. The combination of a schematic layer diagram and a detailed parameter panel makes it possible to

construct non-trivial architectures without writing code, while still exposing enough detail to match the flexibility of typical deep-learning scripts.

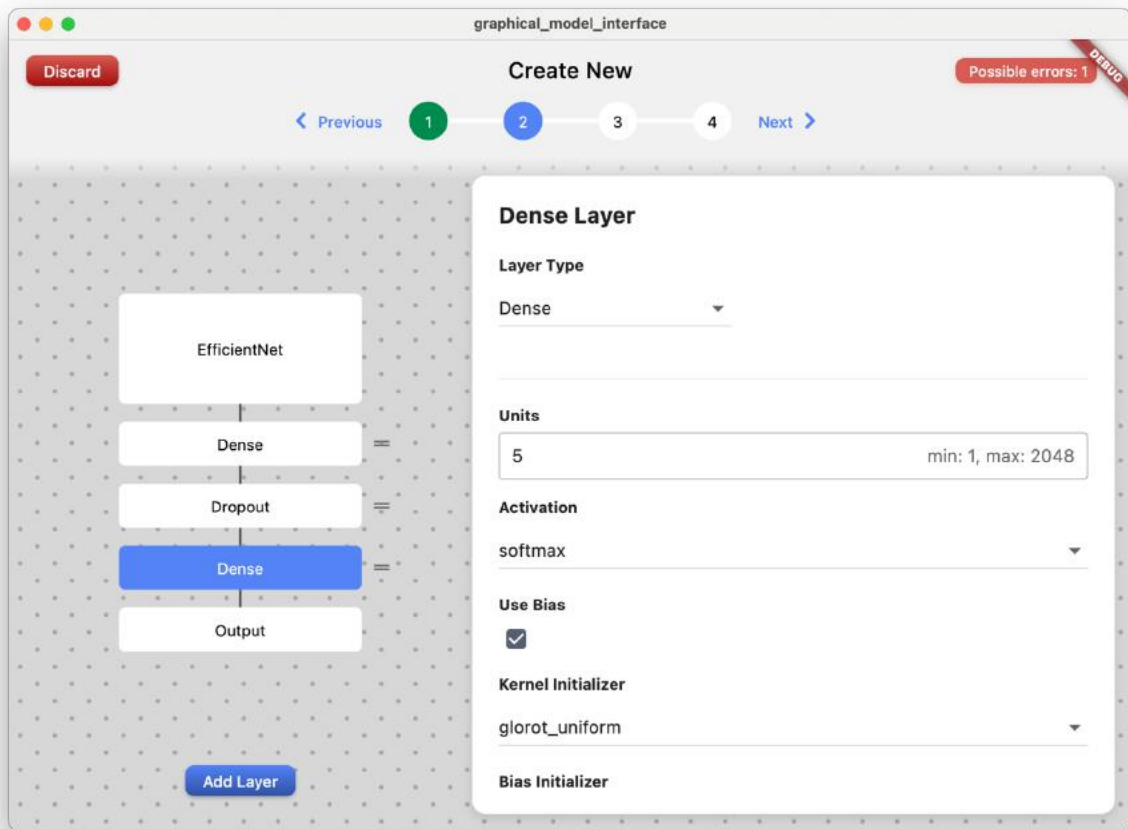


Fig 2.8 Step 2: Layers Configuration Page

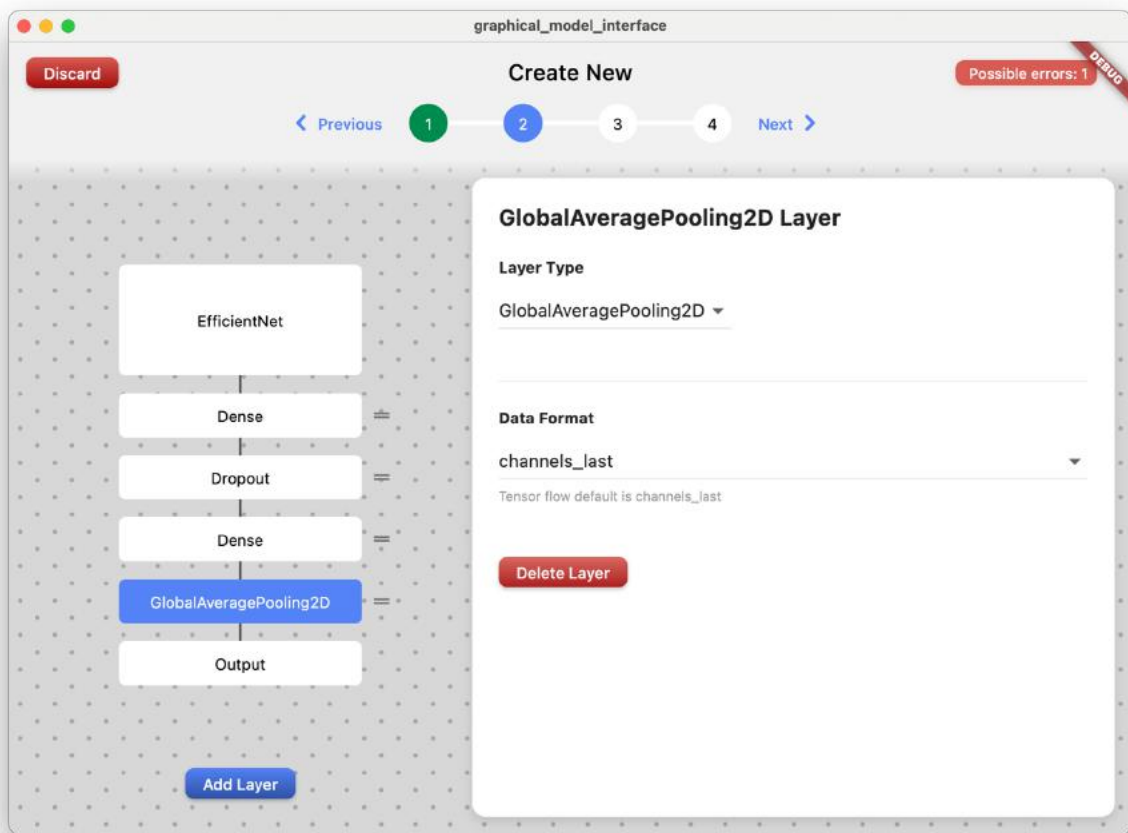


Fig 2.9 Adding and deleting a layer

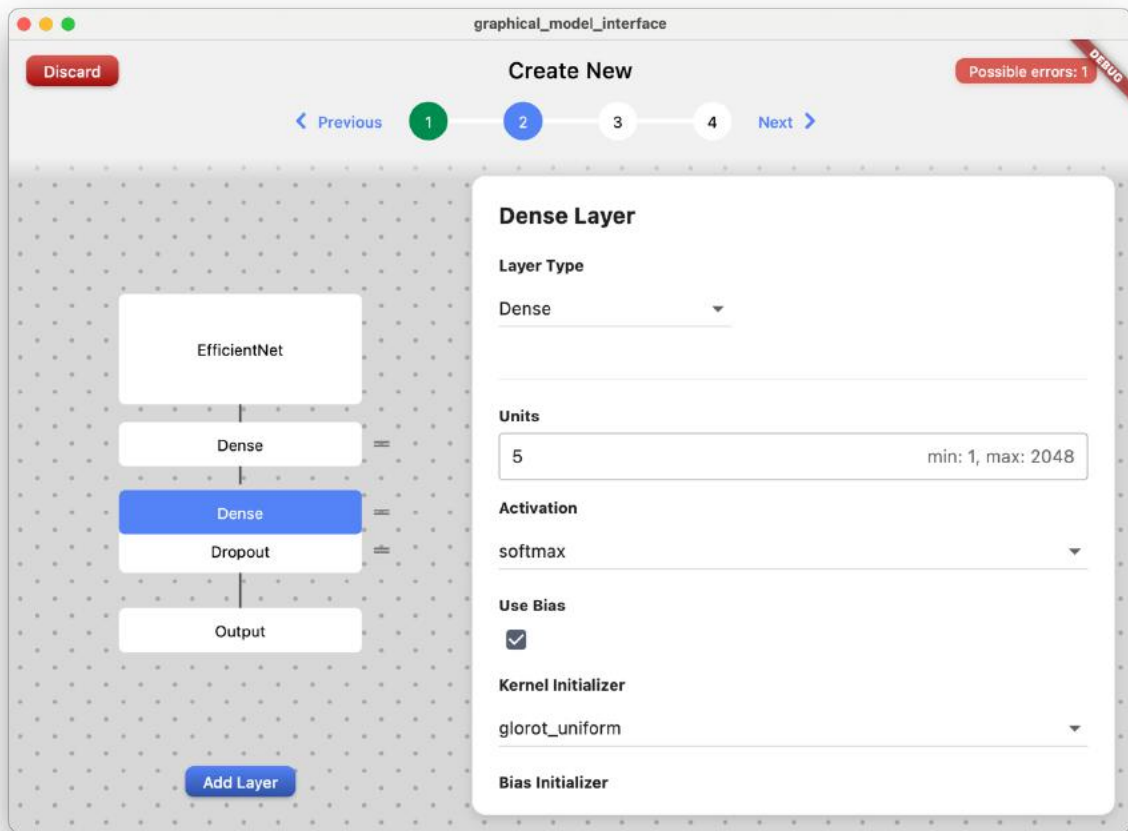


Fig 2.10 Drag and drop reordering of layers

2.2.2.3 Parameters Configuration Page

The Parameters Configuration Page presents all training options in a single scrollable view that behaves like a responsive “control panel” for the model. The main content is laid out as a masonry-style grid of parameter cards: a layout calculation determines how many cards can fit per row based on the current window width (between one and five), and then arranges them in a staggered grid so that wide screens show several compact groups side by side while smaller windows fall back to fewer, wider columns. Each top-level entry under the parameters section of the configuration (such as augmentation settings, optimizer, learning-rate schedule, early stopping, transfer learning, class balancing, metrics, training settings, hardware, checkpointing, and research compliance) is turned into a titled card, with the title derived automatically from the underlying key by splitting on

underscores and capitalizing words, which keeps the layout readable without duplicating labels manually.

Within these cards, individual controls are generated automatically from a separate options description, rather than being hard-coded into the page. For every parameter key, this options description specifies the variable type (integer, floating point, boolean), recommended minimum and maximum values, the smallest step size, default values, hints, and the preferred control style (for example, slider, toggle, single-choice selector, or multi-selector). When the underlying value is a simple number, the page creates a slider element that combines a custom-styled slider with a numeric input field; the slider reads its range, step, and integer/decimal behavior from the metadata and snaps to appropriate values, while the knob is drawn with a 3D gradient thumb whose colors indicate whether the current value is inside or outside the recommended range (blue tones when in range, switching to red tones when the value moves beyond the configured limits). This design allows the user to deliberately push parameters slightly beyond typical bounds for experimentation, while still receiving a clear visual warning through the thumb color.

For parameters that are themselves structured groups (for example, `augmentation_settings`, `optimizer_details`, `transfer_learning_options`, or `training_settings`), the page builds a “value list” card where each row corresponds to one child setting. Each row uses the metadata to choose a suitable control: boolean options are shown as compact switches or checkboxes; categorical options that list allowed values and specify a selector style are represented by selector components that display each choice as a clickable pill within a wrapping layout; and some advanced sets (such as secondary metrics) can be controlled by a multi-selector, which shows a pill-like row with a checkbox and label for each possible option, allowing multiple selections to be toggled on or off independently. Hints from the options description, such as recommended values from prior experiments or from a research paper, can be displayed as helper text within these cards, so that each control carries its own contextual guidance. Because all of this is driven from the configuration metadata, adding a new parameter or changing its constraints can be done by updating the configuration description without modifying the user-interface code, which makes the system adaptable to new models and research settings.

The page tracks edits using an unsaved-changes indicator that appears in the top row. An internal set remembers which fields have been modified but not yet “committed,” and when at least one

such field exists, the top-left of the header displays an information icon and a short “unsaved changes” label. When the mouse hovers over this label, a small dark popover appears, explaining that the user should confirm each edited field (for example, by pressing Enter in a text box) to save it into the main configuration, and disappears when the pointer leaves the area. As the user drags sliders, types into numeric fields, or changes selectors, the associated keys are marked as unsaved; once the user confirms the value, the page writes it back into the central configuration map, removes it from the unsaved set, and triggers a background validation run that checks for inconsistent or risky parameter combinations. In this way, the page encourages deliberate confirmation of changes and surfaces potential configuration issues early, without interrupting the workflow with modal dialogs.

On the right side of the header row, an Import button is provided to load a previous configuration from the local library. Activating this button opens a dialog that lists all stored configurations, each shown with a human-readable name (taken from the saved configuration’s own name when available) and a formatted creation date and time, selectable via radio buttons. The first available configuration is pre-selected when the dialog opens so that a valid choice is always available; when a different configuration is selected, a descriptive message beneath the list explains that importing will replace the entire current configuration, including the model’s layer definitions, but will not load the original dataset associated with that configuration. Confirming the import replaces the active configuration with the chosen one (preserving the current name if necessary) and closes the dialog, leaving the parameter grid immediately updated to reflect the imported values. Together, the responsive masonry layout, metadata-driven control generation, rich selector components, permissive but visually guided sliders, unsaved-change indicators with hover help, and configuration-import workflow make the Parameters Configuration Page a dense but navigable control surface where complex training setups can be adjusted safely and transparently without editing JSON or code.

graphical_model_interface

Create New

Possible errors: 1

Previous

1

2

3

4

Next

Parameters

Import

Learning Rate

0.0001

Epochs

20

Batch Size

8

Lr Schedule

Type: ReduceLROnPlateau

Decay Factor: 0.905

Decay After Epochs: 1

Min Lr: 1e-7

Patience: 1

Monitor: val_loss

Mode: min

Initial Patience: 10

Constant For First Epochs: 10

Early Stopping

Enabled: ☒

Patience: 25

Monitor Metric: val_loss

Preprocessing Options

Normalization: custom

Resize Width: 300

Resize Height: 200

Hardware

Gpu Memory Growth: ☒

Mixed Precision: ☒

Distributed Training: ☐

Device: auto

auto cpu gpu

Gpu Device Id: 0

Regularization

Dropout Rate: 0.45

Weight Decay: null

Advanced Settings

Custom Callbacks: [lr_schedule]

Inference Settings: {test_on_sa}

Class Balancing

Enabled: ☒

Method: oversampling

oversampling undersampling

class_weights none

Target Samples Per Class: 1103

Strategy: random

random smote adasyn

Augmentation Settings

Rescale Factor: 0.0039215686

Rotation Range: 9

Width Shift Range: 0.1

Height Shift Range: 0.05

Zoom Range: 0.1

Horizontal Flip: ☒

Vertical Flip: ☒

Fill Mode: reflect

reflect wrap constant

Optimizer

Type: Adam

Adam SGD RMSprop

Adagrad Adadelata

Beta1: 0.9

Beta2: 0.999

Epsilon: 1e-7

Transfer Learning

Freeze Base Layers: ☒

Fine Tuning Epochs: 10

Fine Tuning Lr: 0.00001

Unfreeze At Epoch: 30

Gradual Unfreezing: ☐

Metrics

Primary: accuracy

accuracy precision

recall f1_score

Secondary: [top_3_accuracy, a]

Macro Average: ☒

Save Confusion Matrix: ☒

Training Settings

Steps Per Epoch: null

Validation Steps: null

Shuffle: ☒

Seed: 42

Use Multiprocessing: ☐

Workers: 1

Max Queue Size: 10

Checkpointing

Save Best Only: ☒

Save Weights Only: ☐

Monitor Metric: val_accuracy

val_accuracy val_loss

Fig 2.11 Parameters Configuration Page it its widest layout

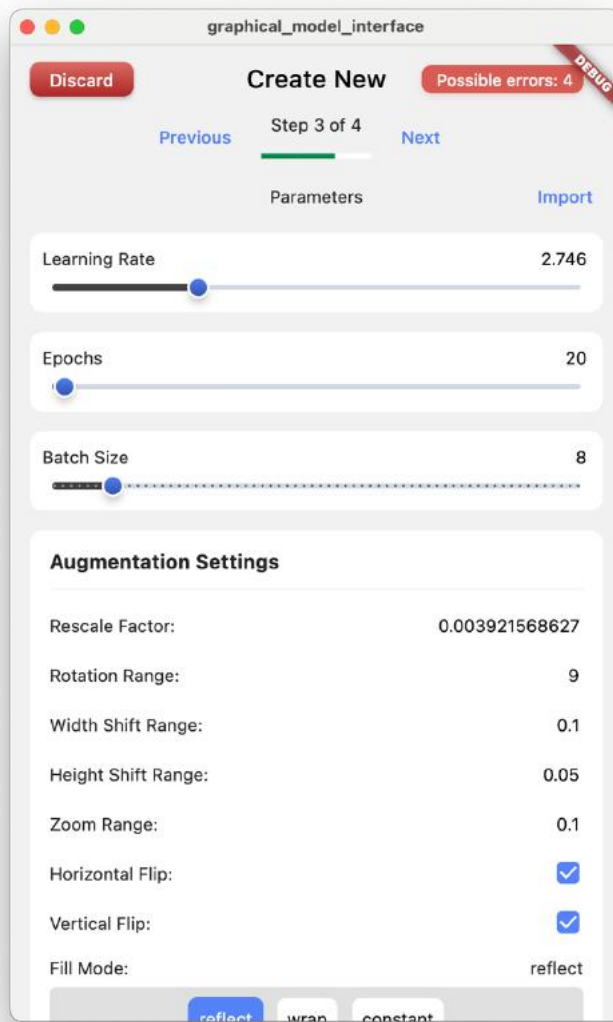


Fig 2.12 Parameters Configuration Page in its narrowest layout

graphical_model_interface

Discard

Create New

Possible errors: 1

Previous

1

2

3

4

Next

Parameters

Import

Unsaved Changes

Learning Rate

0.0001

Lr Schedule

Type: ReduceLROnPlateau

Decay Factor: 0.905

Decay After Epochs: 1

Min Lr: 1e-7

Patience: 1

Monitor: val_loss

Mode: min

Initial Patience: 10

Constant For First Epochs: 10

Hardware

Gpu Memory Growth: ☒

Mixed Precision: ☒

Distributed Training: ☐

Device: auto

auto cpu gpu

Gpu Device Id: 0

Epochs

40

Early Stopping

Enabled: ☒

Patience: 20

Monitor Metric: val_loss

Preprocessing Options

Normalization: custom

Resize Width: 300

Resize Height: 200

Metrics

Primary: accuracy

accuracy precision

recall f1_score

Secondary: [top_3_accuracy, a

Macro Average: ☒

Save Confusion Matrix: ☒

Batch Size

8

Regularization

Dropout Rate: 0.45

Weight Decay: null

Advanced Settings

Custom Callbacks: [lr_schedule

Inference Settings: {test_on_sa

Class Balancing

Enabled: ☒

Method: oversampling

oversampling

undersampling

class_weights none

Target Samples Per Class: 1103

Strategy: random

random smote

adasyn

Augmentation Settings

Rescale Factor: 0.0039215686:

Rotation Range: 9

Width Shift Range: 0.1

Height Shift Range: 0.05

Zoom Range: 0.2

Horizontal Flip: ☒

Vertical Flip: ☒

Fill Mode: reflect

reflect wrap

constant

Optimizer

Type: Adam

Adam SGD RMSprop

Adagrad Adadelata

Beta1: 0.9

Beta2: 0.999

Epsilon: 1e-7

Transfer Learning

Freeze Base Layers: ☒

Fine Tuning Epochs: 10

Fine Tuning Lr: 0.00001

Unfreeze At Epoch: 30

Gradual Unfreezing: ☐

Training Settings

Steps Per Epoch: null

Validation Steps: null

Shuffle: ☒

Seed: 42

Use Multiprocessing: ☐

Workers: 1

Max Queue Size: 10

Checkpointing

Save Best Only: ☒

Save Weights Only: ☐

Monitor Metric: val_accuracy

val_accuracy val_loss

Fig 2.13 Parameters Configuration Page with modified unsaved elements

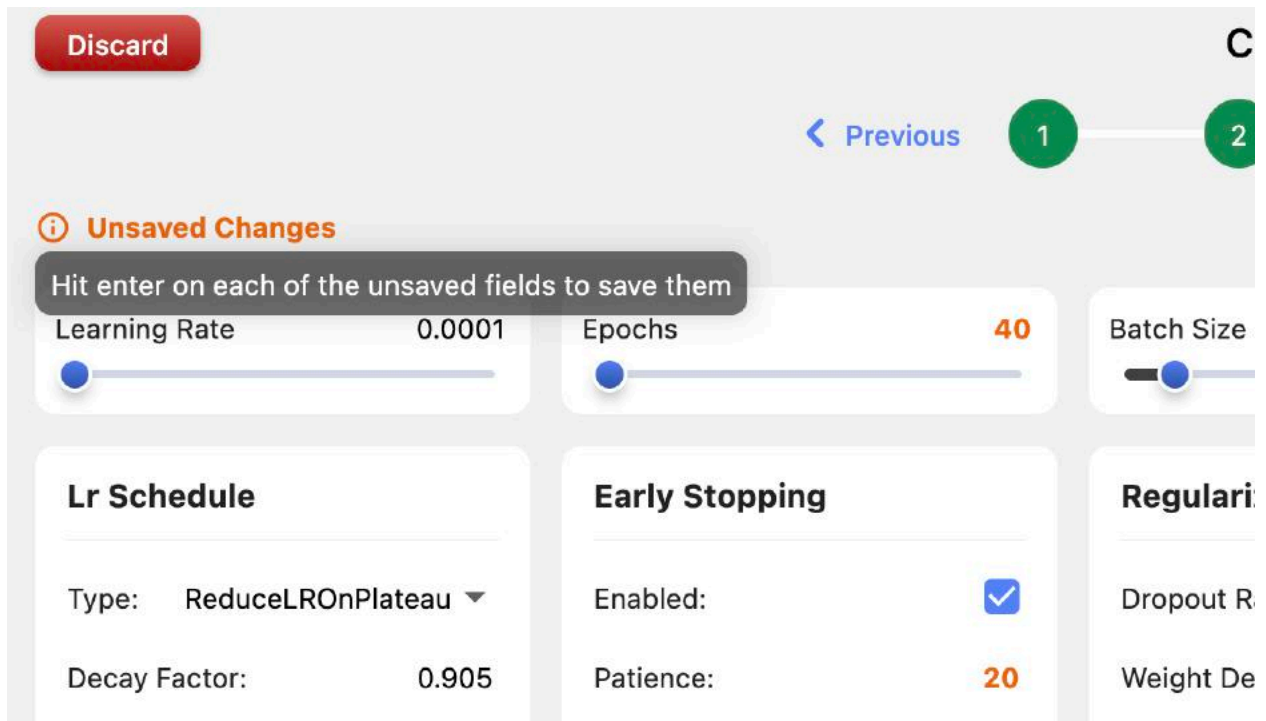


Fig 2.14 Tool Tip for Unsaved Changes that appears on mouse hover

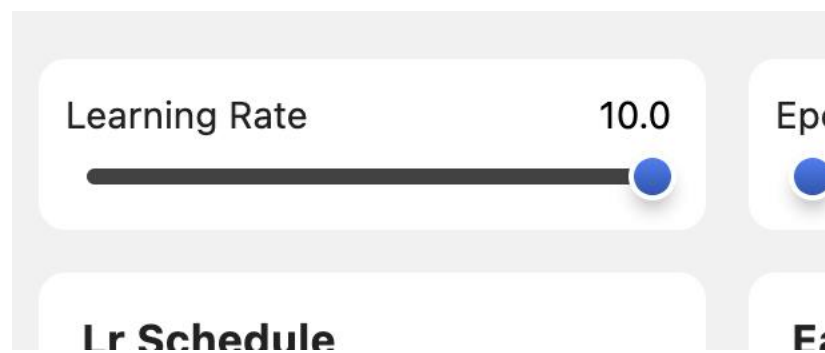


Fig 2.15 A slider at maximum value

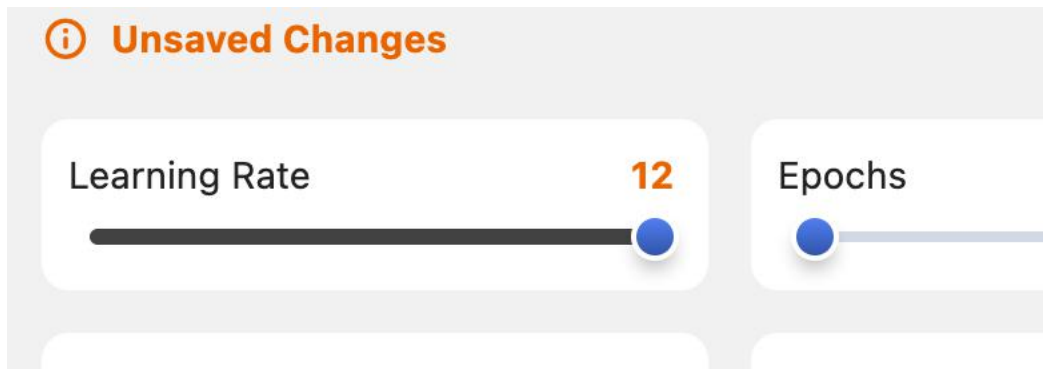


Fig 2.16 A value typed into the slider element that exceeds its maximum limit

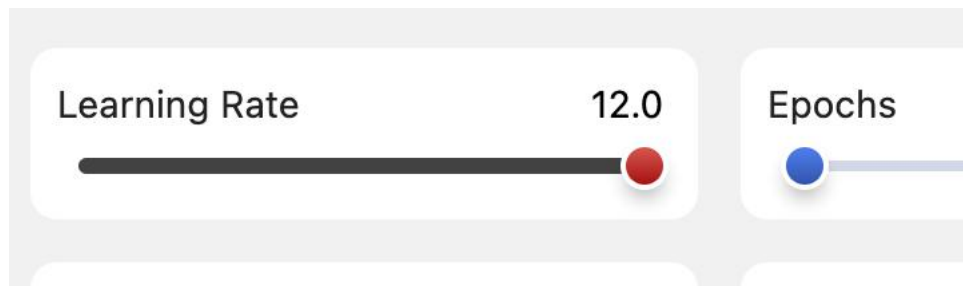


Fig 2.17 Slider knob turns red to indicate a non-recommended saved value

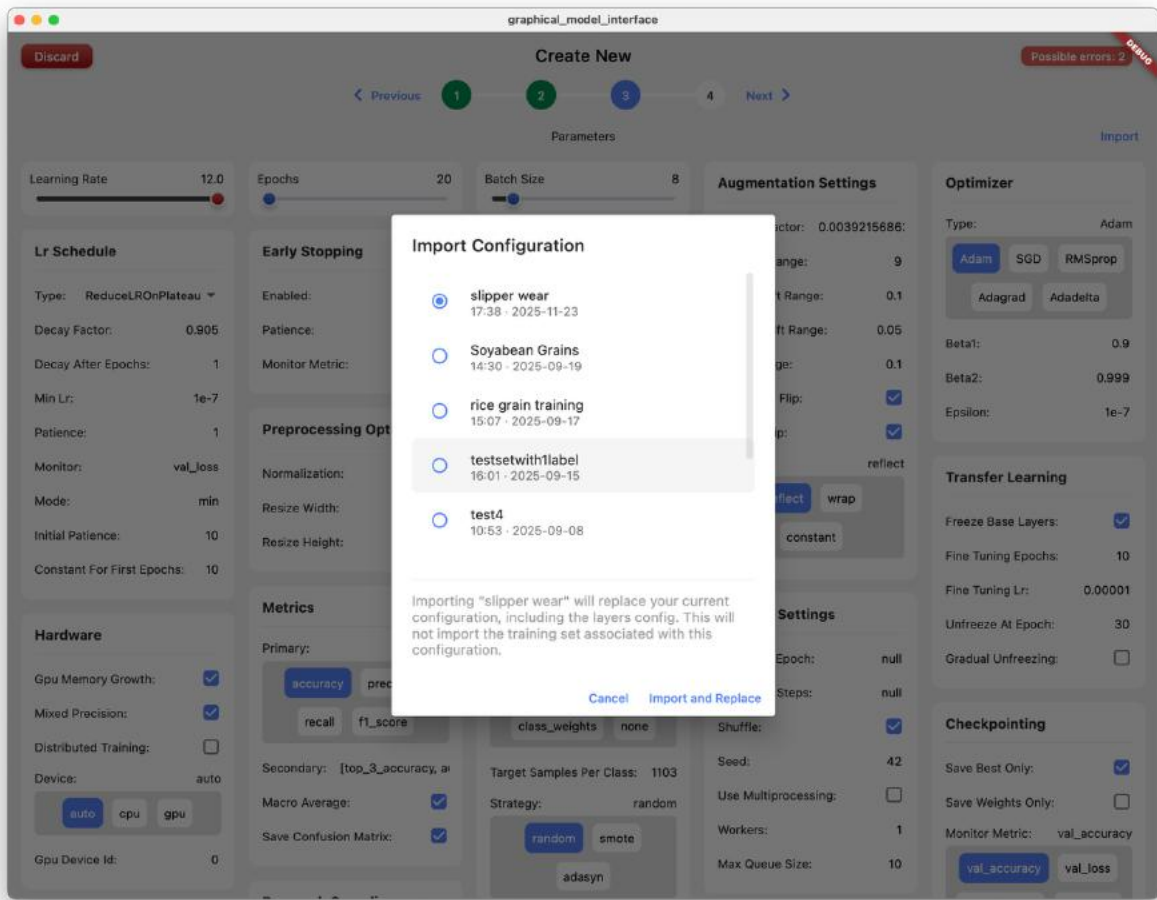


Fig 2.18 Import Utility

2.2.2.4 Split Selection Page

The Split Selection Page presents a focused view for configuring how the dataset is divided into training, cross-validation, and testing subsets, using two interactive split sliders and a shuffle option. The layout is centered with symmetric horizontal padding that scales with screen width, so that the controls remain visually balanced on different display sizes. At the top, a “Split” heading is shown, followed by a first SplitSlider that represents the ratio between the training set and the combined “Testing + Cross Validation” set; this slider is drawn as a horizontal bar split into two colored segments, with a small draggable handle between them. As the user drags horizontally, the cursor changes to a column-resize cursor, and the relative widths of the two segments update in real time, while the numeric percentages inside each colored block are updated accordingly.

Each percentage label inside the SplitSlider is directly editable: clicking on the text turns it into a centered text field with a text cursor, and the user can type an exact percentage for either side. When Enter is pressed or when the user clicks outside the field, the value is validated to ensure it lies between 0 and 100, after which the bar is recomputed so that both sides still add up to 100%, and the label returns to a non-editing display state. This behavior allows precise numerical control for users who prefer typing, while still supporting quick, coarse adjustment via dragging. A second SplitSlider, styled with amber and dark-amber colors, then subdivides the “Testing + Cross Validation” portion into separate “Cross Validation” and “Testing” percentages using the same interaction model of dragging and direct numeric entry. Labels under each slider (“Train” vs “Testing + Cross Validation”, and “Cross Validation” vs “Testing”) clearly indicate what each side of the bar corresponds to.

At the bottom of the page, a compact row contains a checkbox labeled “Shuffle Data”, which controls whether the samples are shuffled before the split is applied. Toggling this checkbox updates the split configuration in the underlying full configuration map so that downstream training respects the user’s choice. By combining responsive layout, drag-based adjustment, direct percentage input with a text cursor, and an explicit shuffle toggle, the Split Selection Page provides a clear and intuitive interface for defining complex train/validation/test splits without requiring the user to reason about raw fractions or edit configuration files manually.

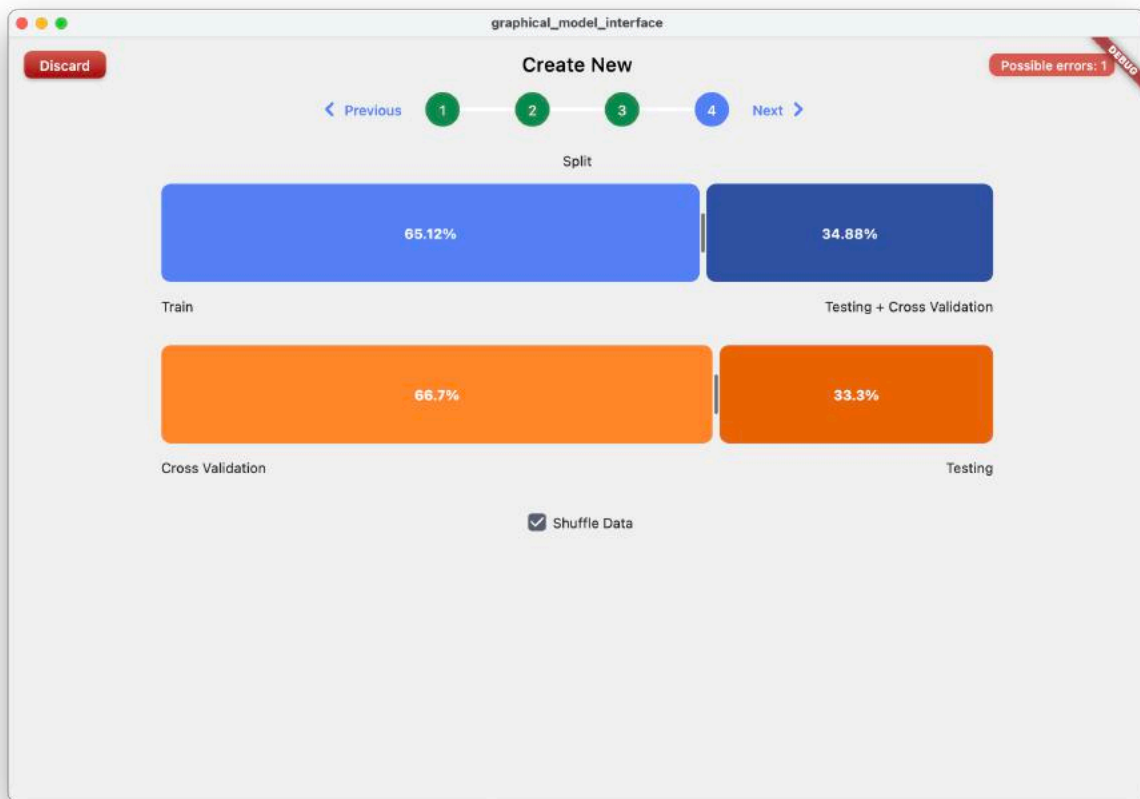


Fig 2.19 Split Page

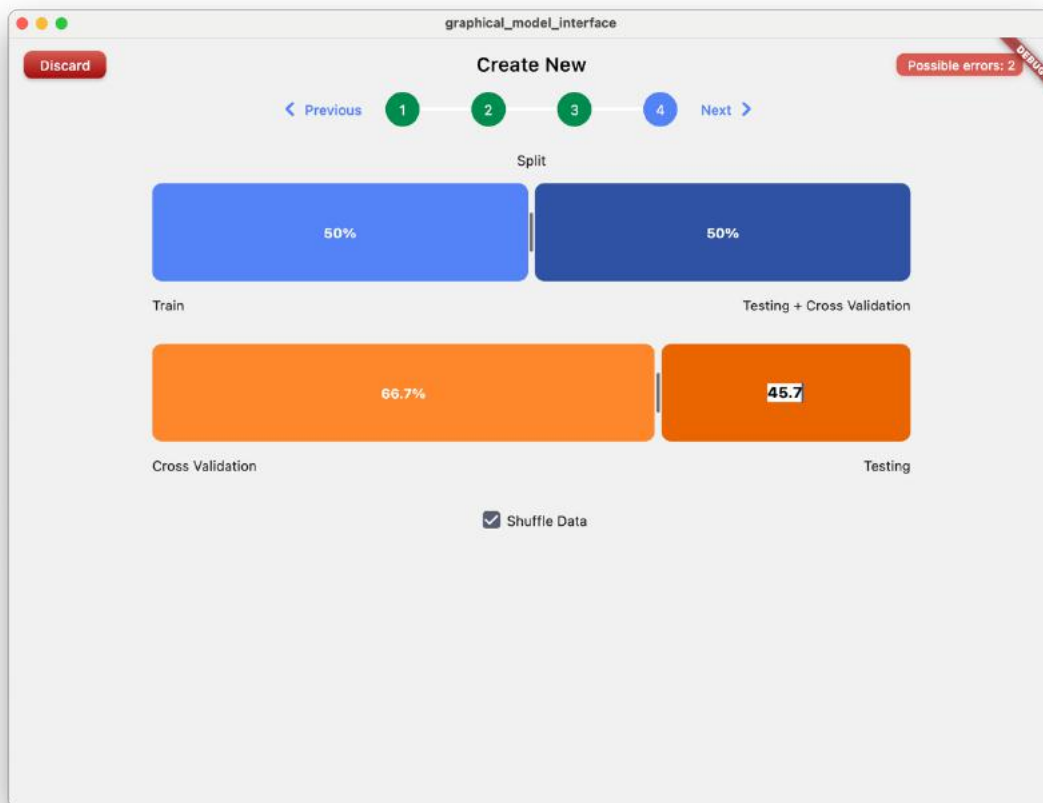


Fig 2.20 Split Page with Text Input

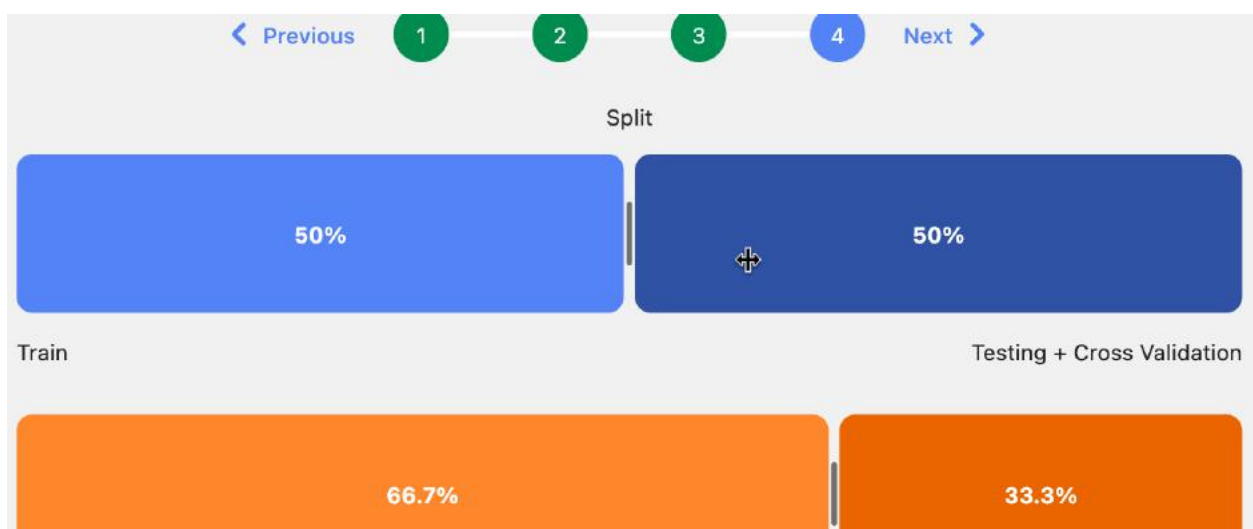


Fig 2.21 Slider Mouse Cursor style while hovering on slider

2.2.2.5 Setup Complete Page

The Setup Complete Page serves as the final confirmation step of the Create New workflow, summarizing that all previous configuration stages have been completed and focusing the user on naming and generating the training set. At the top, a “Setup Complete” message is displayed in bold with a green accent color, visually indicating success and readiness to proceed. Below this, a single, centered text field of fixed width is provided, where the user can type a descriptive name for the training set; the field shows a hint text (“Name your training set”) and a helper message reminding the user to press Enter when finished, and any edits are reflected immediately in the internal configuration so that the chosen name is stored with the model. After a short vertical gap, a prominent “Generate Training Set” gradient button is presented; this button remains disabled until a non-empty name is present in both the text field and the configuration, preventing accidental generation of unnamed or ambiguously named training sets. When the button is active and clicked, the page navigates to the export screen, where the dataset zip and configuration file are actually created and can be saved or sent, thereby completing the creation pipeline that was introduced in the mid-evaluation presentation as “Name training set and export as zip”.

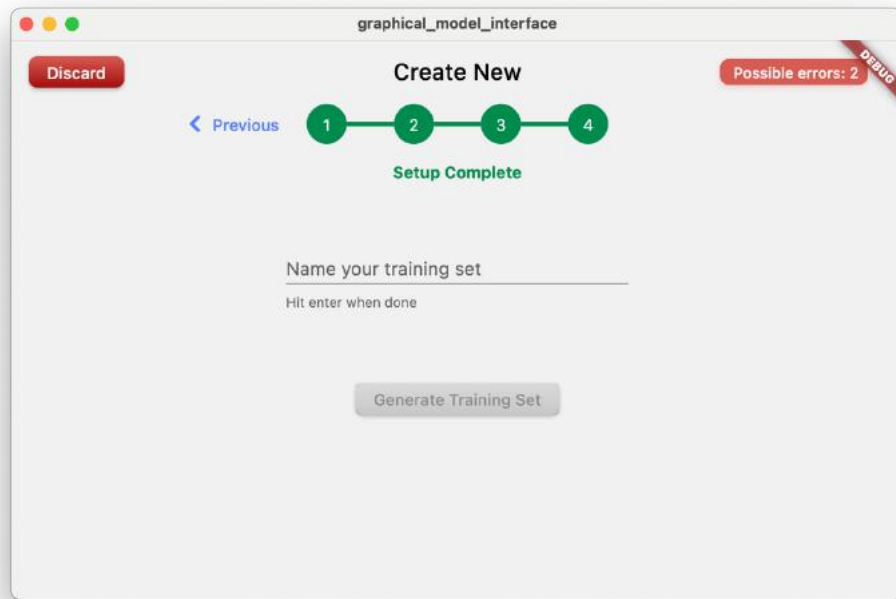


Fig 2.22 Setup Complete Page

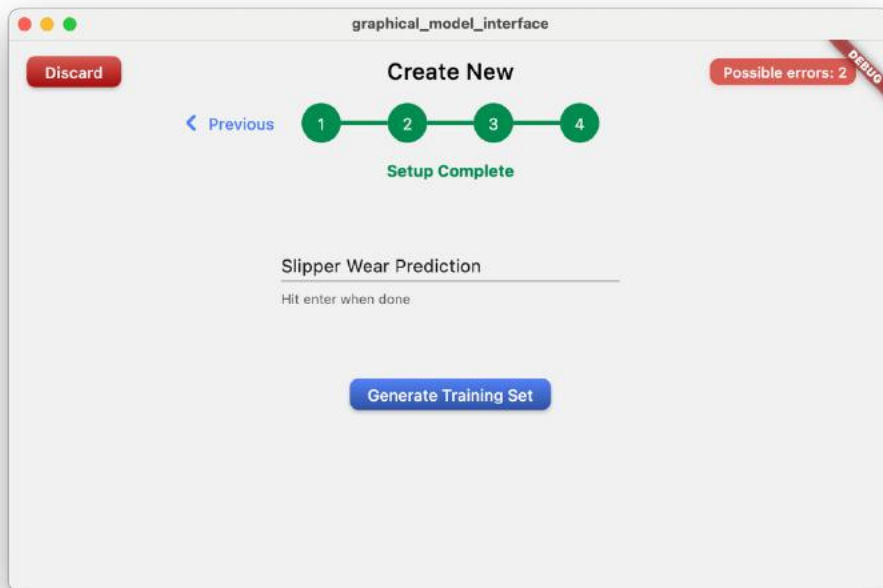


Fig 2.23 Setup Complete Page with a valid name

2.2.2.6 Error Checking Service

The error checking service is designed as a continuous background validation system that monitors the complete training configuration while a new model setup is being created. Its goal is to detect problematic or inefficient choices early, summarize them in a structured way, and surface them through a compact interface element so that users are guided, rather than blocked, as they move through the four configuration steps. The goal is to warn the user, while still giving them complete control on their configuration.

At the core of this service, a single shared validator object is used for the entire application session, so that all pages operate on one consistent view of configuration health. When validation is run, an internal list of issues is rebuilt; each issue is stored as a structured record containing a broad category (such as architecture, training, data, hardware, or system), the specific configuration item involved, a human-readable message, a severity level (error, warning, or informational suggestion), and an optional suggestion that describes a concrete fix. This design ensures that each problem is described in clear language and grouped meaningfully, which assists users in understanding not only what is wrong but also where in the configuration it is located and how it can be improved.

A dedicated background validation service is used to monitor the global configuration map that aggregates choices from all the Create New steps (labels, layers, parameters, and splits). At regular intervals, this service compares the current configuration snapshot to the last one that was validated; if any change is detected, a short delay is introduced and then a full validation pass is triggered, so that rapid edits are grouped and the user interface is not overwhelmed by constant re-checks. This strategy ensures that validation follows the user's work naturally in the background and that any new inconsistency is detected shortly after it is introduced, without explicit action or manual "run checks" buttons.

During each validation pass, several families of checks are applied to the configuration so that a broad range of potential problems is covered. The model architecture is first examined: the presence and support of the chosen base model are confirmed, the input shape is inspected for common image mistakes (such as non-RGB channel counts or unrealistically small spatial

dimensions), model-specific constraints such as InceptionV3's minimum input size are enforced, and the head section is checked to ensure that at least one classification layer is present.

Layer-to-layer compatibility is then examined to catch structural conflicts that would cause runtime errors or meaningless architectures. The relationship between the base model's pooling setting and the first head layer is checked to avoid redundant or missing pooling, and the sequence of head layers is scanned for patterns that are known to be problematic, such as duplicated global pooling layers, global pooling applied after flattening, or convolutional layers being placed after dense layers. Finally, the last dense layer is compared to the number of labels in the dataset so that mismatches between output units and class count are surfaced as explicit errors, with suggestions to align them. In addition to these architectural checks, further passes are performed over training parameters, data configuration, hardware requirements, and research-compliance flags, and any issues in those domains are also recorded with appropriate severities and suggestions.

The Create New page initiates validation explicitly when it is first opened and again whenever the user moves between steps in the wizard, in addition to the background checks, so that major navigation points are always accompanied by an up-to-date assessment of configuration health. The validator's results are not used to hard-block step transitions; instead, they are collected and made visible through a compact badge in the app bar on the right, alongside the stepper and main title, keeping the primary workflow uncluttered. This integration ensures that error feedback is always available but never intrusive, which aligns with the goal of enabling non-expert users to explore advanced configurations without being confronted with modal errors at every change.

The `ErrorBadge` component is used as the main user-facing indicator of validation status and is kept in sync with the validator through a listener mechanism. When no issues are present, the badge area remains empty and does not affect the layout; once any errors, warnings, or suggestions exist, a small rounded pill is shown that displays the total number of "possible errors" and is coloured according to the highest severity currently present (blue when only suggestions exist, amber when warnings exist, and red when at least one true error is present).

When the badge is clicked, a detailed dialog is opened that summarises the situation and lists each individual issue. At the top of this dialog, a compact summary row is displayed with icons and counts for errors, warnings, and suggestions, which allows users to understand at a glance whether blocking problems exist or only optimisations are being proposed. Below this, a scrollable list is

shown where each entry presents the issue's category in a coloured title, the affected configuration field, a clear explanation of the problem, and, when available, a short suggestion line that describes how the configuration can be changed to resolve it; colours again reflect severity so that critical items stand out visually. Through this layered feedback—badge, summary, and detailed messages—the error checking service provides continuous guidance during configuration, reduces the chance of sending invalid jobs to the backend, and makes complex deep-learning design rules approachable to users who may not have prior expertise in model architecture design.

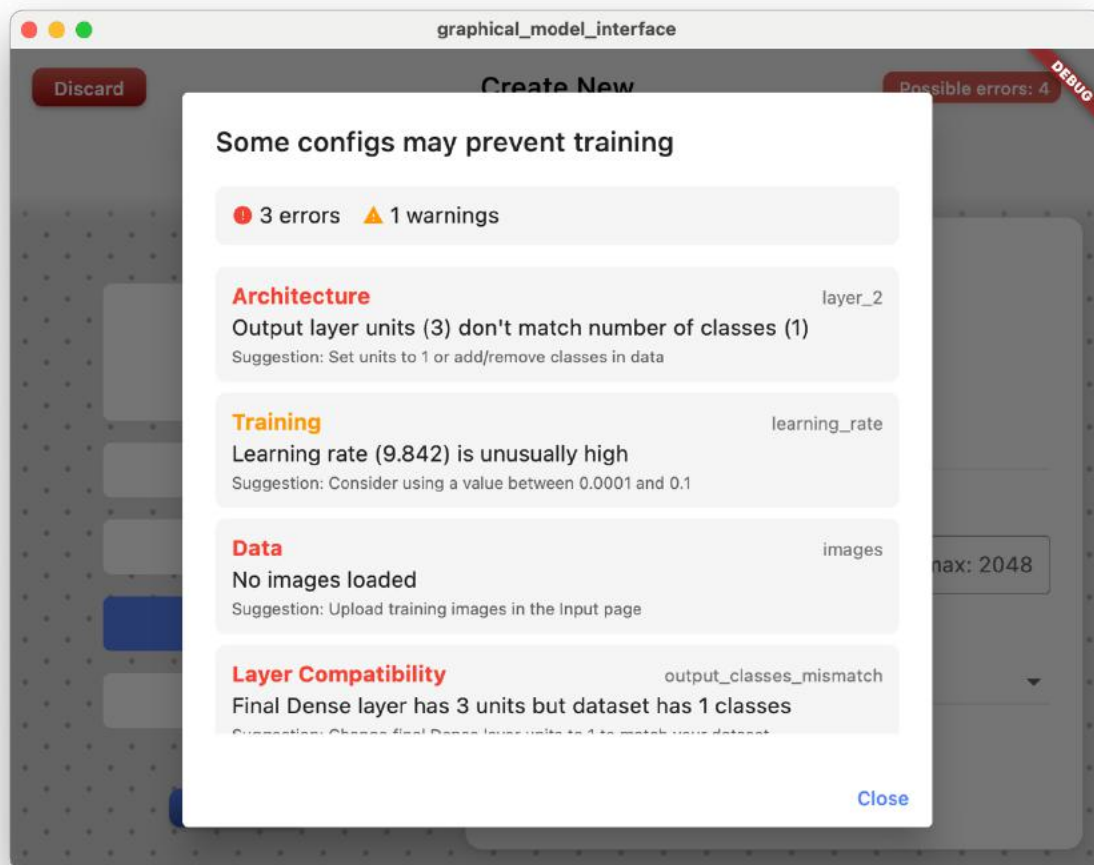


Fig 2.24 Error service dialog in Create New Page

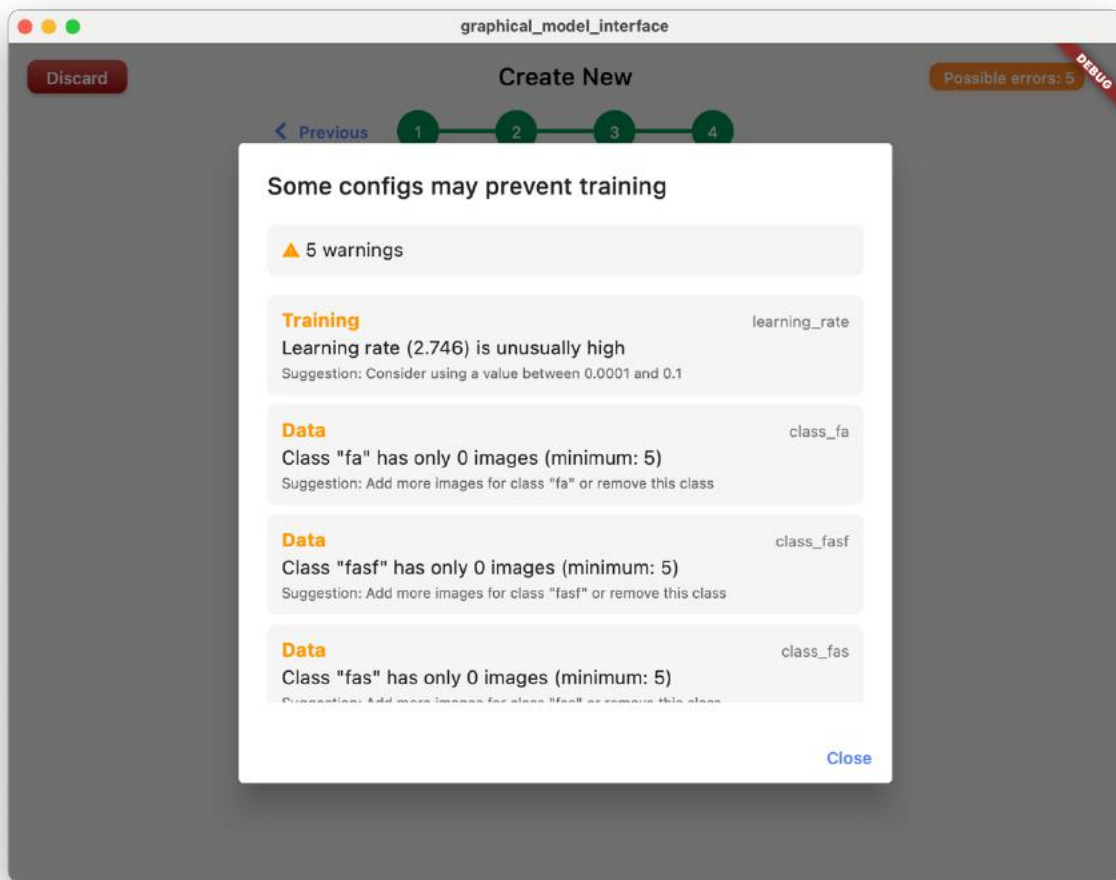


Fig 2.25 Error service with warnings only and no serious errors

2.2.3 Export Page

The Export Page is used after the setup is complete to build the training-set archive and store the configuration in the local library. As soon as the page is shown, the export process is started automatically in the background, either by creating a zip file on disk (desktop platforms) or by building an in-memory zip for download (web), while the screen remains responsive and shows live progress. The top app bar contains a single gradient button on the left, labeled “Abort” while the export is in progress and “Exit” after completion; pressing it clears the in-memory labeled data, discards any temporary export state, and returns the user to the home page. This design ensures

that the export step is treated as a short, focused operation with a clear way to cancel or leave once the archive is ready.

The main body of the page presents a centered column that guides the user through the export status. A large status line at the top shows one of three texts depending on the stage: “Exporting...” while images are being gathered into the archive, “Encoding and Writing...” when the image collection is done and the final zip is being encoded and written to storage, and “Export complete” once the archive is fully generated. Below this, either an error message is displayed in red if something failed, or a compact summary is shown with the number of labels and the total number of images in the dataset, followed by a line such as “Processing: 73% complete” driven by a progress value supplied from the export service. A horizontal progress bar with rounded corners visualizes the same progress fraction, so that both numeric and visual feedback are provided as the archive is built.

At the bottom of the column, a “Save As” gradient button is placed, which remains disabled until export has finished, preventing attempts to save an incomplete file. On desktop platforms, pressing this button opens a standard file save dialog with a suggested zip filename derived from the configuration name and timestamp; after a location is chosen, the generated zip file is copied there and a brief message is shown confirming the save path. On the web, the same button triggers download of the in-memory zip by creating a virtual file with the generated bytes and filename, and a confirmation snackbar is displayed to indicate that the browser download has started. During the export itself, the ArchiveService constructs a folder structure inside the zip with one folder per label, adds each image with a cleaned-up filename, and writes a metadata.json file that records the full training configuration, helper options, labels, per-label image counts, timestamps, and a special note when the dataset happens to be empty, while continuously updating progress so that the Export Page can reflect each stage clearly.

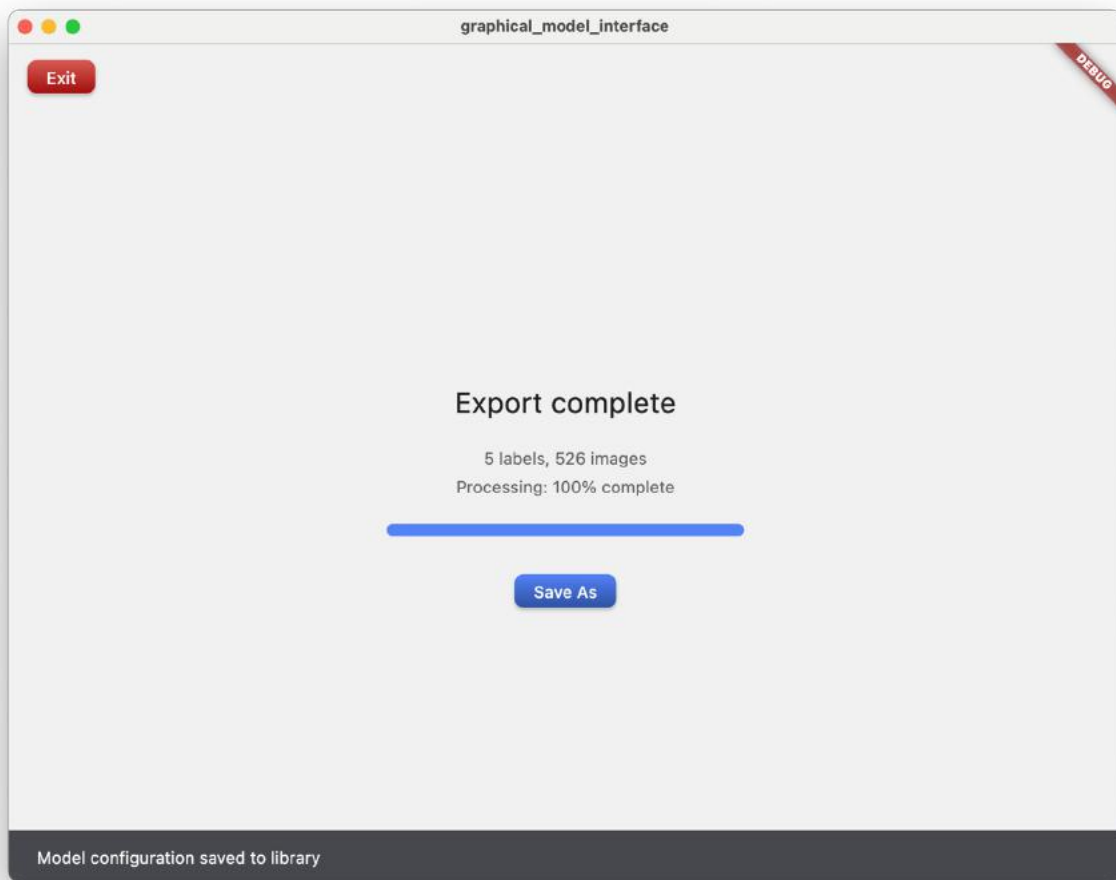


Fig 2.26 Export Page

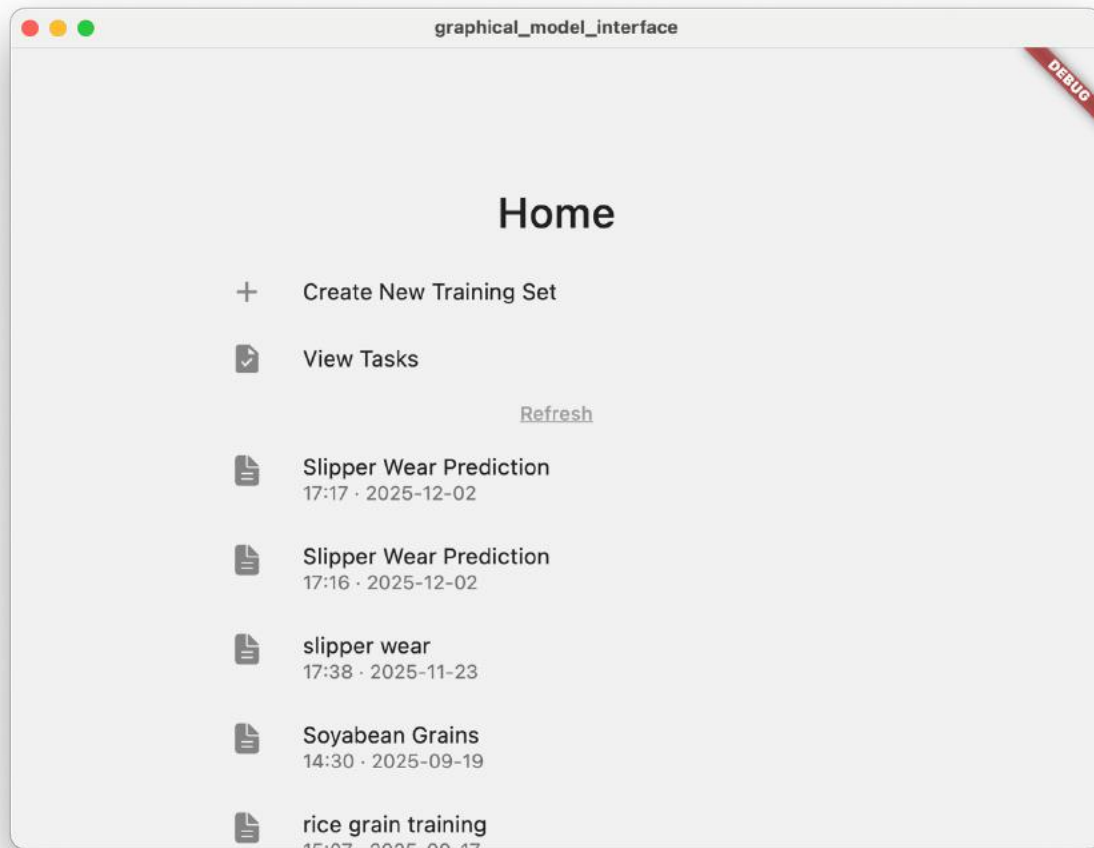


Fig 2.27 On exit, the app navigates back to the Home Page, which now has the displays the newly created training set at the top below the Refresh button

2.2.4 Training Set Details Page

The Training Set Details Page is used for inspecting and managing a single saved training set configuration, combining high-level statistics and actions on the left with a full JSON view of the stored configuration on the right. The page is opened from the home screen when a training set entry in the library is selected, and it uses the metadata and paths that were recorded when the configuration and dataset zip were saved.

The page is presented as a scaffold with a dark app bar at the top that displays the human-readable name of the training set taken from the stored configuration, so that the current context is always visible. Below the app bar, the body is divided into two equal vertical panels using a row layout: the left panel contains creation details, dataset statistics, and action buttons, while the right panel displays the complete configuration in a scrollable text area. This side-by-side arrangement allows the summary information and the underlying JSON configuration to be viewed together without switching screens.

At the top of the left panel, the creation timestamp of the configuration is shown in a compact “Created: YYYY-MM-DD HH:MM” format, which is derived from the stored ISO timestamp and reformatted for readability. Below this, a light-colored information card is displayed that shows dataset statistics loaded asynchronously: while the statistics are being read from disk, a small progress indicator is shown; once ready, the card lists file size, number of labels, and total images in neatly aligned rows. Under these rows, all label names are displayed as a bulleted list in a scrollable area with a fixed maximum height, so that long label lists remain visible without stretching the page layout. If the underlying dataset file or metadata cannot be read, a short message indicates that the statistics could not be loaded instead of leaving the area blank.

Below the statistics card, three vertically stacked gradient buttons are shown. The first button, “Save Training Set To ...”, allows the associated dataset zip file to be copied to a user-chosen location using a native save dialog; while this operation is in progress, the label of the button changes to “Saving...” and the button is disabled to prevent duplicate actions, and any errors such as a missing dataset file are reported in a red error message area beneath the buttons. The second button, “Send to Server”, is reserved for triggering a server-side training workflow and is currently presented as an active control placeholder, clarifying the intended future extension of the system.

The third button, “Delete”, is coloured as a warning action and opens a confirmation dialog explaining that both the configuration and its associated dataset file will be removed; if confirmed, the stored JSON configuration is deleted from the `model_configs` directory and the dataset zip on disk is also deleted when possible, with a snackbar message summarizing the result and an automatic return to the home page. A small italic note at the bottom of the left panel reiterates that deleting the configuration also removes the dataset file, reinforcing this consequence even before the delete dialog is opened.

The right-hand panel focuses on transparency by exposing the full stored configuration. At the top of this panel, a “Configuration” heading is shown, followed by a bordered container that fills the remaining height and is given a light grey background to distinguish it from the rest of the UI. Inside this container, the entire configuration map (including `modelConfig`, helper options, labels, image counts, export paths, and timestamps) is pretty-printed as indented JSON and displayed as selectable text, allowing sections to be highlighted and copied for documentation, debugging, or replication in scripts. The text area is wrapped in a scroll view so that even very large configurations remain accessible without shrinking the font, and a monospace font is used so that keys, values, and nesting are visually aligned, making the structure easy to interpret.

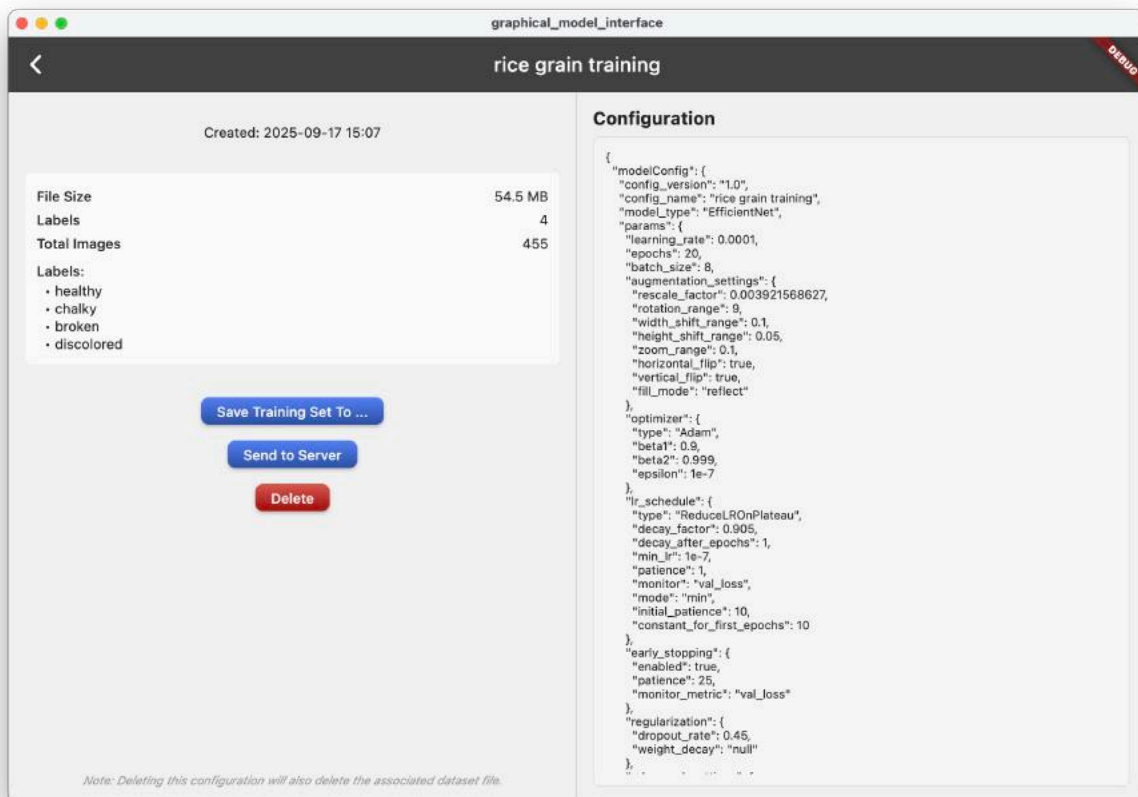


Fig 2.28 Training Set Detail Page

2.2.5 View Tasks Page

The View Tasks Page is used to monitor all model-training jobs that have been submitted from the app to the ML training server. It is built as a simple scaffold with a standard app bar and a vertically centered column whose width is limited to keep the content readable on wide screens. At the top of this column, the heading “Tasks” is displayed in a large font, followed by a “Refresh” text button that appears as grey underlined text and has no splash effect, so that it behaves like a subtle hyperlink rather than a primary action button. When this button is pressed, the list of jobs is fetched again from the server endpoint that is stored in the app settings (jobsEndpoint), which typically points to the /jobs route of the FastAPI server described in the server code.

The main content of the page is provided by an area that reacts to the loading state of the jobs list. While jobs are being fetched, a Cupertino-style activity indicator is shown to indicate that network

activity is in progress; if an error occurs (for example, the server is unreachable), a short text message with the error description is displayed instead; and if the server returns an empty list, a neutral message “No tasks found.” is shown. Once jobs are available, they are laid out in a non-scrolling ListView embedded inside the outer scroll view, so that the entire page scrolls as a single unit while still handling potentially long lists gracefully. Before display, the jobs are sorted by their creation time in descending order, ensuring that the most recent jobs appear at the top, which is consistent with the way the /jobs endpoint collects metadata from Redis for each job_* key.

Each job is represented by a list tile that summarizes its most important attributes. The title line shows the configuration name that was recorded when the training job was created on the server (for example, “rice grain training”), and the subtitle combines the human-readable creation time, formatted as “HH:MM · YYYY-MM-DD”, with the model type used for that job (such as EfficientNet or ResNet). On the right side of the tile, the current status string is displayed, which reflects the status value maintained in Redis by the training task, such as QUEUED, PROCESSING, COMPLETED, or FAILED; this status is updated by the Celery task as it moves through dataset extraction, validation, script generation, training, and completion, or when an error is recorded. Tapping a tile opens a dedicated Task Details page (TaskInfoPage) that is provided with the job ID, status, configuration name, model type, and creation timestamp, so that more detailed progress information and result downloads can be accessed for that specific job.

Below the list, a small grey line of text indicates how many tasks have been fetched and echoes the jobs endpoint URL, which serves as a subtle diagnostic aid when server addresses are changed in the settings. This page therefore forms the bridge between the graphical desktop application and the headless, containerized server: jobs that are created by uploading a configuration and dataset to the /train endpoint and are processed asynchronously by Celery workers are surfaced here in a compact, refreshable overview, with clear feedback about their creation time, configuration, model type, and lifecycle state.

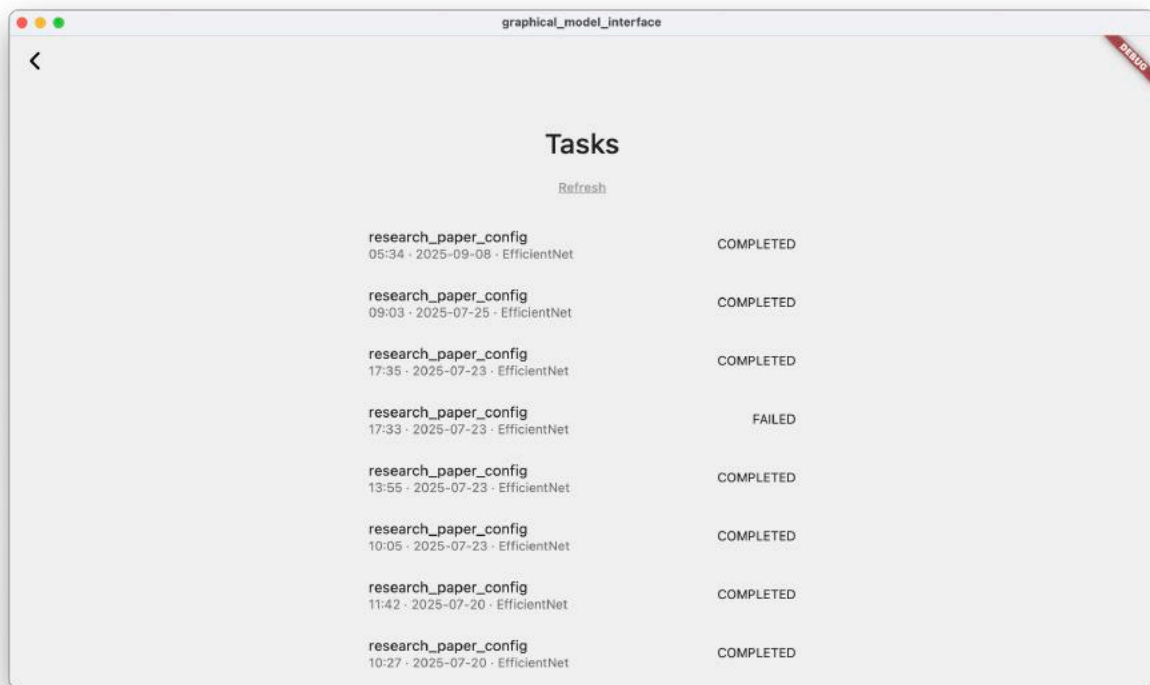


Fig 2.29 View Tasks Page

2.2.6 Completed Task Information Page

The Completed Task Information Page is used after a training job has been created so that all results for a single job can be inspected and downloaded in one place. The page is opened from the View Tasks Page with the job identifier, status, configuration name, model type, and creation time already supplied, and these values are used to populate the header and metadata section.

The page is built as a scaffold whose app bar contains the configuration name as the centered title, so that the specific experiment being inspected is always clear. On the right side of the app bar, the current job status (for example, QUEUED, PROCESSING, COMPLETED, or FAILED) is shown as plain text, mirroring the status that is maintained in Redis and exposed through the `/job/{job_id}` endpoint on the server. The body is organised as a horizontal row: a left-hand information and download panel of fixed width is placed next to a vertical divider, and an empty expanded area is reserved on the right for future additions such as metrics or visualisations.

In the upper part of the left panel, three lines of selectable text are shown: the job ID, the creation timestamp, and the model type. These fields are made selectable so that identifiers and timestamps can be copied directly into logs, emails, or research notes without retyping, which is particularly helpful when many experiments are being compared. The creation time corresponds to the value that was recorded in Redis when the `/train` endpoint accepted the configuration and dataset and created the training job, so a consistent trace from upload to completed results is maintained.

Below the metadata, a “Download all” button is placed inside a small download section. When this button is pressed, a complete results archive is requested from the server’s `/job/{job_id}/download-complete` endpoint, which streams a ZIP file containing all available outputs from the job’s outputs directory together with the original configuration and any error logs. On desktop and mobile platforms, a native save dialog is opened so that the user can choose a destination filename; the suggested name is constructed from a sanitised version of the configuration name, the model type, and a timestamp, which keeps exported result archives uniquely identifiable.

During the download, the button text is changed to “Downloading...” and the control is disabled so that duplicate downloads are not triggered, while a horizontal progress bar and status text are made visible directly underneath. As data is streamed from the server, the progress bar is updated based on the number of bytes received, and the status line is updated with human-readable information that is derived from helper functions: total size is formatted in B, KB, MB, or GB, and instantaneous speed is formatted in B/s, KB/s, or MB/s; when the content length is known, an estimated remaining time in seconds is also computed and appended. A separate line shows the numeric percentage completed while the transfer is in progress, giving a precise sense of how far the download has advanced.

If the user cancels the save dialog, the download state is reset to idle with no residual progress, and no network request is kept active. On successful completion, the progress bar is filled, the status text is changed to “Download completed!”, and a green snackbar is shown at the bottom of the window that confirms success and, on desktop, also reports the name of the saved file. In case of any failure—such as an HTTP error from the server or a file-system problem—an error message string is stored, a red snackbar with the error description is shown, and a small red “Error: ...” line

is displayed under the progress area so that the failure reason remains visible after the snackbar disappears.

When the same page is used from a web build, the behaviour is adjusted so that no native save dialog is required. Instead, the ZIP archive is streamed entirely into memory from the same `/job/{job_id}/download-complete` endpoint, and the same progress bar and status text are updated using the content length and download rate, providing equivalent feedback about bytes transferred, speed, and estimated time remaining. The code path is prepared so that a client-side file-saving utility can be plugged in to trigger a browser download using the accumulated bytes, which keeps the architecture consistent across platforms even though the actual file write is handled differently.

Below the download controls, a section lists all individual result files that have been generated for the job. It waits for a call to the server's `/job/{job_id}/files` endpoint, which scans the job directory and its outputs subdirectory for known artifacts such as the final trained model, best checkpoint, training history JSON, architecture description, learning-curve image, confusion matrix CSV, classification report JSON, training log, and error log, and returns only those entries that exist for the current job. While this information is being loaded, a Cupertino activity indicator, which is a circular loading indicator, is shown; if an error occurs, the error message is presented as simple text.

When the file list has been loaded, it is rendered as a non-scrollable list inside the panel so that the entire left side continues to scroll as one unit. Each file is represented by a list tile that shows the filename on the first line and the size in megabytes on the second line, where the size has already been computed on the server and is included in the response. This list acts as a transparent summary of what will be contained in the downloaded ZIP archive and what evidence is available for reporting and debugging; for example, the presence of an error log entry indicates that a failure occurred during training even if the main status was already seen, and the presence of a training history file and visualization indicates that full metric traces are available for later analysis.



Fig 2.30 Completed Task Info Page

2.2.7 Settings Page

The Settings Page centralises all global preferences that affect image handling, communication with the training server, processing behaviour, memory usage, and local storage, and it presents them as a vertically scrolling list of clearly separated sections. The page is opened from the home screen using a full-screen scaffold with a custom app bar that places a “Save and Exit” gradient button on the left, the title “Settings” in the centre, and empty space on the right; pressing this button saves all modified settings and closes the page, and if any key image-processing parameters have changed, the temporary working files are cleared first so that stale pre-processed images are not reused.

The first section is labelled “IMAGE” and controls how images are pre-processed during labeling. A “Resize images while labeling” row enables or disables automatic resizing and includes two compact numeric fields in the trailing area where target width and height can be entered; these

fields accept only digits and update the stored resize dimensions immediately, while disabling them when resizing is turned off ensures that inconsistent combinations cannot be set. A “Center crop images to avoid squeezing” option appears beneath and is enabled only when resizing is active; this setting determines whether images are cropped to the central region rather than stretched, which preserves aspect ratio and reduces distortion in the training set. A third option, “Rotate images”, controls whether images whose orientation does not match the target resize orientation are rotated automatically, again only when resizing is enabled; both cropping and rotation changes are treated as key parameter changes that will trigger cache clearing on save, because they change the way processed images should look. An information row with an icon explains that down-sizing images reduces export size and transfer time and that changing these image settings will clear the cache, making the consequences explicit.

The “COMMUNICATION” section configures how the app talks to the ML training server. An “API Endpoint” row contains a text field pre-filled with the current base URL for the server, allowing users to switch between servers or ports without rebuilding the application; this value is used by other pages, such as the tasks and job-details pages, to construct URLs for /train, /jobs, /job/{id}, and download endpoints. A “Ping health” row appears below with a chevron icon; tapping it sends a simple request to the server’s health-check endpoint (typically /health), and displays either a “Ping successful” snackbar when the server responds with a healthy status or an error snackbar showing the HTTP code or exception, which helps users quickly confirm that the backend is reachable and correctly configured.

Under the “PROCESSING” label, an option “Multithreaded image processing” is presented with a descriptive subtitle noting that this mode is in an alpha stage and may be slower for very large images. This checkbox is enabled only when resizing is active and the app is not running on the web, because multithreaded processing is intended for desktop platforms where multiple CPU cores and local file access are available; when turned on, the image processing utilities can distribute work across cores while constructing the labeled dataset. The “MEMORY” section contains a “Load images in memory” option that chooses between an in-memory workflow and a cache-based workflow: on desktop platforms this checkbox can be toggled, and helper text explains that enabling it speeds up processing at the cost of higher RAM usage and disables disk

caching, whereas on web it is always treated as true and the checkbox is disabled with explanatory text, because browsers naturally keep image data in memory.

Below this, the current cache usage is shown together with controls to clear it. A composite row displays the size of the processed-images cache by querying the temporary directory and summing files whose names contain the configured prefix, then formatting the result into human-readable units such as KB or MB. A “Clear cache” gradient button next to it triggers a routine that first evicts image data from the app’s internal image cache and then attempts to delete matching files from the temporary directory, including a retry mechanism with exponential backoff on Windows to handle locked files; after clearing, a key is updated so that the size display refreshes to reflect the new state, and a snackbar informs the user of success or failure. An additional information paragraph explains that cached files retain their original filenames and that, if a new upload shares a name with an existing cached file, the cached version will be reused, so users are advised either to use unique filenames or to clear the cache when re-processing different content with the same names.

In the “STORAGE” section, the app displays how much space is being used by its permanent data directory, which includes saved model configurations and other persistent files; this value is obtained from the `PermanentData` helper that recursively sums file sizes in the application-support directory, converts the sum to B, KB, MB, or GB, and returns a formatted string. This gives users a quick sense of how much disk space is consumed by stored training sets without requiring operating system tools. At the bottom of the page, an unlabelled final section presents a small footer text, styled in grey, that can be used to convey version or disclaimer information, and the entire settings body is wrapped in a scroll view with horizontal padding that scales on wider screens, keeping the layout comfortable on both laptops and large monitors.

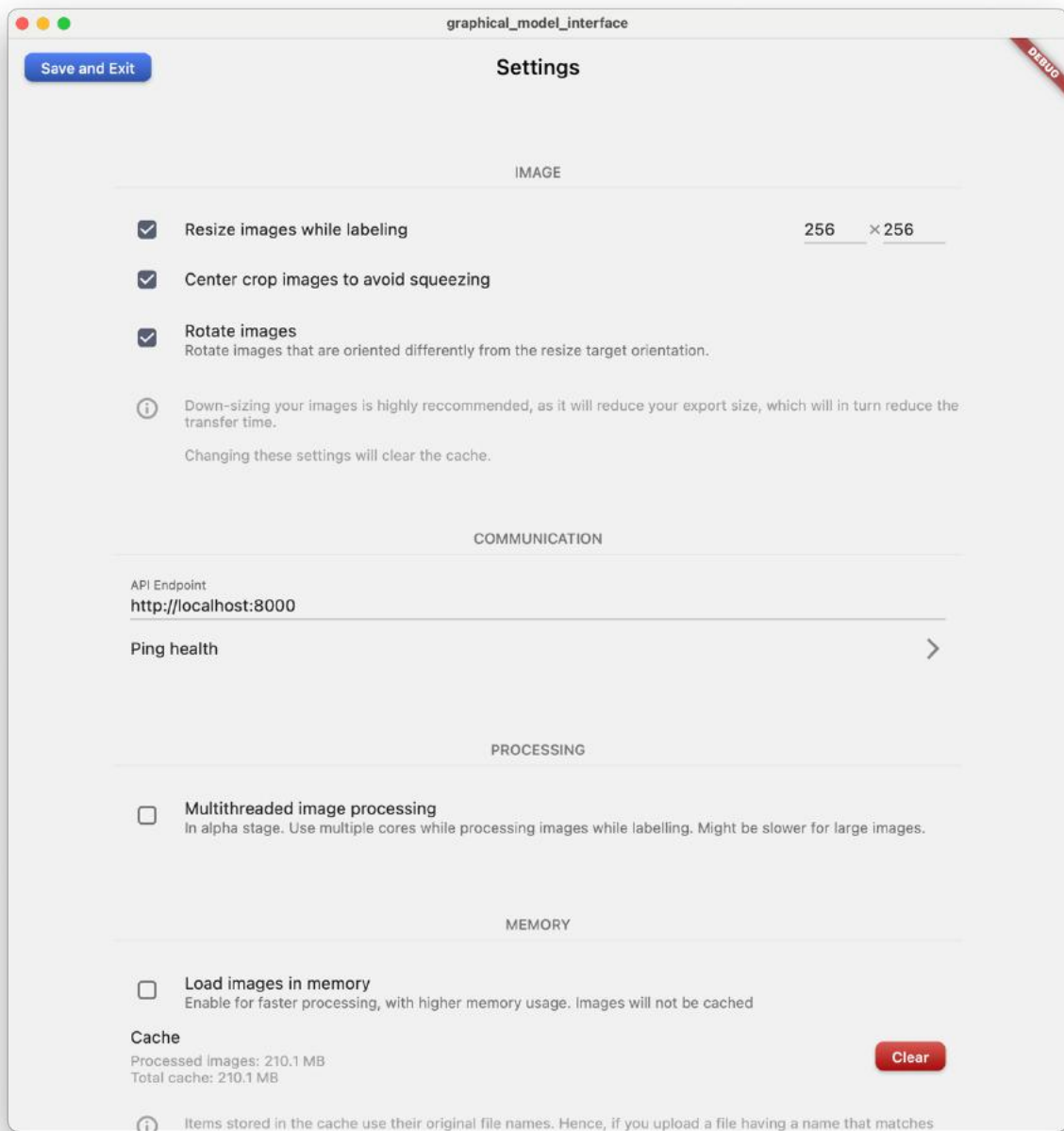


Fig 2.31 Settings Page (part 1)

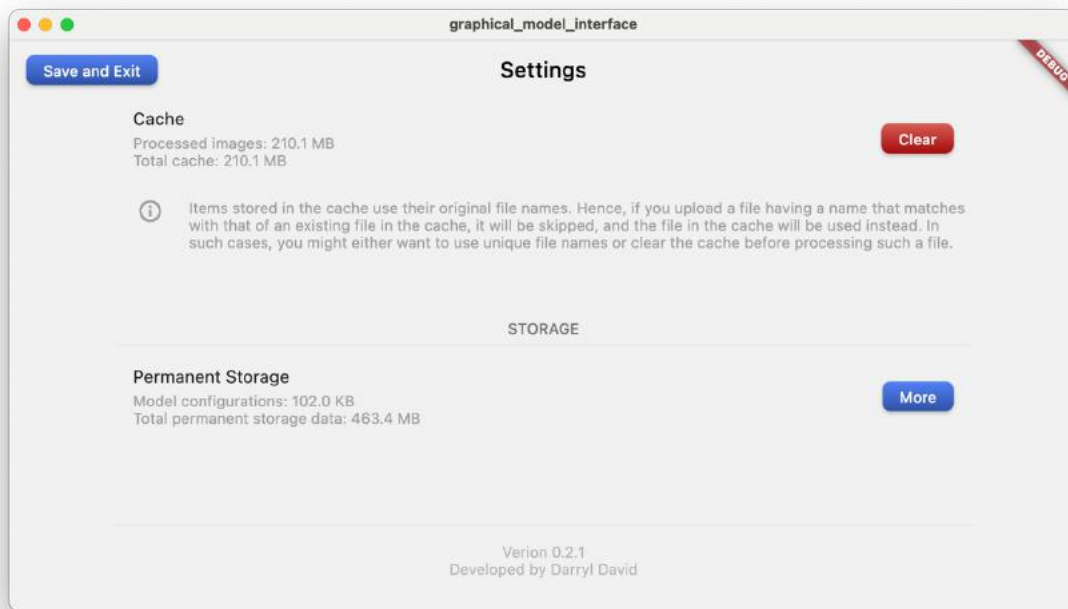


Fig 2.32 Settings Page (part 2)

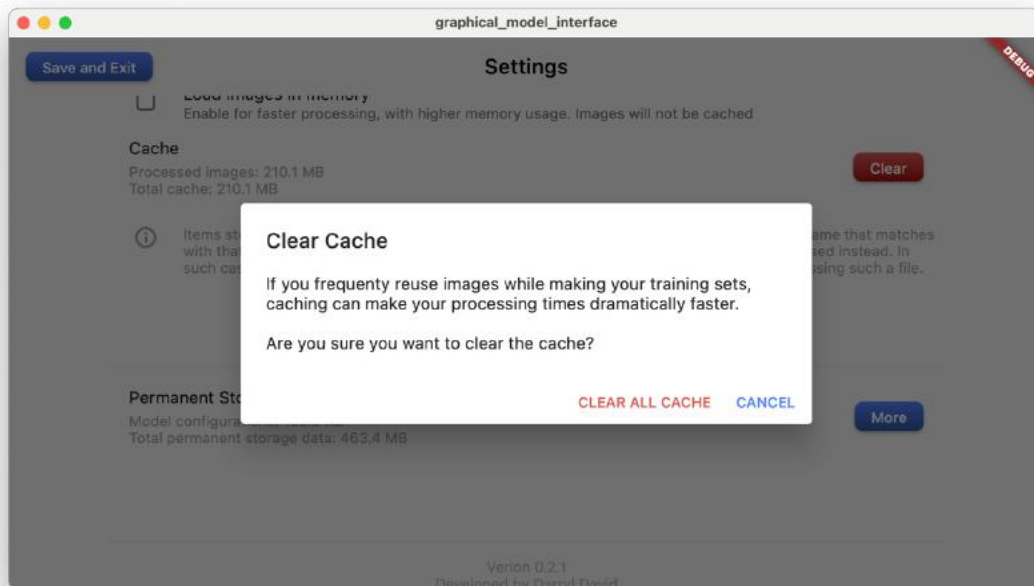


Fig 2.33 Clear Cache Confirmatory Dialog

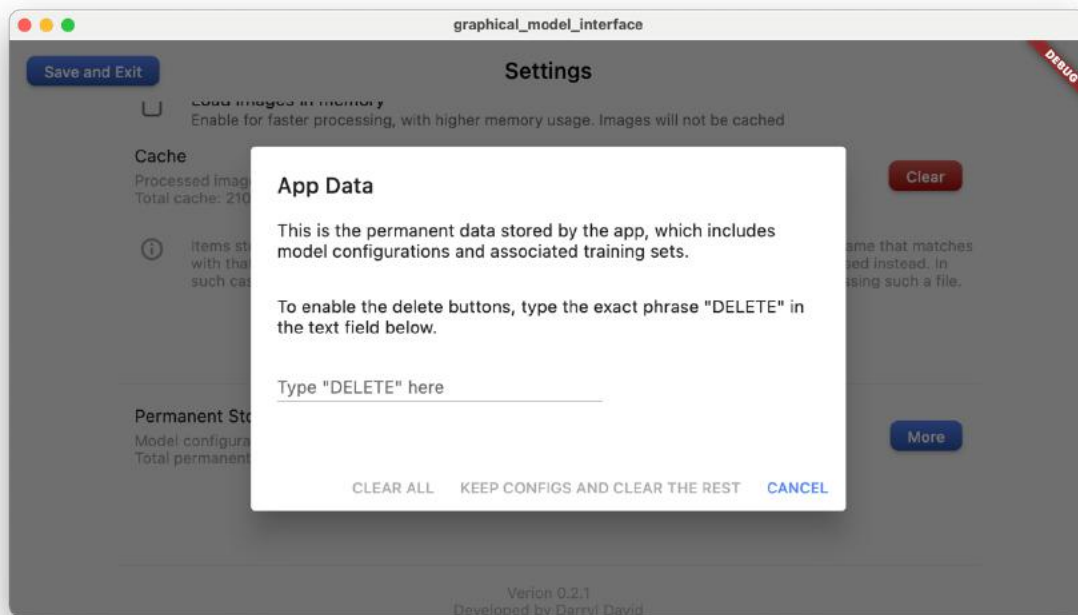


Fig 2.34 Clear Permanent Storage Confirmatory Dialog with extra safety

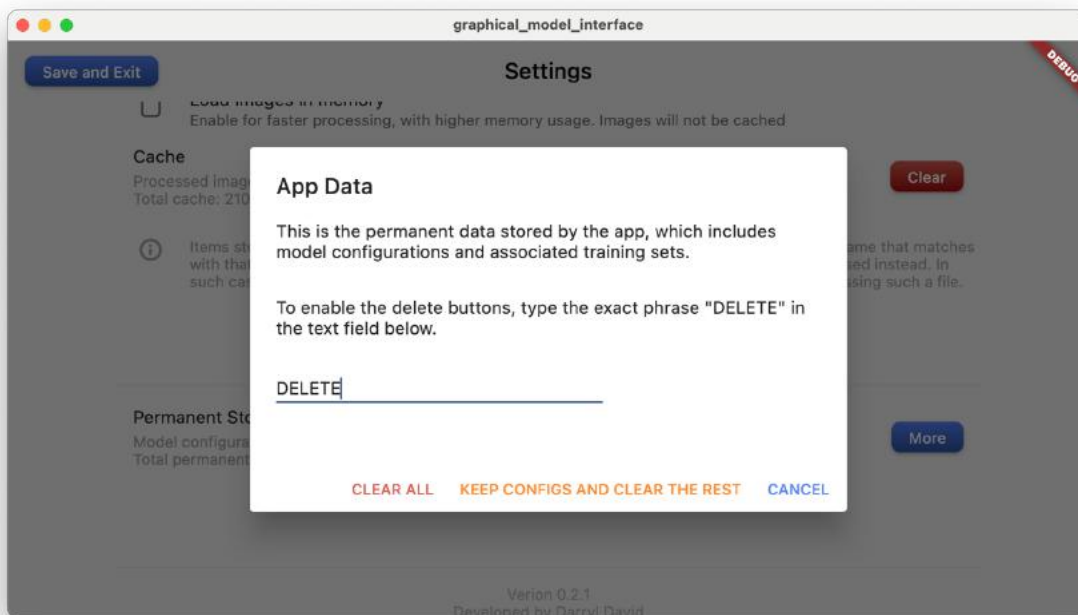


Fig 2.35 Clear Cache Confirmatory Dialog with clear buttons activated

2.3 Backend

The backend is organized as a self-contained cluster of Docker containers that run independently of the Flutter application, so that model training is handled on a dedicated machine while the user interacts only with a simple graphical interface. This separation allows heavy work such as dataset extraction, GPU-accelerated training, and result packaging to be performed on a server, while the desktop app remains responsive and lightweight, only sending configurations and datasets over HTTP and then polling for status and results.

The backend is deployed as three cooperating containers orchestrated by a single compose file: a Redis service, an API service, and a worker service. The Redis container provides a small in-memory database that stores job metadata and acts as the message broker between the API and the training workers; it exposes the standard Redis port and persists its data in a named Docker volume. The API container runs the FastAPI application, and exposes HTTP endpoints for job creation, health checks, job listing, metrics, file listing, and downloading individual artifacts or complete result archives; it shares a host directory for job folders with the worker container so that both services see the same configuration files, datasets, and outputs. The worker container runs a Celery worker process inside a TensorFlow GPU image; it consumes training jobs from Redis, executes the whole training pipeline, and writes outputs such as trained models, histories, plots, confusion matrices, and logs into the shared jobs directory that the API later serves back to the app.

The Flutter application communicates with the backend solely over HTTP, using the base API endpoint configured on the Settings page. When a training run is started from the app, the configuration JSON and dataset ZIP are uploaded to the `/train` endpoint; when tasks are viewed, the app calls `/jobs` and `/job/{id}` to retrieve status summaries; when results are opened, the app requests `/job/{id}/files`, `/job/{id}/metrics`, or `/job/{id}/download-complete` to obtain detailed outputs or a full ZIP archive. All these HTTP details are hidden behind simple buttons and status pages, so the user is not required to type URLs or interact with the server directly; the only visible setting is the API base URL and an optional “Ping health” action that checks the `/health` endpoint and displays a short success or failure message.

SSH is used only for administration and deployment, not for normal interaction from the app. The server is run as a headless Ubuntu machine that is accessed from the developer’s workstation

through key-based, password less SSH, which allows commands such as pulling code, building images, and starting docker compose to be executed securely without a graphical interface. Once the containers are running, the backend is exposed on a fixed port, and the Flutter app treats it like any other HTTP service, so end users are not required to know about SSH, Docker, or Linux; those tools remain part of the deployment environment rather than the user workflow.

When a job is created, the API service writes the uploaded configuration and dataset into a new job directory under `/var/ml-server/jobs/job_<id>`, registers a record for that job in Redis—including configuration name, model type, epochs, batch size, and creation time—and enqueues a Celery task that refers to the job identifier. The worker container picks up this task, updates the job status in Redis to reflect progress states such as processing and validating, and then unpacks the dataset ZIP, checks that the directory structure and image counts match the labels from the config, and validates that the configuration is sensible for the dataset size. A full Python training script is then generated from a Jinja2 template: the JSON configuration from the app is converted into Python objects, and all training options such as base model, augmentation, learning rate schedule, early stopping, transfer-learning freeze stages, metrics, batch size, and research-compliance modes are embedded into a standalone script file.

This generated script is invoked inside the worker container and carries out the entire training procedure: dataset splitting according to training, validation, and test fractions; construction of Keras data generators with the chosen augmentation; model construction using the selected CNN backbone; optional mixed-precision or class-balancing strategies; training with callbacks for early stopping, learning-rate reduction, and checkpointing; fine-tuning if requested; and final evaluation on the test set with computation of accuracy, precision, recall, F1 scores, confusion matrices, and detailed classification reports. All relevant artifacts are written into the job's outputs directory, including trained model files, history JSON with compact metrics and configuration echo, visualisation images of training curves, CSV confusion matrices, JSON classification reports, and text logs capturing console output and errors; paths and summary statistics are returned as the Celery task result and also reflected in Redis, from where the API endpoints serve them to the app.

The system is designed primarily for a workstation-class Linux server that runs Docker and has an NVIDIA GPU, but it remains portable to other environments. The docker-compose file defines a dedicated Docker network and volume, and specifies that the worker service reserves NVIDIA

devices through Docker's device-reservation mechanism; environment variables in both the compose file and the worker image indicate that all GPUs are visible and that the necessary driver capabilities for compute and utility operations are enabled. The worker image is based on an official TensorFlow GPU build and enables GPU memory-growth settings, so that the training job adapts to the available GPU memory and avoids pre-allocating all resources, while the training template inspects available GPUs and prints their details before starting.

Because both the API and worker containers mount the same `./jobs` directory from the host, the setup can also be run on a single development machine: on such a system, the Docker stack can be started locally, the Flutter app's API endpoint can be set to `http://localhost:8000`, and training will still use the GPU if an NVIDIA driver and container toolkit are installed. This architecture therefore keeps computation, storage, and scheduling on the server side while exposing a clean HTTP interface to the desktop app, allowing non-technical users to create, submit, monitor, and download complex CNN training runs without being aware of Celery, Redis, Docker, or GPU configuration details.

CHAPTER 3: IMPLEMENTATION

ML Vision Studio has been implemented as a two-part software system in which all user interaction is handled by a cross-platform Flutter desktop application, while computationally intensive training is executed on a separate, containerized backend server. The desktop program, which is optimized for Windows, macOS, and Linux, is used to construct labelled image datasets, configure CNN architectures and hyperparameters through a guided four-step workflow, and manage a local library of training sets; in this chapter, this graphical front-end will be referred to as MLVS 1. The server-side component, referred to as MLVS 2, has been realized as a cluster of Docker containers running headless on a powerful machine, where a FastAPI web service exposes HTTP endpoints, Celery workers execute training tasks, and supporting services such as Redis coordinate job management and status tracking. Communication between MLVS 1 and MLVS 2 is carried out over standard HTTP on a local network using JSON payloads and file uploads, so that training sets exported by the desktop application are transmitted to the backend as self-contained packages and the resulting models and metrics are retrieved asynchronously without requiring direct user access to the server. Through this architecture, a clear separation is maintained between the graphical, human-facing environment and the headless training infrastructure, while still allowing both parts to be deployed either on the same physical machine or on different systems within the laboratory network.

3.1 MLVS 1

MLVS 1 has been designed as a fully graphical desktop program that runs natively on Windows, macOS, and Linux, with additional but limited builds for Android, iOS, and the web, so that the same codebase can be used across all machines in the laboratory. The application has been implemented in Flutter using Material-style widgets and custom components, and the core logic has been written in Dart, which provides reactive UI rendering, a single-threaded event loop for user interaction, and explicit support for desktop windowing, keyboard input, and file-system access. Within this program, the complete CNN-training workflow has been encapsulated as a

four-step wizard that guides the user through labelled image import, layer-head configuration on top of pre-trained backbones such as EfficientNet, MobileNet, ResNet, and Inception, hyperparameter specification through a metadata-driven parameter grid, and definition of train–validation–test splits using interactive split sliders.

3.1.1 Flutter

Flutter is an open-source UI software development kit created by Google for building natively compiled applications for mobile, web, and desktop platforms from a single Dart codebase. In contrast to traditional cross-platform toolkits that wrap platform-native widgets, a custom rendering engine based on Skia (and Impeller on newer pipelines) is used to draw all visual elements directly onto a GPU-accelerated canvas, which enables consistent look-and-feel and high frame rates across operating systems.

The framework is organized into three main layers a Dart framework layer that defines widgets, rendering, animation, and gesture libraries; an engine layer written primarily in C++ that embeds the Dart runtime and interfaces with Skia; and platform embedders that integrate with each operating system’s windowing, input, and event systems. Application code in Dart constructs a widget tree, which is transformed into an element tree and then into a render-object tree; during each frame, layout and paint phases are executed, and the resulting render objects are converted into composited layers that the engine submits as drawing commands to Skia on the GPU thread. A retained-rendering strategy is employed in which only regions of the scene that have changed are repainted, which reduces overdraw and supports smooth animations at approximately 60 frames per second or higher on capable hardware.

3.1.1.1 The Dart Programming Language and Compilation

The Dart programming language is used for both application code and most of the Flutter framework, and its toolchain supports two complementary compilation modes. During development, Dart code is executed by the Dart VM with just-in-time (JIT) compilation, which enables stateful hot reload so that code changes can be injected into a running process without a full restart, significantly shortening the edit–run–inspect cycle. For release builds, ahead-of-time

(AOT) compilation is applied to produce native machine code (for example, ARM or x64 binaries), which removes the runtime JIT overhead and yields startup times and steady-state performance that are comparable to manually written native applications.

Concurrency in Flutter applications is handled through Dart isolates, which are independent heaps and event loops that communicate via message passing, while garbage collection is managed by the Dart runtime to reclaim memory in the background. Interaction with platform-specific APIs is provided through platform channels, where method calls and structured data are serialized on the Dart side, transferred over a channel, and deserialized on the native side (such as Kotlin/Java on Android, Swift/Objective-C on iOS, or C++ on desktop embedders), allowing access to file systems, networking stacks, and hardware features without exposing these low-level details at the UI layer.

3.1.1.2 Desktop and plugin ecosystem

Native desktop executables for Windows, macOS, and Linux are produced by compiling the same Dart source code and linking the engine with platform-specific embedders that create a native window, forward keyboard and mouse events, and host an OpenGL, Metal, or Vulkan surface for the Flutter engine. Flutter’s desktop support additionally extends to plugins, so that packages offering HTTP clients, file pickers, window management, and system dialogs can provide separate implementations for each desktop platform while exposing a single Dart API to the application. This ecosystem, combined with the built-in Material and Cupertino widget libraries, allows complex interfaces to be assembled from composable widgets while still respecting platform conventions where desired.

3.1.1.3 Rationale for using Flutter in MLVS 1

Flutter was selected for MLVS 1 so that the same GUI for dataset construction, model configuration, and training-set management could be compiled once and run on Windows, macOS, and Linux desktops that are commonly used in the laboratory, with additional web and mobile builds available if future experiments require them. The custom rendering pipeline based on Skia was leveraged to implement responsive layouts, custom controls such as interactive split sliders

and 3D-styled parameter sliders, and canvas-based visualizations of model layers without depending on native widget sets that differ across platforms. Dart's hot-reload development workflow and JIT runtime were used to accelerate iterative design of the HCI aspects of MLVS 1, while AOT-compiled desktop binaries were produced for end-users so that image-intensive operations and large forms could be handled with native performance.

The availability of mature plugins for HTTP networking, file-system access, and platform-specific dialogs allowed MLVS 1 to integrate image importing, zip-file export, cache management, and HTTP communication with the MLVS 2 server without custom native code for each operating system, which kept the implementation modular and largely application-agnostic. Finally, Flutter's widget-centric, declarative architecture aligned well with the requirement that the same core logic be reused across different CNN image-classification applications, since the UI for labels, layers, parameters, and splits could be described as data-driven widget trees that remain independent of the domain being studied.

3.1.2 Project Structure

The MLVS 1 Flutter project is organized with a standard Flutter top-level structure, while almost all application logic, user-interface code, and data-handling code are concentrated in the lib directory.

3.1.2.1 Project Tree

```
├── analysis_options.yaml
├── android/
├── build/
├── devtools_options.yaml
├── Documentation.md
├── Features.md
├── graphical_model_interface.iml
├── ios/
├── lib/
├── linux/
├── macos/
└── pubspec.lock
```

- pubspec.yaml
- README.md
- support/
- test/
- web/
- windows/

3.1.2.2 Lib Tree

```
lib/
├── components/
│   ├── error_badge.dart
│   ├── gradient_button.dart
│   ├── image_grid_item.dart
│   ├── label_list_tile.dart
│   ├── multi_selector.dart
│   ├── popup.dart
│   ├── selector.dart
│   ├── slider_theme.dart
│   ├── split_slider.dart
│   └── stepper.dart
├── data/
│   ├── archive_service.dart
│   ├── cached_data.dart
│   ├── configs.dart
│   ├── image_processing.dart
│   ├── labeled_data.dart
│   ├── permanent_data.dart
│   └── settings_data.dart
├── main.dart
├── pages/
│   ├── create_new/
│   │   ├── create_new_page.dart
│   │   ├── input_page.dart
│   │   ├── layers_page.dart
│   │   └── params_page.dart
│   ├── export_page.dart
│   ├── home.dart
│   ├── loading_page.dart
│   ├── settings_page.dart
│   ├── task_info_page.dart
│   ├── tasks_page.dart
│   └── trainingset_info_page.dart
```

```
├── services/  
│   ├── background_validator.dart  
│   └── config_validator.dart  
└── theme.dart
```

In this structure, `main.dart` defines the root widget tree and global application wiring, while `theme.dart` centralizes colors, typography, and other styling constants so that they can be reused consistently across all pages. The components directory contains reusable, page-agnostic widgets such as `gradient_button`, `split_slider`, and `multi_selector`, which implement custom UI elements used in multiple places (for example, the parameter grid and split-selection page) without duplicating code. The data directory groups all modules that manage data and persistence: dataset archiving, cached and permanent storage, configuration structures, image processing for resizing and caching, label representations, and settings storage, so that these concerns remain independent of any particular screen. The pages directory contains the actual screen layouts and navigation endpoints—`home`, `create_new`, `tasks_page`, `settings_page`, and others—each implemented as stateful or stateless widgets that compose the lower-level components and invoke the data layer. Finally, the services directory collects non-visual background logic such as the configuration validator and continuous background validation service, which operate on shared configuration state and surface results through UI widgets like `error_badge` without being tied to a single page.

3.1.2.3 Relation to common Flutter layouts

Compared with other common Flutter folder layouts, this project follows a concern-based, semi-layered organization rather than a pure feature-first or flat screens/widgets/models split. In many introductory Flutter projects, all UI code is placed under `lib/screens` and reusable widgets under `lib/widgets`, with models and services mixed together; in contrast, MLVS 1 separates reusable UI (components), page-level UI (pages), data manipulation and persistence (data), and cross-cutting background logic (services), which makes the boundaries between presentation, data handling, and validation explicit. This organization remains compatible with Flutter's standard tooling and generated platform folders, while providing a clearer mapping from high-level responsibilities—such as dataset storage, image preprocessing, parameter validation, and page

layout—to specific directories in the lib tree, which benefits maintenance and extension of the application.

3.1.3 Dependencies

In MLVS 1, external Dart and Flutter packages are declared as dependencies in the `pubspec.yaml` file so that the build system can fetch compatible versions from the `pub.dev` package repository and make their APIs available to the application code. When `flutter pub get` is run, these version constraints are resolved, the corresponding packages are downloaded into the project's `.pub-cache`, and the generated `.packages / package_config.json` metadata allows imports such as `import 'package:http/http.dart'` as `http`; to be resolved at compile time

```
dependencies:
  flutter:
    sdk: flutter
  cupertino_icons: ^1.0.8
  file_picker: ^9.0.0
  image: ^4.5.2
  shared_preferences: ^2.5.2
  path: ^1.9.1
  path_provider: ^2.1.5
  archive: ^4.0.4
  file_selector: ^1.0.3
  flutter_staggered_grid_view: ^0.7.0
  http: ^1.5.0
```

The entry `flutter: sdk: flutter` declares a dependency on the Flutter SDK itself, which provides the core framework libraries (`material`, `cupertino`, `widgets`, `services`) and the engine bindings required to compile MLVS 1 for desktop, web, and mobile targets. All page widgets (`Home`, `CreateNew`, `SettingsPage`, parameter grids, and others) are built using these core Flutter libraries, as can be seen from imports such as `package:flutter/material.dart` and `package:flutter/cupertino.dart` in the UI files.

cupertino_icons:

The `cupertino_icons` package supplies the iOS-style glyph fonts and icon data that back the `CupertinoIcons` class, extending the built-in Material icon set with a library of platform-consistent symbols. In MLVS 1, this package is used wherever Cupertino-style icons are displayed, for example in the Home page where `CupertinoIcons.doctest_fill` and `CupertinoActivityIndicator` are imported from `package:flutter/cupertino.dart` and rendered alongside Material components to indicate documents and loading states in a familiar way for macOS and iOS users.

file_picker:

The `file_picker` plugin provides a cross-platform abstraction over native file-selection dialogs, exposing methods like `FilePicker.platform.pickFiles()` and `FilePicker.platform.saveFile()` that return absolute file paths or byte streams selected by the user. In MLVS 1, this dependency is used in the dataset input and export flows to open native file pickers when images are added to labels and when an exported training-set ZIP or results archive is saved to a user-chosen location, thereby integrating tightly with each desktop operating system's file explorer while keeping the Dart code platform-agnostic.

image:

The `image` package is a pure-Dart image-processing library that supports decoding common formats (such as PNG and JPEG), arbitrary resizing, cropping, rotation, and pixel-level manipulation without relying on platform-native APIs. In MLVS 1, this package is employed inside `image_processing.dart` to implement automatic resizing, center-cropping, and orientation correction of images during labeling; it operates on in-memory `Image` objects so that down-sampling, cropping windows, and rotation by 90 degrees or other angles can be applied consistently before the processed images are written back to disk or cached.

shared_preferences:

The `shared_preferences` plugin offers a simple key-value storage abstraction that maps to `NSUserDefaults` on iOS, `SharedPreferences` on Android, and suitable equivalents on desktop, allowing primitive types to be persisted across application restarts. In MLVS 1, this package is wrapped by `settings_data.dart` so that global preferences such as resize dimensions, image-processing flags, API endpoint URLs, and multithreading options are stored and retrieved transparently when the Settings page is opened or saved, ensuring that users' choices persist between sessions without a custom configuration file format.

path:

The `path` package provides platform-independent utilities for manipulating file-system paths, including functions for joining segments, normalizing separators, extracting basenames and extensions, and resolving relative paths. In MLVS 1, the helper modules under `data/` such as `archive_service.dart`, `cached_data.dart`, and `permanent_data.dart` use these utilities to construct paths for cache directories, dataset ZIP locations, and configuration files in a way that works correctly across Windows, macOS, and Linux, avoiding manual string concatenation with hard-coded separators.

path_provider:

The `path_provider` plugin returns well-known directories on the local file system, such as the application documents directory, temporary directory, and application-support directory, for each supported platform. MLVS 1 integrates this package in `permanent_data.dart` and `cached_data.dart` to locate persistent storage for model configurations and training-set archives, as well as temporary storage for processed-image caches, so that these files are placed in OS-appropriate locations rather than in arbitrary relative folders inside the project tree.

archive:

The archive package implements format-agnostic compression and archive handling in Dart, including creating and reading ZIP files by operating on raw byte lists and file streams. In MLVS 1, this library is used by `archive_service.dart` to bundle labeled image directories into ZIP archives when a training set is generated, and to unpack ZIP files received from the backend when results or datasets need to be inspected; this ensures that the dataset structure and configuration JSON are encapsulated in a single portable file per training set.

file_selector:

The `file_selector` plugin offers another layer of abstraction over native file and directory selection dialogs, particularly for desktop and web, using APIs such as `openFile` and `getSaveLocation` that return high-level descriptors rather than just paths. In MLVS 1, this package complements `file_picker` in export-related pages by providing a uniform interface for save dialogs and file-selection interactions that behave consistently on desktop platforms and in web builds, especially where browser security restrictions prevent direct access to absolute file paths.

flutter_staggered_grid_view:

The `flutter_staggered_grid_view` package supplies a set of advanced grid layouts, including masonry and staggered grids, built on top of Flutter's sliver and grid infrastructure to support variable item sizes while maintaining high performance. In MLVS 1, this package is imported in `params_page.dart` to render the parameter cards as a responsive masonry-style layout, where the number of cards per row adapts to the available width and cards of different heights are arranged without excessive gaps, resulting in a compact, information-dense control surface for hyperparameter configuration.

http:

The `http` package provides a high-level, platform-independent HTTP client for Dart, with functions like `get`, `post`, and `Client.send` that support headers, query parameters, and streamed bodies. In MLVS 1, this library is used in the Settings page and task-related pages to communicate

with the MLVS 2 backend: the Settings page’s “Ping health” action sends a GET request to the configured health endpoint, the tasks view fetches job lists and statuses from the jobs API, and the task detail and export flows download result archives and file lists via HTTP, all using the base URL stored in `SettingsData.apiEndpoint`.

Overall, these dependencies collectively support file selection, image processing, persistent configuration, archive management, advanced layout, and network communication in MLVS 1, while the core Flutter SDK and Dart tools orchestrate their integration through the `pubspec.yaml` dependency system.

3.1.4 Utilities and Data

3.1.4.1 Image Processing

Images are represented and processed in MLVS 1 through a compact `ImgData` holder and a central `ImgProcessing` utility that coordinates decoding, resizing, cropping, rotation, caching, and optional multithreading. The newer `dart:ui`-based pipeline has been adopted in preference to a legacy `package:image` pipeline to exploit Flutter’s native image codecs and hardware-accelerated canvas operations for better performance and visual consistency.

ImgData: image representation and lifecycle

Each image selected during labeling is stored as an `ImgData` object, which carries both identifying metadata and storage strategy.

- `name` is a required `String` and corresponds to the original file name returned by `file_picker` (for example, "sample_01.png").
- `bytes` is an optional `Uint8List` that holds the raw image bytes in memory when an in-memory workflow is required, for example when `SettingsData.loadImagesInMemory` is enabled.
- `path` is an optional `String` that points to the image file on disk, either to the original location or to a cached, processed copy stored via `CachedData.saveToCache`.

The methods on `ImgData` manage when and how bytes are present:

- `loadBytes()` loads bytes from path lazily only when bytes is null and a valid path is available; this prevents unnecessary I/O for images that are never displayed after processing.
- `getBytes()` returns bytes if already loaded, otherwise attempts to read from path, and finally returns null if neither is available; this function enables consumers to treat `ImgData` as a unified image source regardless of whether the image currently resides in memory or on disk.
- `clearBytes()` clears the in-memory bytes only when `SettingsData.loadImagesInMemory` is false, thereby releasing memory in cache-based workflows while intentionally keeping bytes resident when the user requests a fully in-memory mode.

Through this design, image storage remains flexible: the same `ImgData` type supports (a) path-only lazy loading for large datasets on low-RAM systems and (b) byte-only, path-less storage for fast repeated access when memory permits.

ImgProcessing: responsibilities and overall workflow

`ImgProcessing` encapsulates the complete pipeline from user file selection to a list of `ImgData` instances that refer to resized, optionally center-cropped and rotated images, with caching and timing-aware progress reporting.

Key responsibilities include:

- `pickFiles()` calls `FilePicker.platform.pickFiles` with filters for jpg, jpeg, and png, and with `withData` controlled by `SettingsData.loadImagesInMemory`, so that either file paths or raw bytes are obtained at selection time depending on the chosen memory mode.
- `_processImageConditionally()` performs decoding, rotation, cropping, and resizing of a single image, prioritising a dart:ui-based native path and falling back to `_resizeImageLegacy` only if a native error is encountered.

- `_batchMapAndProcessImages()` processes images sequentially (but batched) on a single isolate, applying optional resizing, reusing cached processed files by name, and updating a progress callback with estimated remaining time using `_updateProgressWithTimeEstimate`.
- `processImages()` chooses between the single-isolate batch path and a multithreaded path based on `SettingsData.multithreadImageProcessing`, and in multithreaded mode coordinates multiple `compute()` calls, receives incremental progress from worker isolates via a `ReceivePort`, and performs final cache writes in the main isolate.

The Settings page directly exposes the main knobs that drive this utility: target resize width and height, whether to resize while labeling, whether to center-crop and rotate images, whether to perform multithreaded processing, and whether to keep images in memory or use the processed-image cache. This integration turns `ImgProcessing` into a backend engine whose behaviour is fully controlled by user-visible configuration.

Native conditional processing with `dart:ui`

Decoding and setup via `dart:ui`

The static method `_processImageConditionally(Map<String, dynamic> params)` expects a parameter map containing the image bytes and target geometry:

- `params['bytes']` is the input `Uint8List` holding the compressed image (JPEG or PNG).
- `params['width']` and `params['height']` are the desired target dimensions.
- `params['centerCrop']` is a `bool` indicating whether a center crop should be applied to preserve aspect ratio before resizing.

Within `_processImageConditionally`, a nested function `centerCropResizeRotateNative` performs the actual image manipulation using the `dart:ui` API.

The first steps use Flutter's engine codecs:

```
final codec = await ui.instantiateImageCodec(bytes);
```

```
final frameInfo = await codec.getNextFrame();  
final originalImage = frameInfo.image;
```

This sequence performs the following operations:

1. `ui.instantiateImageCodec(bytes)` constructs an `ImageCodec` for the compressed bytes. The codec is implemented inside the Flutter engine and chooses an appropriate decoder (e.g. libjpeg, libpng, platform decoders), decoding the compressed stream into pixel buffers managed by the engine.
2. `codec.getNextFrame()` returns a `FrameInfo` object; for static images, this call yields the only frame.
3. `frameInfo.image` exposes a `ui.Image` handle, which represents a GPU-friendly image resource known to the rendering pipeline, with integer width and height.

By decoding directly into a `ui.Image`, subsequent operations such as drawing and scaling can be performed using the same pipeline that Flutter uses for rendering widgets, which is typically hardware-accelerated and avoids extra format conversions.

The code then extracts the source dimensions and computes aspect ratios:

```
final srcWidth = originalImage.width;  
final srcHeight = originalImage.height;  
final targetAspect = targetWidth / targetHeight;  
final srcAspect = srcWidth / srcHeight;
```

A rotation decision is made based on the orientation mismatch and the user's rotation preference:

```
final bool needsRotation =  
SettingsData.rotateToMatchResizeOrientation &&  
  ((srcAspect > 1 && targetAspect < 1) ||  
   (srcAspect < 1 && targetAspect > 1));
```

Here, a “landscape vs portrait” comparison is performed: a source with `srcAspect > 1` is landscape, `srcAspect < 1` is portrait, and a mismatch with the target aspect triggers rotation if `SettingsData.rotateToMatchResizeOrientation` is true.

Rotation + center crop branch

When needsRotation is true, a two-stage drawing pipeline is executed.

1. Rotation pass

A PictureRecorder and a Canvas are created:

```
final rotateRecorder = ui.PictureRecorder();  
final rotateCanvas = ui.Canvas(rotateRecorder);
```

Rotated dimensions are defined as swapped width and height:

```
final rotatedWidth = srcHeight;  
final rotatedHeight = srcWidth;
```

The canvas is translated and rotated:

```
rotateCanvas.translate(rotatedWidth.toDouble(), 0);  
rotateCanvas.rotate(pi / 2);
```

This sequence effectively rotates the coordinate system by 90 degrees around the top-left corner, but the initial translation moves the origin so that the rotated content still fits inside the positive quadrant of the canvas. In other words, the original image is drawn as if it had been turned clockwise or counter-clockwise, and the translation compensates for what would otherwise be negative coordinates.

The original ui.Image is then drawn into this rotated coordinate space:

```
rotateCanvas.drawImage(  
    originalImage,  
    ui.Offset.zero,  
    ui.Paint().filterQuality = ui.FilterQuality.high,  
);
```

The PictureRecorder is finalised and converted to a ui.Image of the rotated dimensions:

```
final rotatePicture = rotateRecorder.endRecording();  
final rotatedImage =  
    await rotatePicture.toImage(rotatedWidth, rotatedHeight);
```

The outcome of this step is a new `ui.Image` that has already been rotated, with consistent interpolation and color handling by the engine.

2. Center crop and resize pass

A second `PictureRecorder` and `Canvas` are created to perform cropping and scaling:

```
final cropRecorder = ui.PictureRecorder();  
final cropCanvas = ui.Canvas(cropRecorder);  
final rotatedAspect = rotatedWidth / rotatedHeight;
```

Source crop coordinates are initialised to cover the full rotated image:

```
double srcX = 0, srcY = 0,  
       srcW = rotatedWidth.toDouble(),  
       srcH = rotatedHeight.toDouble();
```

If center cropping is requested, the code branches depending on whether the rotated image is relatively wider or taller than the target:

- o When `rotatedAspect > targetAspect`, the rotated image is wider; the height is kept, the width is reduced to `srcH * targetAspect`, and `srcX` is set so that the crop window is horizontally centered.
- o When `rotatedAspect <= targetAspect`, the rotated image is relatively taller; the width is kept, the height is reduced to `srcW / targetAspect`, and `srcY` is chosen so that the crop window is vertically centered.

The selected region is then rendered into a rectangle of exact target dimensions in a single draw call:

```
cropCanvas.drawImageRect(  
    rotatedImage,  
    ui.Rect.fromLTWH(srcX, srcY, srcW, srcH),  
    ui.Rect.fromLTWH(0, 0, targetWidth.toDouble(),  
    targetHeight.toDouble()),  
    ui.Paint().filterQuality = ui.FilterQuality.high,  
);
```

Here, `drawImageRect` performs both cropping (by restricting the source `Rect`) and resampling (by mapping that rectangle to the destination rectangle) using the engine's scaling algorithms, with `FilterQuality.high` indicating a higher-quality sampling filter.

Finally, the recorded picture is converted to the final `ui.Image`, and then to PNG bytes:

```
final cropPicture = cropRecorder.endRecording();  
final finalImage = await cropPicture.toImage(targetWidth,  
targetHeight);  
rotatedImage.dispose();  
  
final byteData =  
    await finalImage.toByteData(format: ui.ImageByteFormat.png);  
finalImage.dispose();  
originalImage.dispose();  
  
return byteData!.buffer.asUint8List();
```

Resource disposal is performed explicitly for both `rotatedImage` and `finalImage`, as well as `originalImage`, to release underlying GPU memory when processing large batches.

Center crop + resize without rotation

When `needsRotation` is false, a slightly simpler path is used:

1. A `PictureRecorder` and `Canvas` are created.
2. The source crop window is initialised to the full original width and height, and then adjusted based on the comparison of `srcAspect` and `targetAspect` exactly as above, but directly on the original orientation.
3. A single `drawImageRect` call maps the chosen source rectangle to the target rectangle $(0, 0) \rightarrow (\text{targetWidth}, \text{targetHeight})$ with high filter quality.
4. The resulting `ui.Image` is converted to PNG bytes, and both `finalImage` and `originalImage` are disposed.

Error handling and legacy fallback

Both branches are wrapped in a try-catch. On any exception in the native path, `originalImage.dispose()` is still called to avoid resource leaks, an error message is logged, and the original bytes are returned as a safe fallback. At the outer `_processImageConditionally` level, a second try-catch surrounds the call to `centerCropResizeRotateNative`; on failure, `_resizeImageLegacy` is invoked with the same target width, height, and center-crop flag.

The legacy implementation uses the pure-Dart image package (`img.decodeImage`, `img.copyCrop`, `img.copyResize`) and then JPEG-encodes the result with quality 90. This path remains as a robustness fallback, while the normal flow relies entirely on

Batch and multithreaded processing: from `FilePickerResult` to `ImgData`

Single-isolate batched processing

The method `_batchMapAndProcessImages(FilePickerResult? pickedFiles, {Function(double, String)? onProgress})` handles the common case in which multithreading is disabled.

The workflow proceeds as follows:

1. If `pickedFiles` is null, null is returned, indicating that no files were selected.
2. `totalFiles` is computed, and `startTime` and an empty `processingTimes` list are initialised to support later time-estimate calculations.
3. A batch size `batchSize` is read from `SettingsData.imageProcessingBatchSize`, allowing the number of images processed concurrently (via `Future.wait`) to be tuned; batches are formed by slicing `pickedFiles.files` into sublists.
4. For each batch, a `Future.wait` over the batch's files is created. For each `PlatformFile`:
 - o Paths and in-memory bytes are checked; if both are null, the image is skipped.
 - o A per-file start time is recorded.

- o If `SettingsData.resizeImageWhileLabeling` is `true`, bytes are obtained either from `file.bytes` or via a disk read from `file.path`.
 - o A cache lookup is performed via `CachedData.pathOfFile("processed_${file.name}")`; if a processed file with that name prefix exists, an `ImgData` pointing to the cached path is returned, and progress is updated with a “Using cached file” message.
 - o Otherwise, `_processImageConditionally` is called with the configured `resize` width, height, and `SettingsData.centerCrop`.
 - o The elapsed time for this file is appended to `processingTimes`, and `_updateProgressWithTimeEstimate` is invoked to compute an updated remaining-time estimate and pass a progress fraction and message to the UI.
 - o If `SettingsData.loadImagesInMemory` is `true`, a new `ImgData` is created with bytes: `resizedBytes` and no path. Otherwise, the processed bytes are stored via `CachedData.saveToCache` with prefix `'processed_'`, and an `ImgData` with path: `outPath` is returned.
 - o On any exception, an `ImgData` pointing back to the original file path is returned, ensuring that a failure in processing does not break the labeling workflow.
5. Each batch’s non-null `ImgData` results are appended to `allResults`, and at the end of all batches, `allResults` is returned.

This approach keeps the logic simple while still avoiding blocking the UI thread thanks to `await` and per-batch `Future.wait`, and it leverages caching and progress estimation to provide both responsiveness and transparency to the user.

Multithreaded processing with isolates

When `SettingsData.multithreadImageProcessing` is `true` (and thus enabled from the Settings page on non-web platforms), `processImages` switches to a multi-isolate design that distributes file processing across CPU cores.

The multithreaded workflow can be summarised step by step:

1. If multithreading is disabled, processImages simply delegates to `_batchMapAndProcessImages`; otherwise, if `pickedFiles` is null, null is returned.
2. `totalFiles`, `startTime`, and `processingTimes` are initialised as in the single-threaded path.
3. A `ReceivePort` named `progressReceivePort` is created to receive incremental progress messages from worker isolates.
4. A local counter `processedCount` tracks how many files have been processed across all workers.
5. An inner function `processImageBatch(Map<String, dynamic> params)` is defined. It receives:
 - o `files`: a `List<PlatformFile>` for that worker.
 - o `width`, `height`, `shouldResize`, `centerCrop`, and `loadInMemory`, mirroring the global settings at the time of invocation.
 - o `workerIndex`: an integer identifying the worker.
 - o `sendPort`: the `SendPort` of `progressReceivePort`, used to send messages back to the main isolate.

Inside `processImageBatch`:

- o A list `results` is created.
- o For each file, bytes are obtained (if needed) from memory or disk, and `_processImageConditionally` is called when `shouldResize` is true; otherwise, an unmodified `ImgData` referencing the original bytes or path is created.
- o When `resizing` is off and `loadInMemory` is true, original bytes are preserved; when `resizing` is on and `loadInMemory` is false, `ImgData` objects carry processed bytes but a `nullpath`, to be complemented later in the main isolate (since workers cannot safely perform cache writes).

- o After each file, a message of the form {'workerIndex': workerIndex, 'processed': 1} is sent via sendPort, enabling the caller to track progress per file.
- o On error, an ImgData pointing back to the original path is added, and a progress message is still sent so that progress remains monotonic.

Finally, results is returned from each worker.

6. The main isolate determines workerCount as Platform.numberOfProcessors - 1, ensuring that at least one core is left free for the UI and other system tasks.
7. A filesPerWorker value is computed as ceil(totalFiles / workerCount), and the files list is partitioned accordingly into sublists [startIdx, endIdx). For each sublist, a compute(processImageBatch, {...}) call is enqueued, passing the file subset and the configuration parameters, and pushing the resulting Future<List<ImgData>> into workerFutures.
8. The progressReceivePort.listen handler increments processedCount by the processed value from each message and calls _updateProgressWithTimeEstimate with the updated counts and processingTimes, which now contains timing samples from the single-threaded preprocessing path or from previous runs.
9. After launching all workers, the main isolate awaits each future in workerFutures in a loop. For each workerResults list:
 - o If SettingsData.loadImagesInMemory is false, a second pass writes processed bytes to cache via CachedData.saveToCache for each ImgData whose bytes is non-null and path is null.
 - o Once a processed image has been written, image.path is updated with the cache path, and image.clearBytes() is called to minimise in-memory usage.
 - o All workerResults are appended to allResults.
10. After all workers have completed, the ReceivePort is closed, and a log message indicates total processed images; allResults is returned.

This design separates CPU-bound decoding and resizing (performed in worker isolates) from I/O-bound cache writing (performed in the main isolate), enabling multi-core use while keeping file-system access coordinated and respecting isolate boundaries.

Why dart:ui-based resizing is preferred over the legacy path

The native dart:ui-based method supersedes `_resizeImageLegacy` for several intertwined reasons related to performance, fidelity, and integration with the rendering engine.

1. Use of the engine's codecs and sampling

The `ui.instantiateImageCodec` API delegates decoding to Flutter's engine, which is optimised in C++ and usually leverages platform-native decoders for compressed formats. In contrast, the `package:image` path decodes and resamples entirely in pure Dart (`img.decodeImage`, `img.copyCrop`, `img.copyResize`), which imposes higher CPU and memory overhead, especially for large training images or large batches.

2. Single pipeline for rendering and processing

By working with `ui.Image`, the processing path uses the same internal pixel formats and sampling behaviour as the rest of the UI. The use of `Canvas.drawImageRect` with `FilterQuality.high` ensures that center-cropped and down-sampled images match what would be rendered on a Flutter canvas, preserving visual consistency between what the operator sees while labeling and what the exported training images look like.

3. Efficient rotation and crop in canvas space

Rotation is implemented by altering the canvas transform (translate + rotate) and re-rendering `originalImage`, rather than by manually rewriting pixel coordinates or performing an explicit rotation in software. The subsequent center crop and resize are combined into a single `drawImageRect` call for each image, which benefits from engine-level optimizations and avoids multiple passes over the pixel data.

4. Resource-aware behaviour

The `dart:ui` path explicitly disposes each intermediate `ui.Image` as soon as possible, ensuring that GPU memory is not unnecessarily held while many images are processed in sequence or in parallel. The legacy `package:image` code, by contrast, operates purely in main memory and relies on the garbage collector to reclaim large intermediate `img.Image` objects; this tends to be less predictable under heavy load.

5. Fidelity of aspect-ratio handling and orientation

In the native path, aspect ratio handling is expressed in terms of exact pixel rectangles mapped through the engine's transform matrices, which minimizes rounding artefacts and ensures that the post-rotation crop window matches the intended target aspect ratio as closely as possible. Orientation-based rotation is tied directly to `SettingsData.rotateToMatchResizeOrientation`, allowing a clear and deterministic rule: images whose orientation disagrees with the target are rotated once, then cropped and resized.

6. Legacy path as a controlled fallback

`_resizeImageLegacy` is still present and used only under failure conditions for the native path, providing a safety net without duplicating logic in normal operation. It uses a two-step process of center cropping followed by resizing with linear interpolation, and then encodes the result as JPEG for speed; however, this path is marked as legacy and is not preferred because it cannot exploit the engine's low-level optimisations.

In summary, `image_processing.dart` defines a layered architecture in which `ImgData` abstracts storage, `ImgProcessing` orchestrates decoding, rotation, center-cropping, resizing, caching, and multithreading, and `dart:ui` provides an efficient, engine-aligned implementation for the core image operations, with a pure-Dart legacy path retained only as a robust fallback.

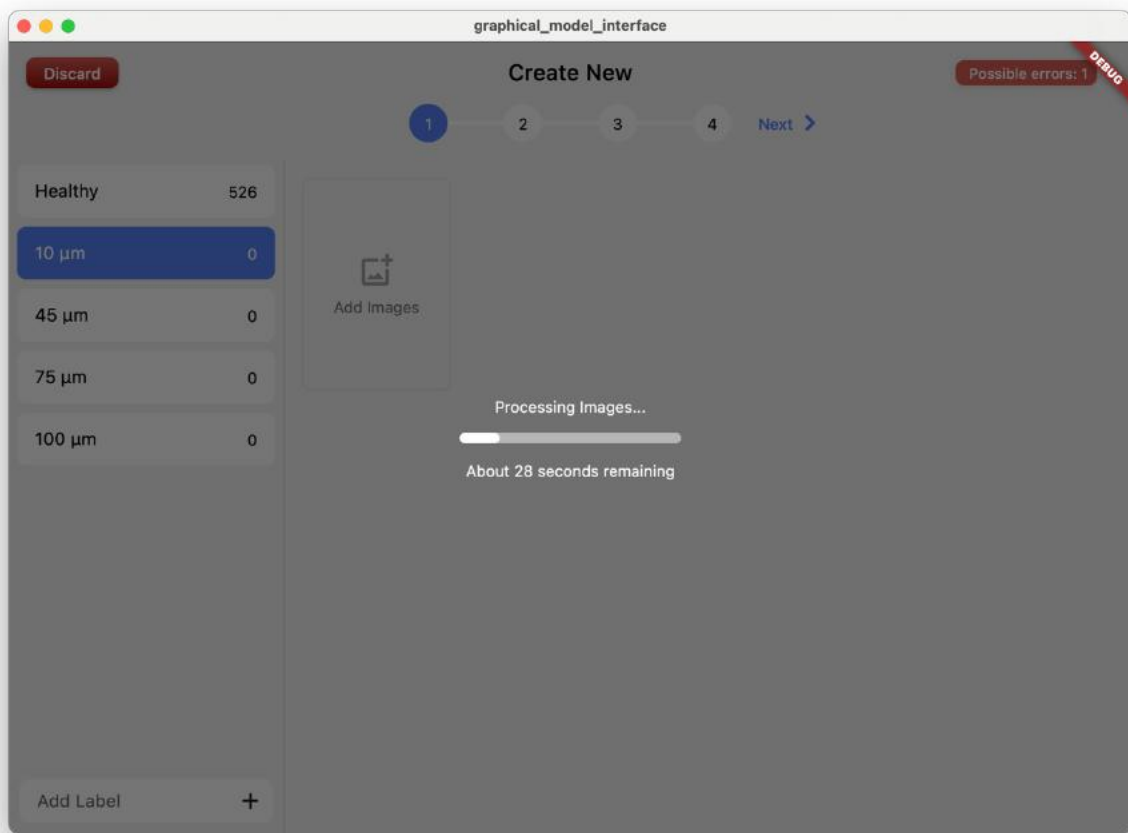


Fig 3.1 Popup shown while processing images

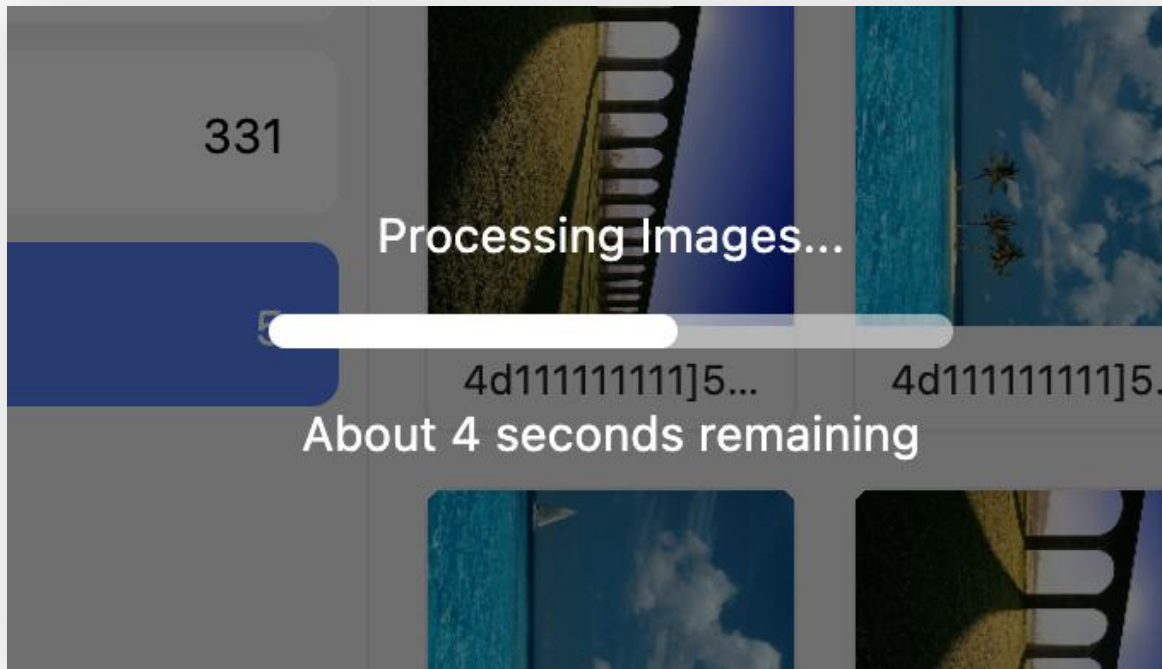


Fig 3.2 Processing images popup while adding images

3.1.4.2 Archive Service

Archive creation for MLVS 1 is handled by ArchiveService, which uses the Dart archivepackage to build ZIP archives entirely in memory and then either writes them to disk or returns them as bytes. The ZIP format is used so that the full training set directory tree (per-label subfolders and a metadata file) is encapsulated into a single compressed file that can be streamed as an application/zip payload to the MLVS 2 backend over HTTP.

ArchiveService and the Archive package

The archive package is employed as a pure-Dart encoder/decoder library that represents an archive as an Archive object containing multiple ArchiveFile entries, which are then converted into a ZIP-formatted byte buffer by ZipEncoder. In archive_service.dart, a new Archiveinstance is created for

each export, image files and a metadata.json manifest are added as ArchiveFile objects with logical paths (including folder separators), and the final byte sequence is produced by ZipEncoder().encode(archive) before being written to disk or returned to the caller.

ZIP is used as a container format that can hold a directory hierarchy of files with names, paths, and compressed contents, and that is widely supported across platforms and tooling. By placing all label folders and configuration metadata into one ZIP, a single HTTP upload or download is enabled, and the MLVS 2 server can reconstruct the exact directory structure and metadata needed for training without requiring multiple concurrent transfers or ad-hoc formats.

Desktop exportDataset workflow

On desktop platforms, ArchiveService.exportDataset is invoked from ExportPage._startExport, where it receives the LabeledData instance, the full modelConfigmap, the configHelper map, and an onProgress callback that expects a progress fraction and a flag indicating whether archive finalisation has started. Inside exportDataset, an Archive object is instantiated, the total image count is obtained via labeledData.getTotalImages(), and a unique filename such as <config_name>_<timestamp>.zip is constructed by sanitising modelConfig["config_name"] and appending the current millisecondsSinceEpoch.

If no images are present, an “empty dataset” path is followed in which only a metadata.json file is added to the archive; this JSON includes modelConfig, configHelper, the list of labels, a zero image count, a creation timestamp, and a human-readable note. The archive is then encoded with ZipEncoder().encode(archive) and written to a file located in a permanent subdirectory obtained via PermanentData.getSubdirectoryPath('dataset_exports'), with the resulting path returned to the caller and used later when saving the configuration into the local library.

For non-empty datasets, all labels in labeledData.mapOfLabelsToImages are iterated, a folder name is derived from each label by replacing spaces with underscores and converting to lowercase, and each ImgData in the label’s list is converted to bytes via image.getBytes() and inserted into the archive. For each image, a sanitized path of the form <folderName>/<index>_<originalName> is constructed using path.join, where spaces in the original file name are replaced with underscores and any characters outside word characters,

whitespace, and dots are stripped; an ArchiveFile with that path, the byte length, and the image bytes is then added to the Archive, and the onProgress callback is updated with the fraction `++processed / totalImages` and `isArchiveProcessDone = false`.

After all images have been added, a final progress update with `isArchiveProcessDone = true` is issued, and a richer `metadata.json` is appended that includes `modelConfig`, `configHelper`, the label list, per-label image counts (`image_counts`), and a fresh creation timestamp. The export directory is then resolved again, any existing file at the chosen path is deleted, the archive is encoded via `ZipEncoder().encode(archive)` and written to disk with `File(outputPath).writeAsBytes`, the in-memory Archive is cleared, and the final path is returned for use by `ExportPage` and by `PermanentData.saveModelConfig`.

Web in-memory export and ExportResult

For web builds, a disk path cannot be relied upon, so `ArchiveService.exportDatasetInMemory` is used instead; this method is called from `_startExport` when `kIsWeb` is true and returns an `ExportResult` object that holds the ZIP bytes and a suggested filename. The in-memory export follows the same logical structure as the desktop path but keeps the archive entirely in memory, updates a simpler `onProgress(double)` callback, and avoids any use of `dart:io` file APIs so that it remains compatible with the browser environment.

In the non-empty case, chunked processing is used: images are grouped into small chunks (for example of size 3), and each chunk is wrapped in an asynchronous task that adds its images to the shared Archive, updates the processed counter and progress fraction, and yields back to the event loop with `Future.delayed(Duration.zero)` after each chunk so that the browser UI remains responsive. When all chunks have been awaited, a `metadata.json` manifest with `model_config`, label list, and `image_counts` is added, a final progress value of 1.0 is reported, and the archive is encoded with `ZipEncoder().encode(archive)`, producing a `List<int>` that is wrapped into an `ExportResult(bytes: zipBytes, filename: 'dataset_export_<timestamp>.zip')`.

On the Export page, this `ExportResult` is later converted to an `XFile` using `XFile.fromData(Uint8List.fromList(_exportResult!.bytes), mimeType: 'application/zip', name: _exportResult!.filename)` and `file.saveTo("")` is invoked to trigger a browser download, so that the

user receives the same ZIP structure as on desktop even though no intermediate file existed on disk.

Archive folder structure and metadata contents

Within each generated ZIP archive, the root contains one metadata.json file and one subfolder per label, where the subfolder name is derived from the label text by replacing spaces with underscores and converting to lowercase. Inside each label folder, individual images are stored under filenames of the form <index>_<sanitizedOriginalName>, with the index corresponding to the position in the label's image list and the original file name sanitised to keep only word characters, whitespace, and the dot used in the file extension.

The metadata.json file on desktop encodes a JSON object with at least the keys modelConfig(the configuration sent to the training server), configHelper (helper structures such as UI constraints or defaults), labels (ordered list of label names), image_counts (map from label to number of images under that folder), and createdAt (ISO 8601 timestamp); in the empty-dataset case, totalImages and a note field describing the emptiness are substituted. On web, structurally similar information is written using snake_case keys (model_config, created_at), but the semantic content remains equivalent so that the MLVS 2 backend can interpret any archive consistently regardless of its origin.

Because a single ZIP file preserves both the folder hierarchy and the metadata manifest, the MLVS 2 backend is able to unpack the archive, validate that the directory layout matches the label and count information recorded in metadata.json, and feed the extracted tree directly into its training pipeline without relying on any external configuration apart from the uploaded JSON and dataset.

Integration with ExportPage and MLVS 2

The ExportPage widget orchestrates the entire export process by triggering ArchiveService.exportDataset or ArchiveService.exportDatasetInMemory in _startExport, monitoring _progress and _archiveProcessingDone to update the textual status between “Exporting...” and “Encoding and Writing...”, and finally offering a “Save As” button that either downloads the in-memory ZIP or copies the generated file to a user-selected location

via `file_selector`. After a successful desktop export, `_saveModelConfigToStorage` calls `PermanentData.saveModelConfig(labeledData, fullConfig, optionsConfig, configName, _exportFilePath!)`, so that the association between the configuration, its training-set archive path, and any later training jobs is persisted in the local model library.

Once the user chooses to send a training set to MLVS 2, this ZIP file is uploaded over HTTP as a single binary stream to the server's train endpoint, which unpacks the archive into a job-specific directory, reconstructs the label folders and image files, and reads `metadata.json` to retrieve the configuration and label statistics before launching the Celery-based training task. Through this design, `ArchiveService` provides a deterministic, reproducible packaging layer that bridges the graphical labeling workflow in MLVS 1 and the containerised, script-driven training workflow of MLVS 2 by means of a single, well-structured ZIP artifact per training set.

3.1.5 Storage System

3.1.5.1 Permanent Storage

Permanent storage in MLVS 1 is implemented around a dedicated application-support directory obtained from `getApplicationSupportDirectory()`, and is abstracted through the `PermanentData` utility so that configuration JSON files and exported training-set archives are stored in a controlled, non-user-facing location.

Application support directory and platform locations

Permanent data is stored under the directory returned by `PermanentData.getPermanentDir()`, which simply awaits `getApplicationSupportDirectory()` from the `path_provider` plugin. This directory is created automatically by the platform if it does not exist and is defined as the place where an application may put support files that are not intended to be directly exposed to end users.

Platform-specific locations are resolved by `getApplicationSupportDirectory()` as follows:

- On Windows, the path is resolved under `C:\Users\<user>\AppData\Roaming\<organization>\<app>`, so support files are placed inside the user's roaming application data rather than in Documents.
- On macOS (and iOS), the path corresponds to `NSApplicationSupportDirectory`, usually under `~/Library/Application Support/<bundle-id>` (or the sandboxed Container variant), which is the standard location for app support data.
- On Android, the directory is mapped to the engine's `getFilesDir()`, typically `/data/user/0/<package_name>/files`, which is private to the application sandbox.
- On Linux and other desktop platforms, the `path_provider` implementation uses a user-specific application data area, commonly `~/.local/share/<organization>/<app>`, again hidden from normal file pickers by default.

This directory type is chosen deliberately instead of `getApplicationDocumentsDirectory()`. `getApplicationDocumentsDirectory()` is documented as a location for user-generated data that should be visible and portable, whereas `getApplicationSupportDirectory()` is documented for support files that are not meant to be directly manipulated by the user and can be recreated or managed by the application. Because MLVS 1 wants to control model configurations and dataset archives entirely from within the app, and to avoid cluttering visible user folders, the application-support directory is used as the root for all permanent storage.

PermanentData: structure and responsibilities

The `PermanentData` class encapsulates all interaction with this permanent storage root. Its main responsibilities are as follows:

- `getPermanentDir()` returns the resolved `Directory` for application support using `getApplicationSupportDirectory()`.

- `getSubdirectoryPath(String subDirectory)` appends a named subdirectory (for example, "model_configs" or "dataset_exports") to the root path using `path.join`, creates it recursively if it does not exist, and returns the resulting path string.
- `saveFile(ByteArray bytes, String filename, {String subDirectory = ""})` writes arbitrary binary data into the permanent directory or a specified subdirectory, overwriting any existing file with the same name, and returns the full path of the written file.
- `pathOfFile(String filename, {String subDirectory = ""})` checks for the existence of a file under the root or a subdirectory and returns its path if present, or null otherwise, thereby allowing callers to test for known exports or configuration files without scanning directories manually.

Two key storage subtrees are created and managed:

- `model_configs`: used to store per-training-set JSON configuration bundles created by `saveModelConfig`.
- `dataset_exports`: used by `ArchiveService.exportDataset` to store ZIP archives containing the labeled images and `metadata.json` for each exported training set.

`saveModelConfig` is responsible for persisting a complete description of a training set: it constructs a combined JSON object containing `modelConfig`, `configHelper`, the list of labels, the total image count, a creation timestamp, a human-readable name, and the `exportFilePath` pointing to the corresponding dataset ZIP. This JSON is written under `model_configs` using a filename derived from the configuration name plus a millisecond timestamp (for example, `RiceConfig_2025-12-03T10-25.json`), ensuring uniqueness while remaining interpretable.

Configurations can be retrieved through:

- `loadModelConfig(String filename)`, which reads a specific JSON file from `model_configs` and returns it as a `Map<String, dynamic>?`.
- `getModelConfigs()`, which enumerates all .json files in `model_configs`, parses each into a map containing filename, user-visible name, `createdAt`, and the full config map, and sorts

the list by `createdAt` in descending order so that the most recent configs appear first in import dialogs.

Deletion and inspection are handled via:

- `deleteFile` and `deleteModelConfig`, which remove specific files by name, either from the root or the `model_configs` subdirectory.
- `getStorageSize({String subDirectory = ""})`, which recursively traverses a directory and sums the sizes of all files to compute total usage, and `formatFileSize(int sizeInBytes)`, which renders that count into B, KB, MB, or GB strings.
- `getFormattedStorageSize({String subDirectory = ""})`, which combines both to provide a single human-readable capacity figure per subtree (for example, for "model_configs" alone or for the entire support directory).

Finally, `clearSubdirectory({String subDirectory = ""})` provides a controlled wipe operation: for a chosen subtree (or the entire support directory if `subDirectory` is empty), the directory is deleted recursively and then re-created empty, returning a boolean to indicate success or failure. This method is disabled on web (`kIsWeb` guard) because the underlying `dart:io` filesystem is not available there.

Integration with the Settings “Storage” section and deletion safeguards

The `SettingsPage` exposes this permanent storage system in its “STORAGE” section, allowing users to inspect and manage disk usage for configurations and exports without manually navigating into OS-specific directories. A `FutureBuilder` named `_appDataSizeDisplay()` calls `PermanentData.getFormattedStorageSize(subDirectory: 'model_configs')` and `PermanentData.getFormattedStorageSize()` to display both “Model configurations” size and “Total permanent storage data” size within a `ListTile` titled “Permanent Storage”.

A `GradientButton` labeled “More” opens an `AlertDialog` describing this permanent data and presenting two destructive actions guarded by a fail-safe mechanism. Inside the dialog:

- A TextField is presented, and the user is instructed to type the exact phrase "DELETE"; until this condition is met, both destructive buttons remain disabled.
- The first button, "CLEAR ALL", calls `_clearAppData("")`, which invokes `PermanentData.clearSubdirectory(subDirectory: "")`. This operation clears the entire application-support tree used by the app, including both `model_configs` and `dataset_exports` and any other permanent files, and then increments `_cacheRefreshKey` to force the size display to refresh.
- The second button, "KEEP CONFIGS AND CLEAR THE REST", calls `_clearAppData('dataset_exports')`, which clears only the `dataset_exports` subtree while leaving the `model_configs` directory intact.

The first option is useful when a complete reset of the local library is desired, including removal of all archived training sets and their associated configuration bundles; the second option is targeted at scenarios where a user wants to preserve configuration JSON files for reuse (for example, importing a previous architecture and hyperparameter configuration into a new training-set creation) but no longer needs the corresponding dataset ZIPs that occupy more disk space.

Both deletion actions are executed on non-web platforms only, and each completion path updates `_cacheRefreshKey` and shows a Snackbar indicating whether the operation succeeded or failed, so that changes in permanent storage are immediately visible in the “Permanent Storage” size display. In this way, permanent data storage is centralised under the application support directory, hidden from casual user access, but still fully inspectable and controllable from within MLVS 1 via a structured API (`PermanentData`) and a guarded UI workflow (the Settings “Storage” dialog).

3.1.5.2 Cache

The caching system in MLVS 1 is implemented so that expensive, deterministic image-processing steps are not repeated unnecessarily; instead, derived image files are written once into a temporary cache directory and reused when possible, which reduces processing time dramatically when images are revisited or reused across training sets. Caching is applied only on desktop platforms

when `SettingsData.loadImagesInMemory` is false, so that disk-backed cache files act as a middle layer between the original images and the resized, center-cropped, and rotated versions that the labeling UI needs.

Temporary directory and its platform mapping

All cached files are stored under the directory returned by `CachedData.getTemporaryDir()`, which internally calls `getTemporaryDirectory()` from the `path_provider` plugin. According to the `path_provider` documentation, `getTemporaryDirectory()` returns a directory intended for caches of downloaded or generated files that are not backed up and may be cleared by the operating system at any time; this directory is scoped to the application and is not shared with other apps.

The concrete path of this temporary directory is platform-dependent:

- On Android, the path corresponds to `Context.getCacheDir()`, yielding a location such as `/data/user/0/<package_name>/cache`, which is private to the application sandbox and may be purged by the system under storage pressure.
- On iOS and macOS, it maps to `NSCachesDirectory`, typically under `~/Library/Containers/<bundle-identifier>/Data/Library/Caches`, which is reserved for transient data and can be cleared by the OS or by the user through system tools.
- On Windows, empirical observations reported for `getTemporaryDirectory()` show paths under `C:\Users\<user>\AppData\Local\Temp`, which serve as the standard per-user temp area; `path_provider` creates an application-specific subdirectory there.
- On Linux and similar desktop environments, the path usually resides under `/tmp` or an equivalent user-level cache directory, again not intended for long-term persistence and subject to periodic cleanup.

This design ensures that cache files are treated as disposable by the platform, in contrast to the application-support directory used for permanent data; as a result, the cache can be regenerated at any time from the original images and configuration if the OS or the user removes it.

CachedData

The `CachedData` class encapsulates all access to this temporary storage and provides the following core methods.

- `getTemporaryDir()` returns the `Directory` obtained from `getTemporaryDirectory()`, acting as the root of the cache.
- `pathOfFile(String filename)` constructs a full path under the temp directory using `path.join(tempDir.path, filename)`, checks whether that file exists, and returns the path or null accordingly; this function is used by the image-processing pipeline to determine whether a processed file for a given image name is already present in the cache.
- `saveToCache(UInt8List bytes, String originalFilename, {String prefix = 'processed_'})` writes processed image bytes into the temp directory. It first reduces `originalFilename` to its `basename`, then builds an output path of the form `<tempDir>/<prefix>_<basename>` using `path.join`. If a file already exists at that path, the existing path is returned without rewriting; otherwise, the bytes are written to a new file and that path is returned.

To support monitoring and management from the Settings page, several size-calculation and formatting helpers are defined:

- `_getProcessedImagesCacheSize({String pattern = 'processed_'})` scans the temporary directory and sums the byte sizes of files whose path contains the given pattern (by default `'processed_'`), which isolates the MLVS 1 image-processing cache from unrelated temporary files.
- `_getTotalCacheSize()` computes the size of all files under the temporary directory without filtering, giving a complete view of how much space is being used in that location.
- `_formatFileSize(int sizeInBytes)` converts the raw size into a human-readable string in B, KB, MB, or GB, using simple thresholds at 1024 , 1024^2 , and 1024^3 bytes.

- `getFormattedProcessedCacheSize({String pattern = 'processed_'})` and `getFormattedTotalCacheSize()` call these size functions and return formatted strings to be displayed in the Settings UI.

Cache clearing is implemented in `clearCache({String pattern = 'processed_'})`, which first returns false on web (where `dart:io` paths are not available), then clears Flutter's in-memory image cache via `PaintingBinding.instance.imageCache.clear()` and `.clearLiveImages()`, and finally attempts to delete files on disk. All files in the temp directory whose paths match the given pattern (or all files if pattern is empty) are deleted in a first pass; failed deletions are accumulated, and on Windows a retry loop with exponential backoff (200 ms, 400 ms, 800 ms) is used to handle transient file locks before returning success if any files were deleted. This robust deletion strategy is important because temporary directories on Windows often contain files that can briefly be locked by the OS or anti-virus software.

Time complexity of image processing with caching

Caching is used in the image-processing pipeline as a way to avoid repeating the same CPU- and I/O-intensive resizing operations when the same original images are reused with unchanged processing settings. When images are labeled and `SettingsData.resizeImageWhileLabeling` is true and `SettingsData.loadImagesInMemory` is false, the following workflow is used in `_batchMapAndProcessImages` within `ImgProcessing`.

1. For each `PlatformFile` selected from disk, bytes are loaded from `file.bytes` or `File(file.path!).readAsBytes()`.
2. Before any processing is performed, `CachedData.pathOfFile("processed__${file.name}")` is called to check whether a processed image with a name prefixed by "processed__" plus the original filename already exists in the temporary directory.
3. If `pathOfFile` returns a non-null path, the cache hit is used: an `ImgData` with `path: pathOfExistingFile` is returned immediately, and progress is updated with a "Using cached file" message; no decoding, rotation, or resizing work is performed, so the operation is essentially reduced to a directory lookup and object creation.

4. If the file is not present in the cache, `_processImageConditionally` is invoked to perform the full `dart:ui` decoding, optional rotation, center cropping, and resizing, and the resulting bytes are passed to `CachedData.saveToCache(resizedBytes, file.name, prefix: 'processed_')`.
5. The returned cache path is then stored in a new `ImgData`, and the bytes may be cleared later when no longer needed, so that subsequent runs can re-open the same processed image directly from disk via the cache.

This design means that the cache is keyed by the original filename (with a fixed prefix) rather than by a hash of the image content. A cryptographic hash of the image pixel data (for example, SHA-256 over the decoded bitmap) would guarantee uniqueness even when different images share the same name, but it would require reading and processing every image fully just to compute the hash, which would negate much of the benefit of caching in the first place. Instead, a pragmatic trade-off is used: filenames are assumed to be reasonably unique in typical lab workflows, and the Settings page explicitly warns that if a new upload uses a filename that matches a cached file, the cache entry will be reused and processing will be skipped.

This filename-based strategy provides two important advantages:

- Cache lookups and writes are $O(1)$ operations that involve only path manipulation and `File.exists()` calls, which are fast compared with image decoding and scaling, especially for high-resolution microscope or camera images.
- The approach integrates naturally with the user’s mental model: if files are reused under the same name, the previously processed version is reused; if a completely fresh preprocessing is desired, files can be renamed or the cache can be cleared.

In MLVS 1, caching therefore acts as a performance layer that maps from “original filename + current resize settings” to “processed image file in the temp directory”, with the understanding that the second part is only valid while the relevant settings remain unchanged.

Cache control, Settings integration, and automatic invalidation

The caching system is surfaced in the Settings page under the “MEMORY” section. The `_cacheSizeDisplayAndClear()` widget uses `CachedData.getFormattedProcessedCacheSize()` and `CachedData.getFormattedTotalCacheSize()` to show two values: “Processed images” (sum of all files whose path contains 'processed_') and “Total cache” (sum of all files in the temp directory). A “Clear” `GradientButton` opens a Clear Cache dialog explaining that frequent reuse of images can make processing dramatically faster and asking for confirmation; the CLEAR ALL CACHE action calls `_clearTemporaryFiles()`, which in turn invokes `CachedData.clearCache(pattern: ")` to delete all files in the temporary directory and reset the counts.

The Settings page also explains an important behavioral detail: “Items stored in the cache use their original file names. Hence, if you upload a file having a name that matches with that of an existing file in the cache, it will be skipped, and the file in the cache will be used instead. In such cases, you might either want to use unique file names or clear the cache before processing such a file.” This text makes explicit why filename-based keys are used and what implications this has for users who reuse filenames for different image content.

Crucially, the cache is automatically invalidated when key image-processing parameters are changed. In the Settings UI:

- Toggling “Resize images while labeling” or editing the resize width or height fields sets the `_keyParamsChagned` flag to true.
- Toggling “Center crop images to avoid squeezing” or “Rotate images” also sets `_keyParamsChagned` to true, as these options affect the geometry of the processed images.
- When the user presses “Save and Exit”, the app bar’s handler calls `_clearTemporaryFiles()` if `_keyParamsChagned` is true before persisting the new settings via `SettingsData.save()`.
- `_clearTemporaryFiles()` uses `CachedData.clearCache(pattern: ")` to remove all cached files and forces a refresh of the cache-size display by incrementing `_cacheRefreshKey`.

This automatic invalidation is essential because a cache entry processed for one set of resize parameters (for example, 256×256 center-cropped without rotation) must not be reused when the user requests a different target resolution or a different crop/rotation policy; otherwise, the cache would silently serve images with incorrect dimensions or aspect ratios. By clearing the cache whenever any of these key parameters changes, the system guarantees that cache hits always correspond to the current configuration, and that any skipped processing reflects a valid reuse of images that have already been transformed exactly as specified.

Through this design, the cache in MLVS 1 operates as a transparent, configuration-aware acceleration layer built on top of the OS-provided temporary directory, with clear semantics, robust deletion, and explicit UI affordances that allow users to understand and control its behavior.

3.1.6 Packaging

MLVS 1 is packaged as a native Flutter desktop application by invoking flutter build for each target platform and then wrapping the resulting binaries in platform-appropriate distribution formats. For macOS, the Flutter tool produces a .app bundle that can be distributed directly or further wrapped as a .dmg, and for Windows the tool produces an .exe and associated runtime files that have been bundled into a proper installer using Inno Setup.

Flutter’s desktop support compiles the Dart code ahead-of-time into native machine code and links it with a small platform-specific runner, producing a self-contained application bundle under the project’s build directory. The developer enables desktop support (flutter config --enable-macos-desktop, --enable-windows-desktop during setup) and then uses platform-specific flutter build subcommands such as flutter build macos or flutter build windows to produce optimized release binaries.

Each flutter build run generates:

- A platform-specific executable (for example, the macOS app binary or Runner.exe on Windows).
- A data directory and dynamic libraries (.dll on Windows) containing the Flutter engine and compiled assets, which must be shipped together with the executable.

These build artifacts are then either distributed as-is (for development or internal testing) or embedded in an installer or disk image for end users.

macOS

On macOS, flutter build macos is used to generate a release-ready application bundle. After running this command in the MLVS 1 project root, Flutter produces a .app bundle in a release output directory such as:

- build/macos/Build/Products/Release/<AppName>.app

This .app is a standard macOS application bundle that contains:

- Contents/MacOS/<AppName>: the native executable that embeds the Flutter engine and the compiled Dart code.
- Contents/Frameworks/: dynamic libraries required by Flutter and any plugins.
- Contents/Resources/flutter_assets/: the compiled Flutter asset bundle, including fonts, images, and other resources.
- Contents/Info.plist: the application's metadata (bundle identifier, version, entitlements), which is configured by Flutter's macOS template and can be customized if required.

This application bundle can be run directly by double-clicking or via open build/macos/Build/Products/Release/<AppName>.app and is suitable for distribution as a standalone .app (for example, via direct download or manual copying). If a more polished macOS distribution format is required, this .app can additionally be wrapped into a .dmg disk image using tools such as appdmg, or signed and notarized for App Store or Gatekeeper distribution, but the core packaging step for MLVS 1 is the production of the .app bundle via flutter build macos.

Windows

On Windows, `flutter build windows` is used to generate a native desktop executable and its runtime environment. Running this command in the MLVS 1 project root yields build artifacts under a directory such as:

1. `build\windows\runner\Release\`

Within this directory, Flutter generates:

- `<AppName>.exe`: the main Windows executable that embeds the Flutter engine and compiled Dart code.
- A data directory holding assets and engine resources required at runtime.
- Several `.dll` files corresponding to the Flutter engine and plugin binaries.

These files together form the minimal set required to run the application on another Windows system; copying only the `.exe` is insufficient because it depends on the data directory and DLLs at the same relative paths. For quick, informal distribution, the entire `Release` folder can be zipped and shared, but this approach is not ideal for end users because it exposes internal files and lacks an installation experience.

To provide a professional Windows installer, MLVS 1 has been packaged further using Inno Setup. Inno Setup is a scripting-based installer framework that takes an existing application directory (containing the `.exe`, data folder, and DLLs) and builds a single `.exe` setup program, which guides the user through installation steps and installs files under Program Files or another designated location. The typical workflow followed is:

1. Build the Flutter Windows app in release mode via `flutter build windows`.
2. Use the resulting `build\windows\runner\Release` directory as the source for the installer, specifying `<AppName>.exe` as the main application executable in the Inno Setup script.
3. Configure installer metadata in the `.iss` script (application name, version, publisher, icon, target install directory, Start Menu entries, and optional desktop shortcuts).

4. Compile the .iss script with Inno Setup to produce a single Setup.exe installer that can be distributed to users.

When this installer is run, it handles copying all necessary files, setting up Start Menu and possibly desktop shortcuts, and optionally registering an uninstaller entry, so that MLVS 1 appears as a first-class Windows application from the user's perspective. This packaging approach hides internal build artifacts from the user, enforces a clean installation directory layout, and simplifies updates and uninstallation.

Other supported platforms and build commands

Although production packaging has been carried out and tested primarily for macOS and Windows, the same Flutter project structure allows builds for other platforms using analogous commands.

- For Linux, `flutter build linux` produces an ELF executable and associated runner files under `build/linux/x64/release/bundle`, which can be distributed as a directory bundle or wrapped into a `.deb`, `.rpm`, or other package by external tooling.
- For the web, `flutter build web` generates a `build/web` directory containing static HTML, JavaScript, and asset files that can be hosted on any web server, although in the MLVS 1 context the web build is described as having limited functionality compared with desktop.
- For mobile (Android and iOS), `flutter build apk`, `flutter build appbundle`, and `flutter build ios` produce platform-specific packages (APK/AAB and an Xcode archive), but MLVS 1 has been optimized mainly for desktop usage.

In each case, the conceptual pattern remains consistent: Flutter compiles the Dart application into a platform-native binary plus a bundle of assets, and external packaging tools (App Store mechanisms, `.dmg`, `.msix`, Inno Setup, or Linux packagers) can be applied on top of these outputs when a polished distribution channel is required.

Rationale for the chosen packaging approach

The chosen packaging strategy for MLVS 1 leverages Flutter's built-in desktop build system to produce reproducible, optimized binaries for each supported OS and then layers platform-appropriate installers on top for better user experience. On macOS, distributing the .app bundle directly is sufficient for research and lab deployment scenarios, while still enabling future extension to .dmg images and notarised distribution if needed. On Windows, using Inno Setup provides a single Setup.exe entry point that encapsulates all of Flutter's Release output, simplifying installation, updates, and removal for non-technical users, which aligns with the goal of making MLVS 1 accessible beyond developers.

3.2 MLVS 2

3.2.1 Docker

Docker is used in MLVS 2 as the core isolation and deployment mechanism for the FastAPI backend, the Celery training workers, and the Redis message broker, so that the complete training server can be started reproducibly with a single docker compose up command on a GPU-enabled Linux host. Each logical role (API, worker, broker) is packaged into a separate image with its own dependencies and runtime configuration, and the services are connected through an internal Docker network and shared volumes.

Docker is based on Linux container primitives (namespaces and cgroups) and is used to run one or more processes in an isolated filesystem and process space using a lightweight “container” rather than a full virtual machine. In this model, an immutable image is defined (typically by a Dockerfile) that contains a root filesystem with exactly the operating system libraries, Python interpreter, and Python packages that are required; at runtime, a container is started from this image and executes the configured entrypoint command. Because images are built in layers, with base images such as python:3.11-slim or tensorflow/tensorflow:2.15.0-gpu reused across builds, network and storage overheads during rebuilds are reduced and version pinning is made explicit.

Role

In contrast to running Python and TensorFlow directly on the host, containerisation ensures that the version of TensorFlow, the CUDA stack, and all Python dependencies used for training are defined declaratively in the images and do not conflict with any other software installed on the same server. This isolation is particularly important for ML training workloads, because mismatches between TensorFlow, CUDA drivers, and system libraries are common failure modes if environments are managed manually. In addition, Docker’s networking model allows each service (API, worker, broker) to be given a stable logical hostname and port inside a private network, while still exposing only the public HTTP interface that is needed by the MLVS 1 client.

The docker-compose.yaml file defines three services: redis, api, and worker, along with a named volume redis_data and a custom bridge network ml-network. The redis service runs the official redis:7-alpine image, exposes port 6379, persists its keyspace under the redis_data volume, and is monitored by a healthcheck that regularly runs redis-cli ping; both the API and the worker depend on this health status before starting.

The api service is built from the api.Dockerfile in the docker directory using the ./app folder as the build context; it exposes port 8000 on the host, mounts ./jobs into /var/ml-server/jobs for job artifacts and ./app into /app for the application code, and receives a REDIS_URL=redis://redis:6379/0 environment variable so that the FastAPI process can enqueue Celery tasks into the Redis broker. The worker service is built from worker.Dockerfile using the same ./app context, mounts the same ./jobs and ./app volumes to see the same code and job directories, reads the same REDIS_URL, and is configured to start a Celery worker process with celery -A celery_app worker --loglevel=info

Containerization of the API and worker

The API container image is defined in api.Dockerfile by starting from python:3.11-slim, setting /app as the working directory, and creating a non-root user appuser with UID 1000 to improve security. The requirements.txt file is copied and installed first, which allows Docker's build cache to reuse previously built layers when only the application code changes; then the rest of the application (including FastAPI code, templates, and Celery configuration) is copied into /app owned by appuser.

A healthcheck is configured in api.Dockerfile so that Docker repeatedly executes a small Python one-liner that imports requests and queries the /health endpoint at http://localhost:8000/health, failing the container if this probe does not succeed; this health status is propagated through depends_on in docker-compose.yaml to govern startup ordering. The container's entrypoint command is uvicorn main:app --host 0.0.0.0 --port 8000, which starts the FastAPI application so that the MLVS 1 client can connect to http://<server>:8000 for health checks, job submission, and job status queries.

The worker container image is defined in `worker.Dockerfile` by starting from `tensorflow/tensorflow:2.15.0-gpu`, which already contains a compatible CUDA stack and TensorFlow GPU build, and then installing `redis-tools` at the OS level for debugging and monitoring. A non-root appuser is again created, `requirements.txt` is copied and installed on top of the base TensorFlow image (including `async-timeout>=4.0.0` for Celery's Redis transport), and the application code is copied into `/app`, giving the worker access to the same `tasks.py`, `celery_app.py`, and training script templates as the API. [file:592689eb-bbf0-4568-b3dd-89e499762e02]

GPU access for training jobs is configured by setting environment variables `NVIDIA_VISIBLE_DEVICES=all`, `NVIDIA_DRIVER_CAPABILITIES=compute,utility`, `TF_FORCE_GPU_ALLOW_GROWTH=true`, and `TF_CPP_MIN_LOG_LEVEL=2` inside the worker image, and by requesting NVIDIA devices in the `deploy.resources.reservations.devices` section of `docker-compose.yaml` with `driver: nvidia`, `count: all`, and `capabilities: [gpu]`. The worker container's default command is `celery -A celery_app worker --loglevel=info`, so that when the service is started, a Celery worker is attached to the Redis broker and waits for training tasks submitted by the API.

Benefits

Through this containerised design, the MLVS 2 backend can be started on any compatible Ubuntu server with Docker and the NVIDIA Container Toolkit installed, without any manual installation of Python, TensorFlow, FastAPI, or Celery on the host. The same `docker-compose.yaml`, `api.Dockerfile`, and `worker.Dockerfile` define exactly the same environment, directory layout, ports, and GPU configuration in development and production, which improves reproducibility and reduces configuration drift.

Separation into three services introduces clear isolation of concerns: Redis runs as a minimal, hardened key-value store in its own container, the API container runs only the HTTP interface and job orchestration logic, and the worker container runs the heavy-weight training jobs with direct GPU access; failures in one service do not corrupt the environments of the others. Shared bind mounts for `./jobs` and `./app` allow the API and worker containers to read and write the same training archives, generated scripts, logs, and trained models under `/var/ml-server/jobs`, while still

keeping this directory under explicit control from the host and making backup or inspection straightforward.

Finally, Docker's restart policies (restart: unless-stopped on all three services) and healthchecks on Redis and the API add robustness: containers are restarted automatically if they crash, and dependent services are only brought up when the broker is confirmed healthy, which prevents common race conditions during startup. This combination of isolation, GPU-aware configuration, reproducible builds, and orchestrated multi-service deployment makes Docker a natural fit for MLVS 2's training backend, which must run resource-intensive jobs while remaining easy to deploy and maintain for non-expert operators.

3.2.2 FastAPI

FastAPI is used as an HTTP interface layer that receives training jobs from MLVS 1, validates and stores the uploaded artifacts, and coordinates with Celery workers and Redis to run long-running training tasks asynchronously. It is chosen because it provides a high-performance, type-driven API framework in Python with first-class support for async I/O, automatic schema generation, and clean request validation, which suits a training backend that must handle large uploads and concurrent clients reliably.

In MLVS 2, a FastAPI application instance is created in `main.py` with a descriptive title and version, and is configured with a CORS middleware that allows cross-origin requests from the Flutter desktop client. A persistent Redis connection is established in the same module and is used by the FastAPI endpoints to store and retrieve job metadata under keys such as `job_<job_id>`, keeping the HTTP layer stateless apart from this key–value store. The application is started inside the api Docker container by running Uvicorn with `uvicorn main:app --host 0.0.0.0 --port 8000`, which exposes the FastAPI app on port 8000 for MLVS 1.

Handling uploads and long-running jobs

A `/health` endpoint is defined as a simple GET route that returns a JSON object indicating that the ML training server is healthy; this endpoint is used both by Docker health checks and by the Flutter settings page for connectivity testing. The central `/train` endpoint is defined as a POST route that

accepts two UploadFile parameters (config and dataset) using FastAPI's file-upload mechanism, enforces that the first is a JSON file and the second is a ZIP file, writes them into a newly created per-job directory under `/var/ml-server/jobs/job_<job_id>`, and then enqueues a Celery task (`train_model.delay(job_id)`) to perform the actual training. Additional endpoints `/job/{job_id}`, `/jobs`, `/job/{job_id}/download/{file_type}`, and `/job/{job_id}/download-complete` are provided to query job status, list all jobs, and stream back individual artifacts or a ZIP archive of all outputs, mapping HTTP calls from MLVS 1 directly onto the filesystem structure and Redis state maintained by the worker.

Benefits

By using FastAPI, strict validation of incoming job requests is achieved (for example, rejecting non-JSON configs or non-ZIP datasets with clear HTTP 400 responses), which prevents malformed jobs from ever reaching the training queue. The automatic OpenAPI schema and interactive documentation that FastAPI provides at `/docs` and `/redoc` can be leveraged during development and debugging to inspect the exact request and response formats without additional tooling, which shortens iteration time when evolving the MLVS 1–2 protocol. Because FastAPI is asynchronous and built for high concurrency, the API container can remain responsive to health checks, job listings, and result downloads even while multiple heavy training tasks are running in separate worker containers, which matches the requirement that the graphical front-end should stay responsive while the backend executes long-running GPU-bound workloads.

3.2.3 Redis

Redis is used as a central in-memory service for both job queuing (via Celery's message broker) and lightweight job metadata storage, which enables fast coordination between the FastAPI layer and the GPU training workers. It is deployed as a dedicated Docker service (`redis:7-alpine`) and is accessed from Python code using the official redis client library over the internal Docker network.

Redis (Remote Dictionary Server) is an open-source, in-memory data structure store that exposes a simple key–value API but supports rich value types such as strings, lists, sets, sorted sets, and hashes. Data is kept primarily in RAM, which allows read and write operations to be served with

very low latency; optional persistence mechanisms can be enabled but are not required for transient workloads such as task queues and progress tracking. Because commands operate on these data structures atomically on a single thread, Redis provides predictable semantics without complex transactions, which is well suited for simple counters, queues, and small hash records that change frequently.

Deployment and connectivity

In the MLVS 2 deployment, Redis is defined as a separate service in `docker-compose.yml` using the `redis:7-alpine` image, and is exposed on port 6379 with a persistent volume `redis_data` mounted at `/data` for durability of the keyspace. A healthcheck periodically runs `redis-cli ping` inside the container to confirm availability; both the `api` service and the `worker` service declare `depends_on` with `condition: service_healthy`, so they only start after Redis has become responsive.

Within the FastAPI application (`main.py`), a Redis client is created with `redis.Redis(host='redis', port=6379, db=0, decode_responses=True)`, using the Docker service name `redis` as the hostname so that all containers can refer to the same broker. The Celery worker code (`tasks.py`) uses an identical connection definition, so both the HTTP layer and the background training tasks share the same logical view of the job store and can read and write to keys such as `job_<job_id>` consistently.

Redis as Celery broker for training jobs

Celery is configured (via `celery_app.py` and environment variable `REDIS_URL=redis://redis:6379/0`) to use Redis as its message broker, meaning that every training job submitted from FastAPI is serialized into a message and pushed into a Redis queue. When `train_model.delay(job_id)` is called in the `/train` endpoint, Celery encodes the task name, arguments (here, the `job_id` string), and metadata into a JSON payload and enqueues it into a Redis list that represents the default task queue; one or more Celery workers then block on that list, pop tasks as they become available, and execute them in the GPU-enabled worker container.

This architecture offloads long-running, GPU-bound training work from the HTTP request/response cycle. The Redis broker ensures that tasks persist across restarts of the API

container and that they can be distributed among multiple workers if horizontal scaling is introduced later, while still keeping the setup simple (a single Redis instance and a single logical queue) for the current deployment.

Redis as job metadata and status store

Beyond its role as a Celery broker, Redis is used directly as a key–value store for job metadata and status, using hash structures under keys of the form `job_<job_id>`. When a new training job is created in `/train`, the server calls `redis_client.hset(f'job_{job_id}', mapping={...})` to store fields such as `task_id`, `status`, `created_at`, `config_name`, `model_type`, `epochs`, and `batch_size`; this creates a small hash record that can be retrieved later in constant time.

The `train_model` task in `tasks.py` uses the same pattern for status updates: when the job begins processing, the hash field `status` is set to `"PROCESSING"`, and once training completes successfully, the same field is updated to `"COMPLETED"`. In case of an exception, the task stores a failure state and writes additional fields such as `error` and `error_type` into the same Redis hash before returning an error result, so that failure information remains available even if Celery's built-in result backend is not consulted.

High-level API endpoints build on this stored metadata:

- `/job/{job_id}` calls `redis_client.hgetall(f'job_{job_id}')` to fetch the stored hash and then augments it with the current Celery task state (`PENDING`, `PROGRESS`, `SUCCESS`, `FAILURE`) by querying `celery_app.AsyncResult(task_id)`.
- `/jobs` lists all keys matching `"job_*`" using `redis_client.keys("job_*")`, then calls `hgetall` per key to construct a compact list of job summaries with fields like `status`, `creation time`, and `configuration name`.

This design uses Redis as a fast, shared, canonical store of job metadata that both FastAPI and Celery can access without touching the filesystem, and without introducing a heavier relational database.

Benefits

Using Redis in this dual role has several concrete advantages for MLVS 2. Because Redis is an in-memory key–value store, job status lookups and list operations execute with very low latency, which keeps the UI of MLVS 1 responsive when polling `/jobs` and `/job/{job_id}` for updates. The same Redis instance serves as both Celery’s broker and the metadata store, so the deployment consists of a single, lightweight service that is easy to operate inside Docker while still supporting future scaling (for example, adding more workers or sharding Redis if throughput requirements increase).

Finally, the data model used in Redis is deliberately simple (one hash per job, plus Celery’s own queue keys), which reduces the risk of schema drift and makes debugging straightforward—keys and fields can be inspected interactively with `redis-cli` if necessary. In this way, Redis provides a compact, high-performance backbone for coordination in MLVS 2, bridging the asynchronous GPU training tasks and the synchronous HTTP API without introducing unnecessary complexity.

3.2.4 Celery

Celery is used as a distributed task queue that executes the long-running training jobs outside the HTTP request–response cycle, using Redis as both broker and result backend. In this architecture, FastAPI only enqueues a `train_model` task and returns quickly, while the heavy work of dataset extraction, validation, script generation, and TensorFlow training is carried out asynchronously by Celery workers in a separate GPU-enabled container.

Celery is a Python framework for distributed task queues, where tasks are defined as Python callables decorated with `@app.task`, serialized into messages, sent through a message broker (such as Redis), and consumed by one or more worker processes. This design allows time-consuming or blocking operations to be moved out of synchronous web handlers, so that web APIs remain responsive while background work is scheduled, retried, and monitored using task states like `PENDING`, `PROGRESS`, `SUCCESS`, and `FAILURE`.

In MLVS 2, Celery is initialised in `celery_app.py` with `Celery("ml_trainer", broker=..., backend=..., include=["tasks"])`, where both the broker and backend URIs are taken from

the REDIS_URL environment variable, defaulting to redis://localhost:6379/0 for development. The configuration specifies JSON serialization (task_serializer, result_serializer, accept_content=["json"]), UTC timekeeping, tracking of task start events, a hard time limit of 30 minutes per task, and result expiry after one hour, which together constrain resource usage and improve observability for long-running training jobs.

Role

The central training logic is wrapped in a Celery task `train_model`, defined in `tasks.py` with `@celery_app.task(bind=True)`, which passes the task instance as `self` so that progress can be reported via `self.update_state`. When a job is submitted through the `/train` endpoint in `main.py`, a unique `job_id` and directory are created, the configuration JSON and dataset ZIP are written to disk, and `train_model.delay(job_id)` is called, which serializes the call and pushes it into the Redis queue configured as Celery's broker.

Inside `train_model`, the following steps are performed in order: the task state is updated to 'PROGRESS' with a human-readable message, the job hash in Redis is set to "PROCESSING", the full configuration is loaded from `config.json`, the dataset ZIP is extracted into a dataset directory, and the dataset structure is validated against the expected label list. The training configuration is then validated, a custom TensorFlow training script is generated via Jinja2 and written to `generated_train.py`, and this script is executed in a subprocess with GPU-related environment variables and logging to `training_log.txt`; on success, the job status hash is set to "COMPLETED" and a structured result containing output paths and metrics is returned, while on failure, error details are recorded in Redis and log files before a failure result is returned.

Configuration and execution environment

Celery's connection and runtime behaviour are tuned in `celery_app.py` to match MLVS 2's workload characteristics. `task_track_started=True` is used so that intermediate states between PENDING and SUCCESS/FAILURE can be observed, and `worker_prefetch_multiplier=1` is set so that each worker process only prefetches one task at a time, which avoids head-of-line blocking when a single long training job is running. Transport options in `result_backend_transport_options` are configured with `retry_on_timeout` and `retry_on_error` for connection timeouts, which improves robustness when Redis is briefly unavailable or under load.

The worker container defined in `worker.Dockerfile` starts Celery with `celery -A celery_app worker --loglevel=info` and runs on top of `tensorflow/tensorflow:2.15.0-gpu`, so the processes that execute the `train_model` task have direct access to the GPU and the full scientific Python stack required by the generated training scripts. GPU visibility and TensorFlow runtime behaviour are further controlled inside `execute_training` by setting environment variables such as `CUDA_VISIBLE_DEVICES`, `TF_FORCE_GPU_ALLOW_GROWTH`, and optional `TF_ENABLE_MIXED_PRECISION`, which are derived from the hardware section of the model configuration.

Scalability is enabled because additional worker containers can be added later, all connecting to the same Redis broker, so that multiple training jobs can be processed in parallel as hardware resources allow, without any change to the FastAPI code or the Flutter client. This combination of Celery's distributed task queue model, explicit configuration in `celery_app.py`, and rich task lifecycle logic in `tasks.py` therefore provides a robust backbone for executing MLVS 2's GPU-bound training workflows in a controlled and observable manner.

CHAPTER 4: APPLICATION

4.1 Objective

In this chapter, our intention is to show that ML Vision Studio can be used to build, train, and evaluate a custom convolutional neural network that predicts slipper wear levels in a swash-plate axial piston pump from tri-axial vibration data. The central goal is to treat the pump application as a rigorous case study that validates both the machine-learning pipeline and the underlying software product, rather than as an isolated experiment.

The first objective is to formulate slipper wear prediction as a supervised five-class classification problem by converting raw accelerometer recordings on the pump casing into a large, labelled dataset of short vibration windows, each tagged as Healthy, 10 μm , 45 μm , 75 μm , or 100 μm slipper wear. This requires that long time records be segmented into approximately 0.2 s intervals containing 340 samples per channel, transients be removed, and each segment be associated unambiguously with one wear level so that it can be used as one training example. The corresponding motivation is to enable quantitative discrimination between closely spaced wear severities, which is not addressed by most existing hydraulic pump studies that focus mainly on fault type rather than graded degradation.

The second objective is to design a representation of these vibration windows that is both physically meaningful and compatible with the image-based workflows of ML Vision Studio. For this purpose, each tri-axial window is required to be transformed using the continuous wavelet transform into three time–frequency scalograms, which are then stacked as the red, green, and blue channels of a single 256×256 image, so that the information from the X, Y, and Z acceleration components is fused at the input level. This design is intended to exploit the ability of wavelet scalograms to expose non-stationary fault signatures in both time and frequency and to allow a standard 2D CNN to learn cross-axis patterns directly from coloured textures, without requiring specialised multi-branch architectures.

The third objective is to use ML Vision Studio’s graphical interface and server backend to configure, train, and monitor a compact CNN that operates on these RGB CWT images and outputs one of the five wear classes, with minimal manual coding effort. A custom architecture with three

convolution–batch-normalisation–pooling blocks, global average pooling, and two dense layers is to be specified entirely through the MLVS layer and parameter editors, exported as a JSON-described training set, and executed by the FastAPI–Celery–Docker training infrastructure with GPU acceleration. The target is to achieve high test accuracy and balanced per-class precision and recall on held-out CWT images, thereby establishing that a non-expert user can obtain state-of-the-art performance through the four-step configuration flow of MLVS.

The fourth objective is to evaluate the robustness and practical utility of the trained model by analyzing confusion matrices and by systematically corrupting the test CWT images with controlled levels of Gaussian noise, in order to mimic measurement noise and installation variability. The aim is to quantify how overall and per-class accuracy degrade as the effective signal-to-noise ratio is reduced (for example, from clean data to approximately 30 dB, 15 dB, and 10 dB conditions) and to verify that the classifier maintains acceptable performance under moderate noise. A final objective is to show that all signal-processing utilities (segmentation, merging, CWT generation, noise addition) and the MLVS workflow are written in an application-independent manner, so that the same software and methodology can be reused for other laboratory classification tasks beyond axial piston pumps.

4.2 Axial Piston Pumps

Axial piston pumps are positive-displacement hydraulic pumps in which several pistons are arranged parallel to a common drive shaft and reciprocate inside a rotating cylinder block. As the shaft turns, each piston moves back and forth along its axis, drawing hydraulic fluid from a low-pressure inlet during the suction stroke and forcing it into a high-pressure outlet during the discharge stroke, so that the pump can generate high pressures and relatively high flow rates in a compact package. Because displacement per revolution is determined by piston area, number of pistons, and stroke, these pumps are widely used in mobile hydraulics, aerospace, and industrial systems where precise flow control, high efficiency, and the ability to operate at high pressures are required.

Two principal configurations of axial piston pumps are commonly used: the bent-axis type and the swash-plate type. In a bent-axis pump, the cylinder block axis is set at an angle to the drive shaft

axis, so that rotation forces pistons to reciprocate within the bores; the piston stroke, and therefore the displacement, depends on the swivel angle between the block and shaft. In a swash-plate pump, which is the configuration relevant to this project, the cylinder block rotates with the shaft, while pistons slide against an inclined swash plate; the inclination of this plate relative to the shaft determines how far each piston moves during a revolution. Both types can be built as fixed-displacement units with a constant angle, or as variable-displacement units where a control mechanism adjusts the angle to modulate the delivered flow without changing shaft speed.

In a swash-plate axial piston pump, each piston terminates in a spherical end that is connected to a small pad called a slipper, and the slipper face slides on the surface of the stationary or pivoting swash plate. As the cylinder block rotates, the inclined swash plate forces the slippers and pistons to move in and out of their bores, creating sinusoidal changes in chamber volume that alternately connect to suction and discharge ports through a valve plate. The angle of the swash plate directly controls the piston stroke: a larger angle produces a longer stroke and higher displacement per revolution, while a zero angle makes the pistons stationary in the axial direction and reduces the theoretical flow to zero.

The slipper–swash-plate interface is critical for both efficiency and durability, because it must transmit large normal and tangential forces while allowing continuous sliding motion. In practice, the slipper pads are designed to form a hydrostatic or hydrodynamic lubricating film on the swash-plate surface so that direct metal-to-metal contact and friction are minimized; when lubrication degrades or geometry changes due to wear, local contact pressures and vibration levels increase. Over time, this interface is subjected to fluctuating loads, cavitation effects, and thermal-elastohydrodynamic phenomena, and the resulting slipper wear alters clearances, tilting behavior, and flow pulsation, which can be detected indirectly through changes in vibration signatures measured on the pump housing.

In the context of this project, a swash-plate type axial piston pump with multiple pistons and slippers has been used as the test object, and the amount of wear at the slipper–swash-plate contact has been treated as the key health indicator. By understanding that pump output flow is governed by the swash-plate angle and piston stroke, while local tribological conditions at the slipper–swash-plate interface control friction and contact forces, the motivation for using accelerometer-based vibration measurements to infer discrete levels of slipper wear becomes clear.

4.3 Slippers

In a swash-plate axial piston pump, each piston is terminated by a small shoe-like pad called a slipper, which is connected to the piston head through a spherical joint and is constrained to slide on the face of the swash plate. The set of slippers is arranged in a circular pattern, one per piston, so that as the cylinder block rotates, all slippers trace circular paths on the swash-plate surface while transmitting the normal and tangential forces required for piston reciprocation. In this way, the slippers act as the mechanical interface that converts the rotary motion of the drive shaft and cylinder block into the axial motion of the pistons, while also accommodating small tilts and misalignments through the ball-and-socket connection.

Slippers are usually manufactured from hardened steel or copper-based alloys and are often provided with surface treatments or coatings to improve wear resistance and fatigue life under high contact pressures. The sliding face of the slipper is typically machined with a shallow recess and one or more feed holes, through which pressurized working fluid is supplied from the piston chamber to create a hydrostatic support pocket between the slipper and the swash-plate surface. The back side of the slipper is shaped as a spherical seat that mates with the spherical end of the piston, so that the slipper can maintain nearly full-face contact with the swash plate even when small angular displacements occur during operation. Mineral-oil-based hydraulic fluids are generally used as both working fluid and lubricant, while water-based fluids are employed in specialized water-hydraulic axial piston machines, where material selection and surface design must be adapted to poorer lubricity.

Lubrication at the slipper–swash-plate interface is governed by a combination of hydrostatic and hydrodynamic effects, because the pressurized pocket in the slipper supplies load-carrying pressure and the relative sliding motion drags fluid into a thin converging wedge. Under normal conditions, a continuous fluid film separates the two solid surfaces, so that friction is low and wear progresses slowly, provided that surface roughness, micro-geometry (such as chamfers), and operating conditions are within design limits. At high loads, low speeds, start-up, or when the fluid is contaminated or degraded, the fluid film can become locally too thin, and mixed or boundary lubrication regimes can occur, in which direct asperity contact and high local stresses promote wear and surface damage.

Several wear mechanisms act on slippers over the service life of the pump, including abrasive wear from solid contaminants trapped in the interface, adhesive wear when boundary lubrication dominates, and surface fatigue leading to micro-pitting or spalling under cyclic loading. As wear progresses, the slipper thickness and flatness change, the effective clearance and tilt between slipper and swash plate increase, and the hydrodynamic conditions in the lubricating film are altered, which modifies friction forces, contact stiffness, and load distribution across the interface. These changes affect the dynamic response of the pump, increasing vibration levels, altering the spectral content of vibration and pressure ripple, and often reducing volumetric and mechanical efficiency through increased leakage and friction losses.

In practical operation, early slipper wear is not usually detected by casual human observation, because the pump continues to deliver flow and the external appearance remains unchanged while only subtle changes in vibration or internal leakage occur. Damage becomes noticeable to operators when wear has progressed to the point that abnormal noise, palpable vibration at the casing, reduced output flow or pressure, increased operating temperature, or visible leakage are observed, at which stage a significant portion of the original bearing and sliding capacity has already been lost and the risk of sudden failure is high. For this reason, quantitative monitoring of vibration characteristics associated with the slipper–swash-plate interface is pursued in this project, so that discrete levels of slipper wear can be inferred before macroscopic symptoms appear and before irreversible damage propagates to other components of the axial piston pump.

4.4 Factors that indicate wear in slippers

As slippers wear in a swash-plate axial piston pump, several measurable mechanical and hydraulic quantities change in characteristic ways because the geometry, clearances, and lubrication conditions at the slipper–swash-plate interface no longer match their design values. The most relevant indicators include output pressure and its pulsation, flow and leakage behavior, efficiency, fluid temperature, noise, and vibration, each of which responds to altered load sharing and oil-film conditions in the worn interface.

Hydraulic pressure and pressure pulsation provide one set of indicators, because increased leakage and disturbed timing at worn interfaces reduce effective chamber pressure and modify the shape of the pressure waveform. As slipper–swash-plate contact degrades, local oil-film thickness and

bearing capacity change, which alters the pressure distribution in the slipper pocket and can increase small, periodic drops in output pressure associated with leakage and uneven piston loading. These effects manifest as higher pressure ripple amplitudes at shaft-order harmonics and, in severe cases, as a reduction in mean delivered pressure at a given displacement setting, indicating loss of volumetric efficiency and disturbed flow balance.

Leakage and case-drain flow are also sensitive to wear, because the designed clearances in tribological pairs such as the slipper–swash-plate interface are intended to pass just enough fluid to form a lubricating film, but not enough to cause significant losses. As wear enlarges the effective gaps and changes micro-geometry, the local flow resistance decreases, so that more fluid escapes across the interface and into the case, increasing internal leakage and case-drain flow for the same operating pressure. This increased leakage reduces volumetric efficiency, can lower outlet flow and pressure in loaded conditions, and raises the internal circulation of hot fluid inside the housing, which contributes to temperature rise.

Fluid and component temperature tend to increase with slipper wear because friction and shear losses in distorted oil films become larger, and recirculated leakage fluid is not cooled effectively before passing through the interface again. Disturbed lubrication at the slipper–swash-plate pair, especially under high load or low speed, can cause local adhesive or mixed-lubrication regimes, which increase friction coefficients and lead to higher heat generation at the contact, raising both oil temperature and local surface temperature. Over time, this temperature rise accelerates degradation of the hydraulic fluid and may further deteriorate lubrication quality, creating a feedback loop that accelerates wear.

Audible noise and structural vibration are strongly affected by wear, because the interface no longer provides smooth, uniform support, leading to impacts, micro-slips, and uneven force transmission from the swash plate to the pistons and then to the casing. As the slipper pair loses stiffness and alignment, fluctuations in hydrodynamic forces and occasional asperity contacts excite the pump structure, producing increased broadband vibration and distinct spectral components at shaft frequency and its harmonics. At advanced stages of damage, operators can often hear abnormal knocking or whining and can feel increased vibration at the housing, but these symptoms become clearly noticeable only after a substantial fraction of the original bearing capacity has been lost.

Among all these indicators, tri-axial vibration measured on the pump casing, as used in this project, provides a particularly rich and sensitive source of information about slipper wear. Vibration signals directly capture the dynamic response of the pump structure to changes in contact stiffness, oil-film thickness, unbalanced forces, and impacts at the slipper–swash-plate pair, and they respond to very small geometric and tribological changes that may not yet produce substantial shifts in mean pressure, flow, or temperature. Measuring acceleration in three orthogonal directions allows directional effects—such as variations along the shaft axis and in the plane of the swash plate—to be resolved, which improves the ability to distinguish different fault types and severities that may excite different modes of the pump housing.

Vibration sensing is also non-intrusive and easy to retrofit, because accelerometers can be mounted externally on the pump casing without modifying hydraulic circuits or adding internal sensors for leakage or temperature at specific interfaces. When processed with time–frequency methods such as the continuous wavelet transform and analyzed using convolutional neural networks, tri-axial vibration data can reveal subtle, non-stationary patterns associated with progressive slipper wear, enabling earlier detection than reliance on macroscopic quantities like average temperature or mean outlet pressure alone. For these reasons, three-axis vibration has been chosen as the primary condition indicator in this work, with other quantities viewed as secondary manifestations that tend to change later in the wear process.

4.4 Measuring Vibrations

Mechanical vibration is defined as the oscillatory motion of a body about an equilibrium position, and it is described in terms of displacement, velocity, and acceleration as functions of time. Displacement is usually expressed in meters or micrometers for low-frequency, large-amplitude motion, velocity in millimeters per second for general condition monitoring, and acceleration in meters per second squared or in multiples of gravitational acceleration g for high-frequency, small-amplitude vibrations. In engineering diagnostics, these quantities are recorded as time-series signals and are further analyzed using frequency-domain tools such as the fast Fourier transform (FFT) or time–frequency methods to identify characteristic components associated with specific faults.

Vibration can be measured using several types of transducers, which convert mechanical motion into electrical signals. Proximity probes (eddy-current sensors) are used to measure shaft displacement directly relative to a fixed reference and are well suited to large, low-frequency motions in fluid-film bearings. Electrodynamic velocity transducers or piezo-velocity sensors measure vibration velocity and have traditionally been used for medium-frequency monitoring of large rotating machines. Accelerometers, which sense acceleration, are now the most widely used seismic sensors because they are compact, have a wide usable frequency range (from fractions of a hertz to tens of kilohertz), and allow velocity and displacement to be obtained by numerical integration when required.

In this project, vibration is measured using a compact tri-axial accelerometer block, in which three mutually orthogonal sensing elements are integrated in a single cube and the sensitive axes are labelled X, Y, and Z on the housing, as shown in the attached photograph. Such sensors are typically based on the piezoelectric effect, where a piezoelectric ceramic element and a small seismic mass are assembled so that inertial forces under acceleration produce mechanical stress in the crystal, which is converted into an electrical charge proportional to the applied acceleration. In integrated-electronics piezoelectric (IEPE) accelerometers, which are common for machinery monitoring, this charge is converted inside the sensor into a low-impedance voltage signal; the sensor is supplied with a constant-current excitation, and the output consists of an AC voltage proportional to acceleration, superimposed on a DC bias that is removed by the signal conditioner or data-acquisition system.

The tri-axial accelerometer used here is preferable to single-axis sensors or non-contact probes for several reasons that are directly relevant to axial piston pumps. First, a single mounting on the pump casing yields three simultaneous acceleration channels along orthogonal directions, capturing radial, tangential, and axial components of the pump's structural response without requiring three separate sensors and alignments; this improves repeatability and simplifies installation on the compact pump housing. Second, accelerometers are particularly well suited to pumps with rolling and sliding contacts, where slipper and bearing faults excite high-frequency, low-amplitude vibrations that are difficult to detect with displacement probes but are clearly visible in acceleration. Third, the wide bandwidth and high signal-to-noise ratio of piezoelectric accelerometers make them effective for time-frequency analysis and CNN-based classification,

because subtle changes in vibration spectra caused by progressive slipper wear can be captured reliably.

The electrical signal returned by this device is an analogue voltage waveform $v(t)$ whose instantaneous value is linearly proportional to the measured acceleration along each axis after calibration, often expressed in units such as volts per g . Humans can interpret this signal by plotting it as acceleration versus time in the time domain, by computing root-mean-square (RMS) or peak values for overall severity, or by transforming it into the frequency domain to obtain spectra in which peaks at specific frequencies indicate particular mechanical phenomena (for example, shaft rotation frequency, piston-passing frequency, or resonance modes of the housing). In this work, the tri-axial acceleration waveforms are sampled digitally, segmented into short windows, and then processed into continuous-wavelet-transform images, which are used as inputs to the convolutional neural network that classifies slipper wear levels.



Fig. 4.1 Vibration Sensor

4.5 Acquiring the Data

For this study, vibration data were acquired from a swash-plate axial piston pump operated under controlled conditions for five discrete slipper-wear classes: Healthy, 10 μm , 45 μm , 75 μm , and 100 μm . The pump contained nine slippers, and for each wear class exactly one slipper was removed from the rotating group, machined or worn manually to the target material loss, and then measured using high-precision metrology instruments to verify that the reduction in thickness (or height) lay within the specified tolerance for that class before re-assembly. The remaining eight slippers were left in their nominal condition so that the only intentional change between classes was the wear level of this single slipper, which made the subsequent comparisons of vibration signals directly attributable to controlled slipper wear.

After the target wear level for a given class had been confirmed, the modified slipper was installed back into the cylinder block, the pump was re-assembled, and the hydraulic circuit was returned to its normal operating configuration. The pump was then run for at most 10 minutes per experiment, and tri-axial vibration was recorded continuously from the accelerometer mounted on the pump casing while the machine reached and maintained steady operation. This procedure was repeated separately for each of the five wear classes, so that a complete set of vibration records was obtained for the Healthy state and for each of the four incremental wear depths on the same physical pump.

For every run, the data-acquisition system sampled the three acceleration components delivered by the tri-axial accelerometer and stored them in comma-separated value (CSV) files with a consistent structure. Each file contained a header row Channel name, X, Y, Z, followed by time-stamped samples in which the first column represented time in seconds, and the remaining three columns represented acceleration along the X, Y, and Z axes in the sensor's calibrated engineering units. The time increment between consecutive samples in the example file is approximately 6.05×10^{-4} s, corresponding to a sampling frequency of about 1.65 kHz, which is sufficient to capture the dominant shaft-order components and higher-frequency content associated with slipper–swash-plate interactions.

The data-collection procedure was repeated multiple times for each wear class to capture natural variability in operating conditions and to increase the number of independent records available for

machine-learning purposes. For each class, several runs were conducted and the corresponding time histories were saved in distinct CSV files, which together formed a pool of long vibration records per class.

Channel name	X	Y	Z
0.0	-0.545170038	9.759454571	5.147208811
0.0006054999999776	3.726288255	-13.04648045	-5.812913371
0.0012109999999836	-4.380038743	1.257279966	1.87762764
0.0018163999999956	-4.793954241	2.785649131	1.184890427
0.0024219000000016	10.90139041	2.527377161	-3.731239836
0.0030273999999792	-4.107949672	-0.381909252	2.825112607
0.0036327999999912	-1.721388414	-6.004495831	-3.180082208
0.0042382999999972	3.441161583	14.67644444	4.987961621
0.0048438000000032	1.513523891	-5.85785282	-0.672356783
0.0054491999999868	1.26172813	-7.591683982	-2.119667825
0.0060546999999928	-9.64428203	5.427996729	4.028061581
0.0066601999999988	9.450815553	0.545674241	-4.613426292

Fig. 4.2 A snippet of the recorded vibration data

4.6 Preparing the Data

The raw vibration records obtained from the axial piston pump consisted of long time series of tri-axial acceleration for each wear condition, stored as large CSV files with four columns: time, X, Y, and Z. Although these signals already contained information about slipper wear, they were not in a form that could be used directly to train a convolutional neural network, because the sequences were too long, included start-up transients and other non-stationarities, and did not

correspond to fixed-length, uniformly labelled examples. For machine learning, it was necessary to convert these continuous records into many shorter, consistently sized windows that each represented a single, approximately stationary operating segment with a known wear class label.

To achieve this, the data had to be modified in a controlled and reproducible way: obvious transients at the beginning of each run needed to be removed, each long record had to be segmented into windows of equal duration (for example, 0.2 s with 340 samples per axis), and corrupted or empty segments had to be discarded so that the final dataset contained only usable examples. It was also important to standardise the file formats and naming conventions across many experiments so that later stages, such as continuous wavelet transform generation and image-based training in ML Vision Studio, could be automated without manual intervention.

Because the original merged CSV files were large and numerous for each of the five wear classes, the preparation steps were automated using Python scripts that operate on entire folders of files in batch mode. These scripts were written as small, focused command-line tools—for example, to merge multiple raw recordings, to clean and trim transients, to split signals into fixed-length segments, to bifurcate data into training and testing sets, and to generate or combine plots—and they were organised into a reusable toolkit that has been placed in the public repository `pyDataManipulation`. Each tool accepts clear arguments (such as input directory, output directory, window length, and overlap) and produces well-structured outputs, which makes the preprocessing pipeline transparent, repeatable, and easy to adapt to other laboratory datasets.

In summary, preparing the data involved transforming large, heterogeneous vibration recordings into a curated collection of uniformly formatted, fixed-length tri-axial segments with reliable labels, by applying a sequence of scripted operations that remove transients, segment the time series, and standardise file structures. These prepared segments then served as the basis for computing continuous wavelet transform images and for constructing the image datasets that were ultimately used by ML Vision Studio to train the slipper-wear classification models.

4.6.1 Merging the data

The raw vibration recordings for each wear class were originally stored as several separate CSV files per class, corresponding to repeated experiments such as “DO200B Ex1.csv”, “DO200B Ex2

5min.csv”, and so on, under a first-level directory structure in which each subfolder represented one of the five wear levels. This organisation was convenient for acquisition, but it complicated later batch processing, because a single class was represented by multiple disjoint time series with overlapping time channels starting again from zero in each file. To simplify the subsequent segmentation and to ensure that all runs for a class could be treated as one continuous signal, the files within each class folder were merged into a single long record.

For this purpose, a dedicated Python tool named CSVMerger was implemented in the merger.pymodule and wrapped by a short command-line interface in main.py. The merger expects a root directory in which the first-level subdirectories correspond to classes (for example, “10 um”, “45 um”, “Healthy”), and each of these class folders contains the individual CSV files to be combined. When executed, the program discovers all class folders, applies a natural sort to the CSV filenames inside each class (so that “Ex1, Ex2, Ex10...” appear in intuitive order), and reads each file into a pandas DataFrame with high floating-point precision to avoid rounding errors in the time column.

Within each class, the DataFrames are then merged row-wise into a single continuous table using a time-offsetting strategy that preserves the original sampling rate while avoiding duplicate timestamps. The merger first identifies the time column in the first DataFrame by searching for a column name that matches the expected “Channel name” field (case-insensitive), and computes the sampling interval as the difference between the last two time values in this first file. The first DataFrame is copied as the initial result, and its last time value plus one sampling interval is taken as the starting offset for the next file. For every subsequent DataFrame in that class, the script again locates its time column, subtracts the first timestamp in that file from all of its time entries, and then adds the current offset, so that the appended block starts exactly one sampling interval after the previous block ends. In this way, the merged time column increases monotonically from start to finish without overlaps, while the original relative timing within each experiment is preserved.

After all files for a given class have been processed, the merged DataFrame is written to a class-specific CSV file named <Class>_merged.csv inside a new output directory hierarchy that mirrors the class structure, and a detailed log file merger_log_YYYYMMDD_HHMMSS.txt is generated at the root of the output folder. The log summarizes, for each class, the list of input files,

the number of rows and columns read from each file, and the total number of rows in the merged output, which allows the merging process to be audited and repeated if necessary. The result of this step is the MergedData tree, where each wear class is represented by exactly one long, continuous vibration record—still of unequal duration across classes, because the original experiments were run for different lengths of time, but now in a uniform, machine-readable format that greatly simplifies subsequent segmentation and wavelet-based feature generation.

4.6.2 Plotting the complete merged data

After merging, the continuous vibration records for all five wear classes were visualised in both the time domain and the frequency domain using a dedicated plotting program that was developed specifically for this purpose. In the time-domain plots, the merged data for all classes were arranged in a 5×3 grid: rows corresponded to the conditions (Healthy, 10 μm , 45 μm , 75 μm , 100 μm), and columns corresponded to the X, Y, and Z acceleration components, each plotted as acceleration versus time (Channel name). Global axis limits were computed across all classes so that each column shared the same vertical scale, which allowed direct visual comparison of vibration levels between wear states along a given axis.

The visualization system was designed to provide both interactive and publication-quality static outputs. Interactive plots were generated with Plotly using `create_vibration_plot`, which arranged subplots as described above and allowed zooming and inspection of local anomalies directly in the browser. For permanent records and inclusion in the thesis, `export_pdf.py` and `plotter_static.py` produced rasterised PDF figures, again in a 5×3 layout, after optional down-sampling of the time series to reduce file size while preserving the overall waveform shape; the resulting PDFs correspond to the “Vibration Data Analysis – Time Domain” and “Vibration Data Analysis – Frequency Domain (FFT)” figures attached.

Frequency-domain representations were obtained by computing the power spectral density (PSD) of each axis for each class using Welch’s method. For each axis, the sampling rate was first estimated from the time increments, and then the signal was divided into overlapping segments of length `nperseg`, windowed with a Hann window to reduce spectral leakage, transformed by FFT, and converted to PSDs that were averaged across segments; square-root scaling was then applied

to express the result as vibration magnitude versus frequency. Welch’s method was selected instead of a single long FFT because averaging periodograms from overlapping, windowed segments yields a PSD estimate with much lower variance and a smoother spectrum, which makes narrow spectral peaks and broad-band changes due to slipper wear easier to interpret in the presence of random noise. This approach is widely recommended in vibration analysis software for random and mixed vibration signals, where stable PSD estimates are more informative than highly fluctuating raw FFT magnitudes.

The time-domain overview revealed clear non-stationarities and artefacts that motivated later cleaning. In the Z-axis plots, all classes exhibited very large amplitudes and dense noise at the beginning of the record, which decayed over time and suggested strong start-up transients or sensor/fixture settling; corresponding frequency-domain plots showed very high low-frequency magnitudes in this region, visible as large PSD values at the left side of all Z-axis spectra. The X and Y axes appeared more stable overall, but contained distinct intervals with abnormal spikes or step changes in acceleration, which were interpreted as short-duration anomalies not representative of steady pump operation; these segments also introduced localised peaks or broadening in the corresponding FFT plots.

Because these transients and anomalous sections would bias both the PSD estimates and the downstream continuous wavelet transform and CNN training, their systematic removal was planned. The global, class-by-class plots produced in this step therefore served as a “bird’s-eye” diagnostic tool: they confirmed that the merger step had produced continuous signals, highlighted axis-specific issues (especially in Z), and provided quantitative frequency-domain evidence—through the elevated low-frequency content and noisy spectra—that initial portions of the data needed to be excluded or trimmed before the clean, steady-state sections were segmented for machine-learning use.

Vibration Data Analysis - Time Domain

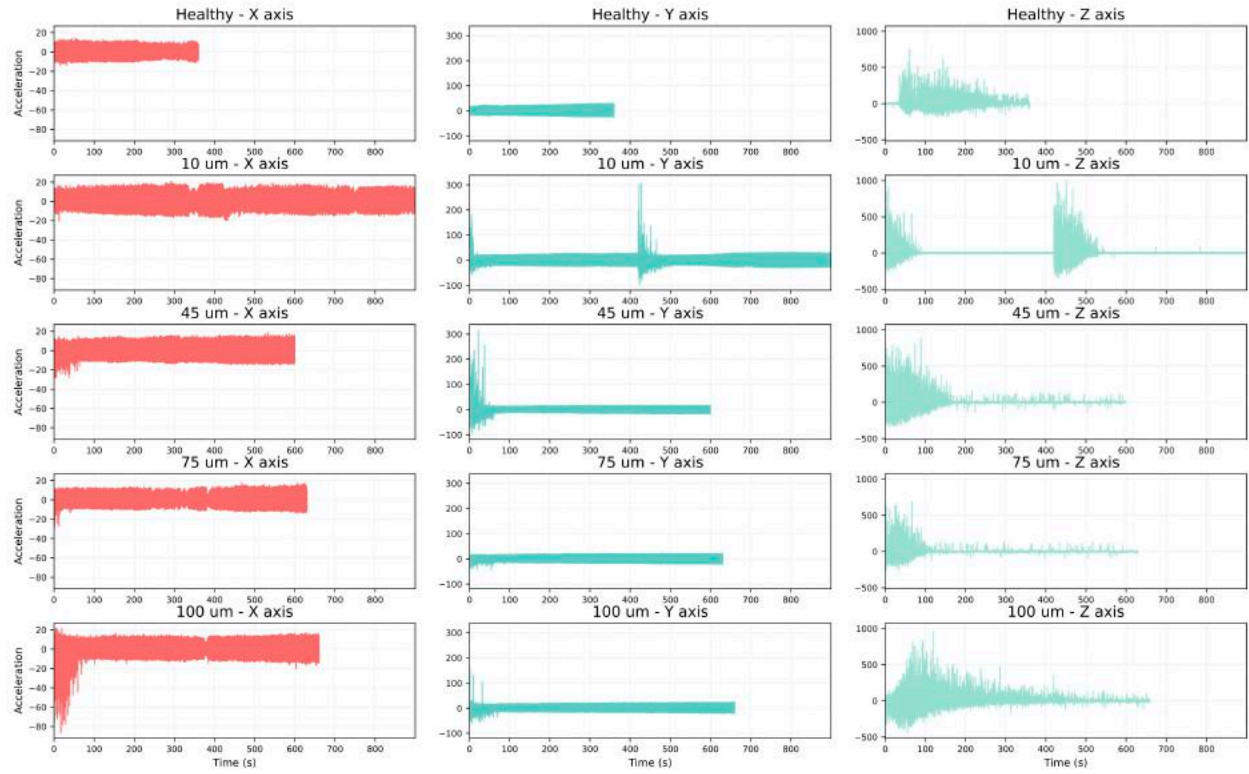


Fig. 4.3 Plot of time-series data of the complete dataset

Vibration Data Analysis - Frequency Domain (FFT)

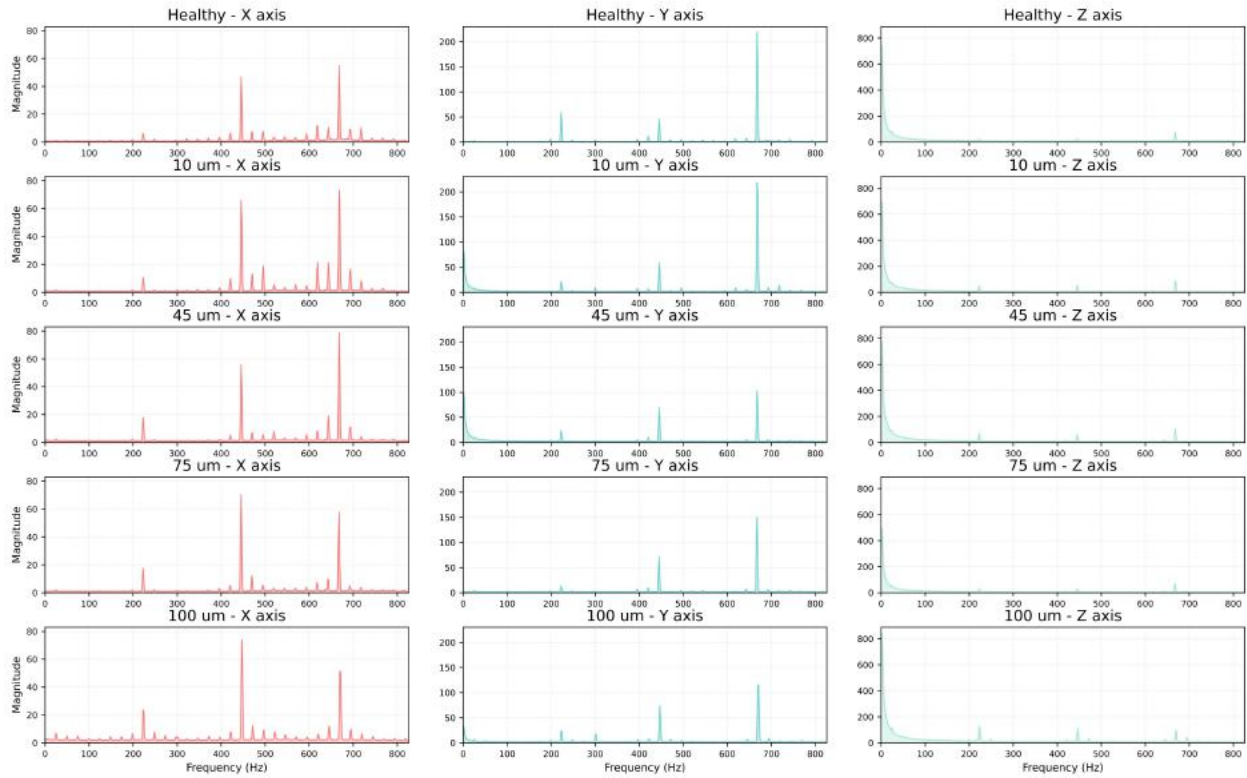


Fig. 4.4 Frequency-domain plot of data of the complete dataset

4.6.3 Removing the transients

After the merged records for each wear class had been plotted, pronounced start-up transients and other short-duration anomalies were observed, especially on the Z-axis and at a few isolated intervals on X and Y, both in the time-series view and in the FFT plots. These artefacts were not representative of steady-state pump behavior and were therefore removed so that the subsequent analysis would reflect only the stationary vibration associated with a given slipper-wear condition. The cleaning step was implemented in a dedicated Python tool named *remover*, which loaded a merged CSV file, allowed abnormal intervals to be selected interactively, deleted the selected samples simultaneously from all three axes, and then closed the gaps by re-computing the time channel so that the sampling interval remained uniform.

In the remover tool, the first column of the CSV was treated as the time channel and the remaining columns were treated as vibration components, which were displayed as three vertically stacked subplots (X, Y, Z) sharing a common horizontal time axis. A `SpanSelector` widget from Matplotlib was attached to each subplot so that a horizontal time range could be highlighted by dragging with the mouse; internally, the start and end times of the selected span were stored once and were applied to all data columns, ensuring that any segment marked on, for example, the Z-axis was removed from X and Y as well, thereby preserving synchronisation between axes. When the Remove button was pressed, the corresponding index range was deleted from the NumPy arrays for every axis, the remaining samples were shifted in time, and a new time vector was generated such that the average sampling interval—previously estimated from the original time differences—was maintained across the shortened record.

To help the user judge whether the chosen interval truly contained only transient noise, a paired FFT view was integrated into the same interface. In FFT mode, the current time-series data for all three axes were transformed using `scipy.signal.welch` to compute power spectral densities, and the resulting spectra were plotted on a logarithmic vertical scale, with options to toggle between “Original” and “Current” data while preserving identical frequency and magnitude scaling so that before-and-after changes could be compared visually. By examining the spectra of the unedited data, broad low-frequency humps and very high PSD values at frequencies close to zero were observed on the Z-axis, confirming the influence of start-up transients; after the corresponding interval was removed, these low-frequency components were greatly reduced while the shaft-order peaks and higher-frequency resonances became more clearly distinguishable.

Once all visually identifiable transients and spikes had been removed for a given class, the cleaned dataset was saved to a new CSV file in an outputs directory, retaining the same four-column structure (time, X, Y, Z) but with a shorter duration and uninterrupted, monotonically increasing time stamps. These cleaned files were then passed back into the earlier plotting pipeline to regenerate the 5×3 time-domain and FFT overviews; in the updated figures, the Z-axis traces no longer contained large start-up bursts, the X and Y traces were free of isolated high-amplitude anomalies, and the corresponding frequency-domain plots exhibited smoother backgrounds with sharply defined peaks that were much better suited for subsequent continuous-wavelet-transform computation and CNN-based classification.

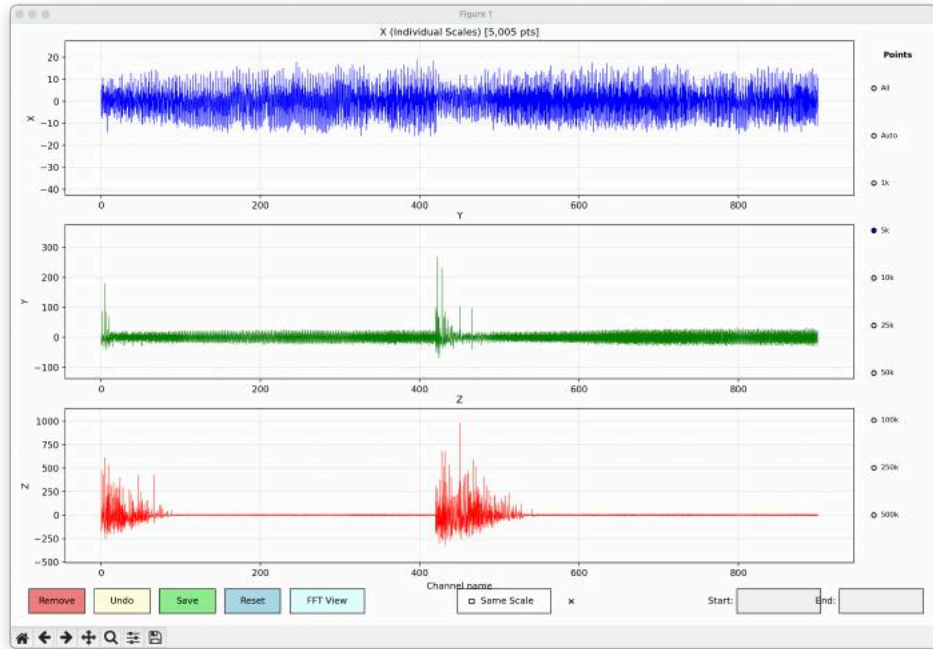


Fig. 4.5 Graphical interface of Remover, which shows the plots with down sampling options for performance. Plots can be dragged and selected

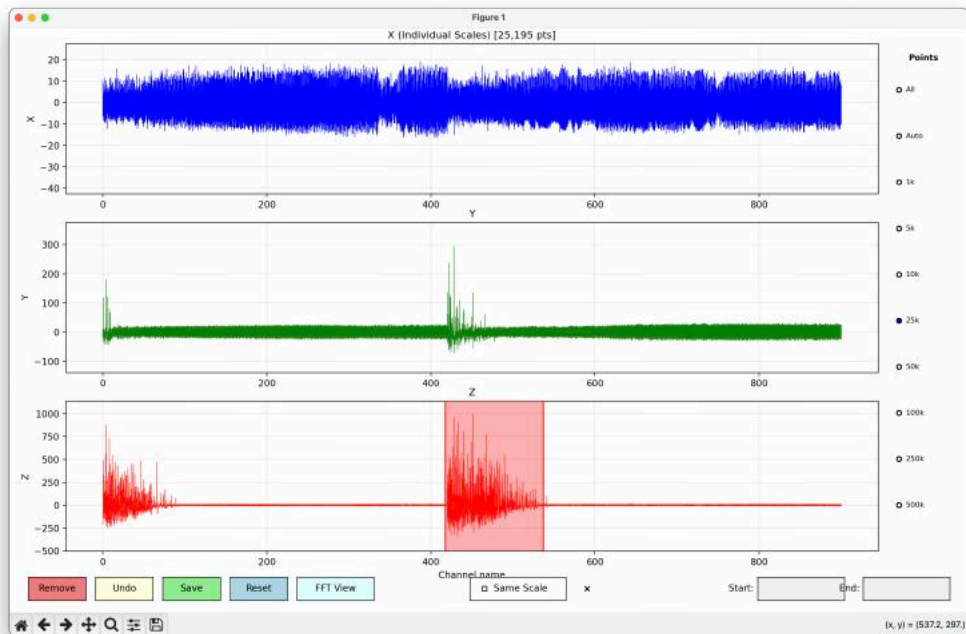


Fig. 4.6 Graphical interface of Remover, which shows one of the plots selected across a range with a simple drag of with the mouse.

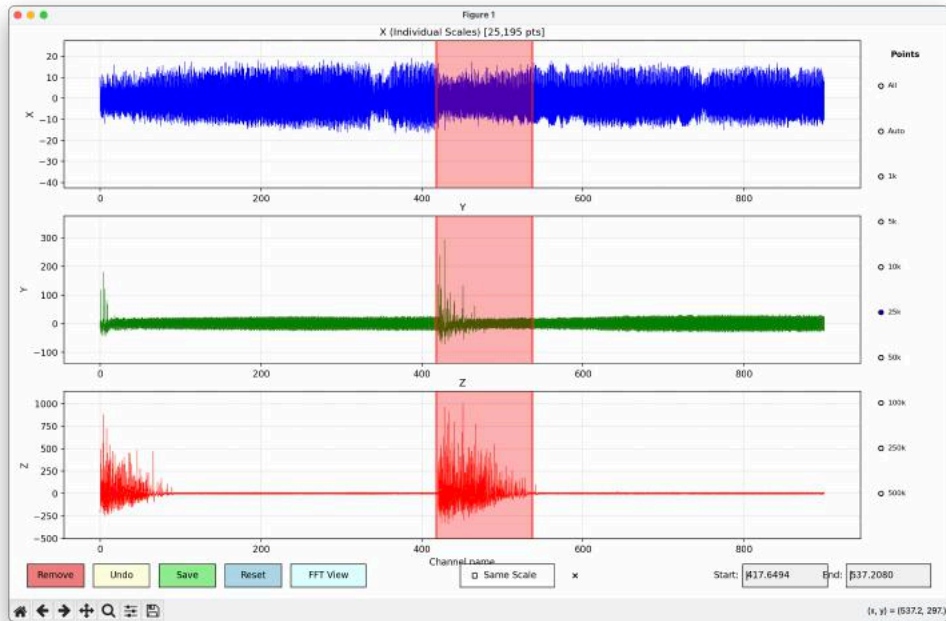


Fig. 4.7 Remove auto selects the same range in the other graphs to synchronize them

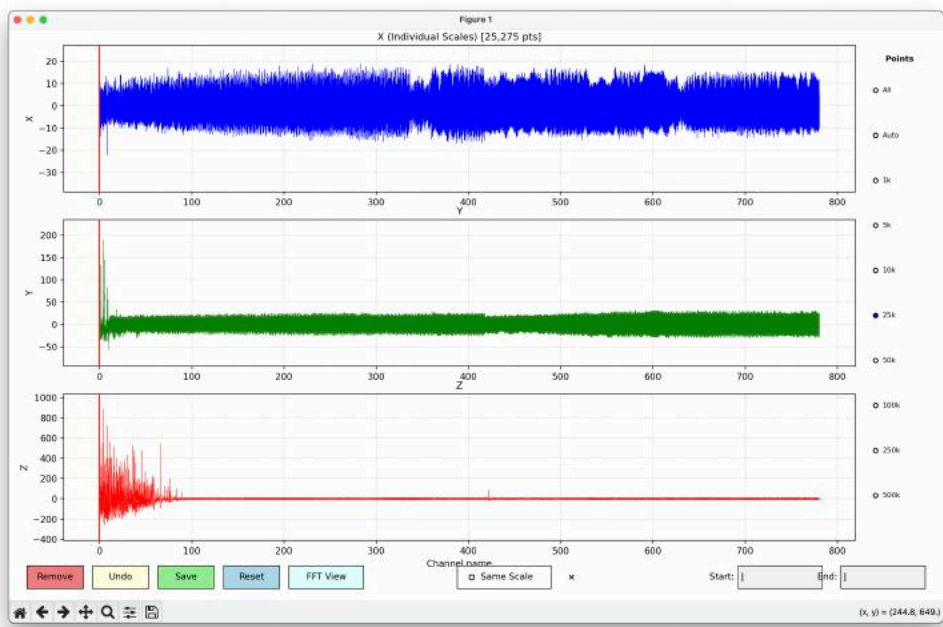


Fig. 4.8 The selected portions of the graphs can be removed with a click, and the gaps are auto filled by shifting the time.

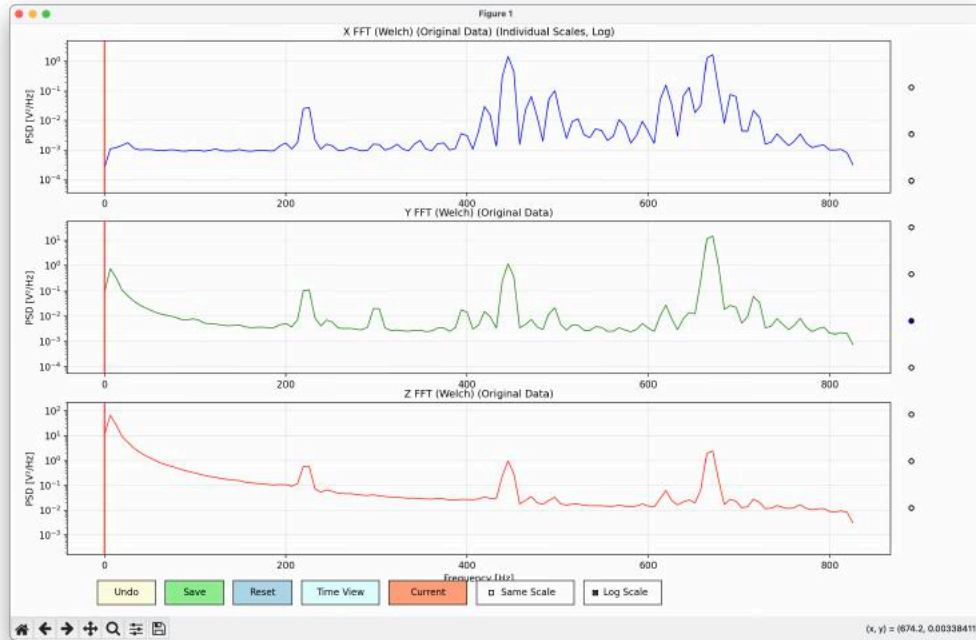


Fig. 4.9 Remover has a built-in FFT view to compare them after reduction. This is the original FFT plot

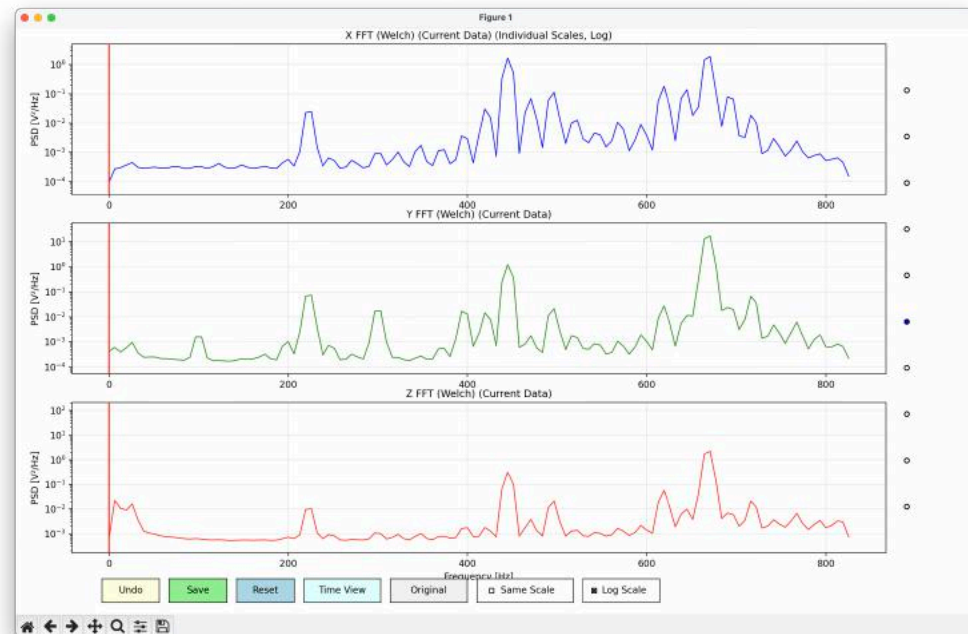


Fig. 4.10 After removal FFT plot

Vibration Data Analysis - Time Domain

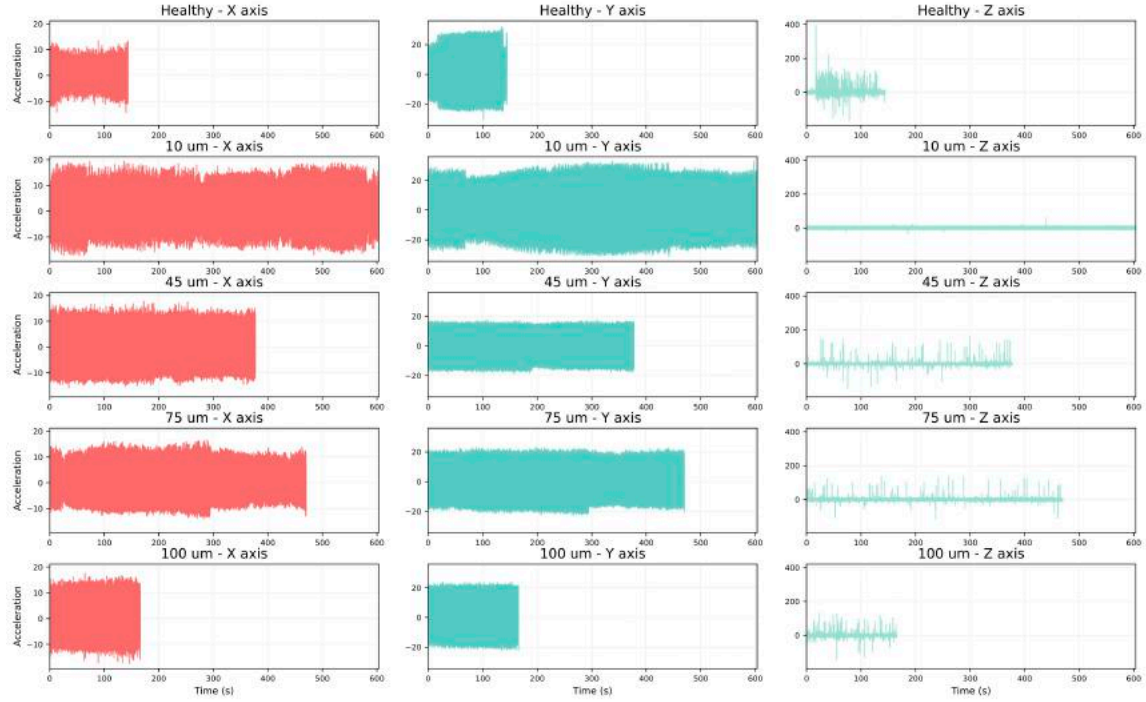


Fig. 4.11 Time-domain plot after removal of transients

Vibration Data Analysis - Frequency Domain (FFT)

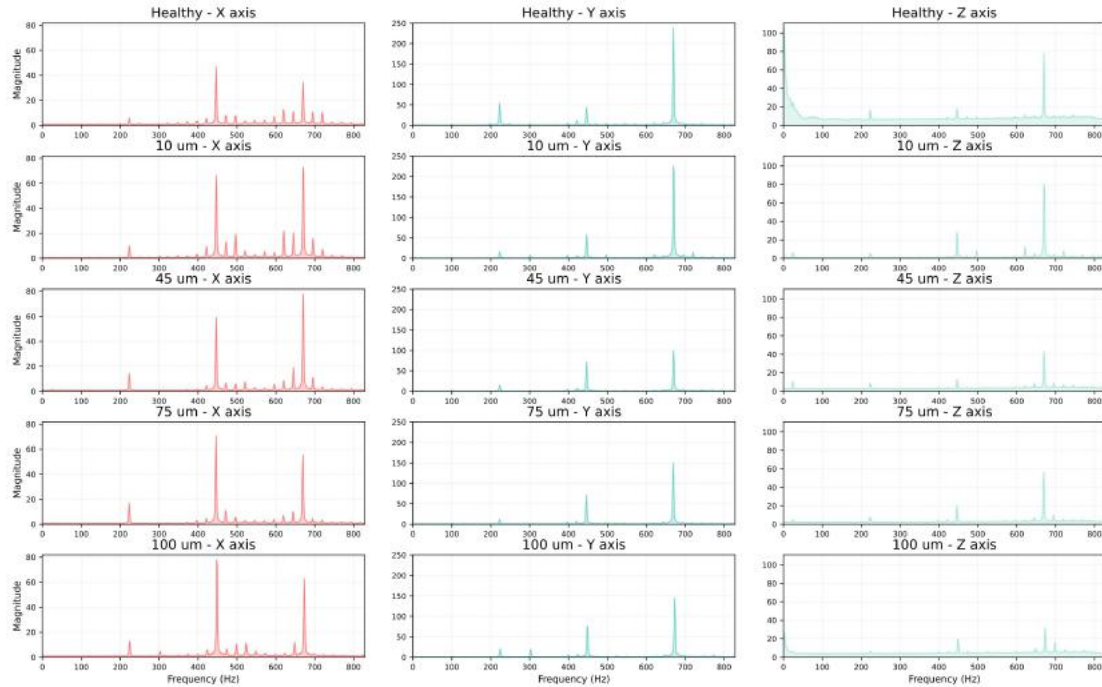


Fig. 4.12 Frequency-domain plot after removal of transients

4.6.4 Splitting the Data

After transient removal, each wear class was represented by a single long CSV file containing a continuous time column and three acceleration columns (X, Y, Z). For training and later real-time inference, this continuous record was required to be converted into many shorter, fixed-length examples, each covering roughly 0.2 s of vibration, because such windows are light-weight to process, still contain several shaft revolutions, and can be adjusted easily if another application requires a different time scale. The healthy class was used as the reference because it contained the least total duration after cleaning (about 144.36 s and 238 435 data points), so the window length was chosen such that this shortest class could still yield roughly 700 non-overlapping segments.

The sampling interval was inferred from the time stamps in the vibration files: for example, in the 45 μm class sample, consecutive values in the Channel name column differ by about 6.05×10^{-4} s, corresponding to a sampling frequency of approximately 1.65 kHz. A 0.2 s window therefore contains about

$$N_{\text{window}} \approx \frac{0.2 \text{ s}}{6.05 \times 10^{-4} \text{ s/sample}} \approx 330 \text{ samples},$$

which already provides many vibration cycles at the dominant shaft-order frequencies. Because an integer number of samples is required, and a slightly longer window improves frequency resolution while still being light-weight, the window size was rounded up to 340 samples; with the same sampling interval, 340 samples correspond to approximately 0.206 s, which is acceptably close to the 0.2 s design target. Using 340-sample windows, the healthy class with 238 435 samples can theoretically provide $\lfloor 238435/340 \rfloor \approx 701$ complete windows, so about 700 windows per class were considered feasible without reusing data.

To automate the splitting, a CSVSplitter class was implemented in `csv_splitter.py` and driven by a small command-line wrapper. The splitter was configured in `split_by='points'` mode with `split_value=340`, so that the DataFrame for a class was divided into consecutive, non-overlapping chunks of exactly 340 rows, and any leftover tail with fewer than 340 points was either discarded or saved separately as a remainder, depending on the chosen settings. For each

chunk, the `normalize_dataframe` method was used to “normalise” the time column by subtracting its minimum value from all entries in that chunk, which caused the first row in every output file to have time 0 while preserving the original relative timing within the window; the X, Y, and Z acceleration values were left unchanged so that only a local time origin shift was applied.

The splitter tool scanned the cleaned CSV files for each class, reported the total number of rows, and then generated a series of numbered chunk files (for example, `Healthy_0001.csv`, `Healthy_0002.csv`, and so on), each containing four columns: time (starting at 0) and the three acceleration components. Because each wear class had a different total duration after transient removal, the number of resulting 340-point chunks varied between classes, but every chunk within a class and across classes represented an equal-length tri-axial vibration segment of about 0.2 s, stored as an independent CSV file that could later be converted into a continuous-wavelet-transform image and used as one labelled training example for the convolutional neural network. One of the chunks have been plotted below

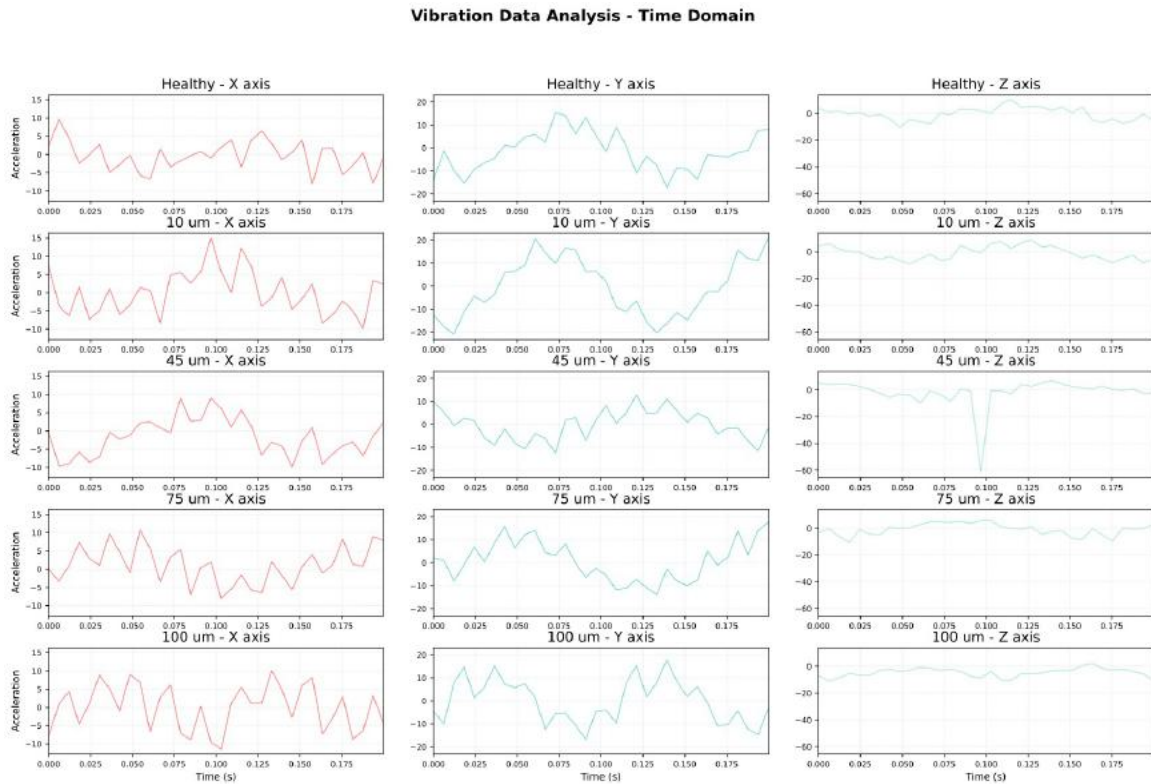


Fig. 4.13 Plot of a chunk of data lasting ~ 0.2s containing exactly 340 points each

4.6.5 Bifurcating the Data into Train, Validation, and Test Sets

Once all vibration windows had been generated as ~ 0.2 s CSV chunks for each wear class, these files were divided into three disjoint subsets for model development: a training set, a validation set, and a test set. The adopted proportions were 70 % for training, 15 % for validation, and 15 % for testing, which fall within the commonly recommended range where roughly 60–80 % of the data is used for training and the remaining 20–40 % is split between validation and test for hyper-parameter tuning and unbiased evaluation. With this choice, the majority of windows in each class contribute to learning the CNN weights, while still reserving enough examples to tune architecture and training settings (validation set) and to measure final performance on unseen data (test set) without re-using the same windows for both purposes.

To perform this division reproducibly, a general-purpose script named `bifurcator` was implemented. The tool scans an input directory tree whose leaf folders correspond to classes (for example, Healthy, 10 μm , 45 μm , 75 μm , 100 μm), filters files by extension so that only the window CSV files are considered, and then, for each class folder independently, computes the number of files that should go into each split according to the specified percentages and a chosen rounding rule. Before assignment, the list of files in each class can be randomly shuffled; when randomisation is enabled, an optional integer seed is used to initialise the pseudo-random number generator, so that the same seed always yields the same train/validation/test partition, which is important for repeatable experiments and fair comparison between different model configurations.

The actual bifurcation proceeds class by class to preserve class balance across splits. For a given class with N windows, the `bifurcator` computes

$$N_{\text{train}} = \text{round}(0.70 N), N_{\text{val}} = \text{round}(0.15 N), N_{\text{test}} = N - N_{\text{train}} - N_{\text{val}},$$

with the exact rounding behavior controlled by the configured “standard”, “floor”, “ceil”, or “proportional” method. The first N_{train} files in the (optionally shuffled) list are copied to a `train` subfolder for that class, the next N_{val} files are copied to a `val` subfolder, and the remaining files are copied to a `test` subfolder, all under a new output base directory. Because the operation is repeated independently for each wear level, every class contributes approximately 70 % of its

windows to training, 15 % to validation, and 15 % to testing, even though the absolute number of windows differs between classes, which yields class-balanced splits and a consistent evaluation protocol across the five-class slipper-wear classification problem.

4.6.6 Running Continuous Wavelet Transformation (CWT)

A wavelet is defined as a short, oscillatory function that is well localized in both time and frequency, and the continuous wavelet transform (CWT) is obtained by correlating a signal with time-shifted and scale-dilated copies of this function. In mathematical terms, for a vibration signal $x(t)$, the CWT at scale a and time shift b is given by $W_x(a, b) = \int_{-\infty}^{\infty} x(t) \psi_{a,b}^*(t) dt$, where $\psi_{a,b}(t) = \frac{1}{\sqrt{|a|}} \psi\left(\frac{t-b}{a}\right)$ is the scaled and shifted “mother” wavelet $\psi(t)$ and $*$ denotes complex conjugation. By varying the scale a , low-frequency behaviour (large scales) and high-frequency details (small scales) are analysed in a single representation, so that non-stationary features such as short transients and evolving resonances in axial piston pump vibration are captured much more clearly than in a single global FFT.

Several wavelet families can be used for CWT, including real wavelets such as Mexican hat and complex wavelets such as Morlet, each providing different time–frequency trade-offs and mathematical properties. In this work, the complex Morlet wavelet implemented in PyWavelets under the name 'morl' was selected; this wavelet may be viewed as a complex sinusoid modulated by a Gaussian envelope, which produces smooth, well-shaped time–frequency ridges that resemble narrowband vibration components and is widely recommended for machinery fault analysis. The use of a complex wavelet allows both amplitude and phase information to be represented, but for image construction the magnitudenitude $|W_x(a, b)|$ is used, because it encodes how strongly each frequency band (scale) is excited at each instant, which is the quantity most relevant for distinguishing slipper-wear levels.

Compared to using raw tri-axial time series directly, CWT scalograms provide a two-dimensional view in which time runs horizontally, scale (or pseudo-frequency) runs vertically, and color encodes vibration intensity, so that patterns associated with particular mechanical phenomena appear as visually distinct structures. In strongly non-stationary hydraulic pumps, slipper wear affects both the strength and temporal localization of harmonics and sidebands; these changes are

dispersed across the 1D waveform but become concentrated into localized blobs and ridges in the CWT, which convolutional neural networks can recognize using the same spatial filters that work well on natural images. This representation therefore acts as a learned feature extractor that combines classical time–frequency analysis with modern image-based deep learning, and it enables the same CNN architectures already supported by ML Vision Studio to be reused without designing pump-specific 1D architectures.

To generate these scalogram images, a dedicated program `cwt-gen` was implemented around the `CWTAnalyzer` class. For each windowed CSV file containing time, X, Y, and Z columns, the script reads the data with `pandas`, extracts the three acceleration channels, and computes the CWT of each channel using `pywt.cwt` with scales from 1 to N (typically 128 scales) and the Morlet wavelet; the absolute value of the resulting complex coefficient matrix is taken for each axis. Each coefficient matrix is then normalized independently to the 0–255 range by subtracting its minimum and dividing by its dynamic range, and the three normalized matrices are stacked as separate color planes so that the X-axis scalogram is mapped to the red channel, the Y-axis scalogram to the green channel, and the Z-axis scalogram to the blue channel, forming a single RGB image that encodes tri-axial dynamics in color rather than in separate grayscale plots. This RGB fusion is particularly powerful, because correlations and differences between axes appear as characteristic color mixtures (for example, features present only in Z appear reddish-blue, while features shared by X and Y appear yellowish), giving the CNN access to inter-axis relationships without requiring three parallel networks or late fusion, which constitutes a novel and compact way of exploiting tri-axial vibration information in axial piston pump diagnosis.

The RGB array is finally converted into a `PIL.Image` object and resized to a fixed resolution (for example, 256×256 pixels) using Lanczos interpolation so that all samples share identical input dimensions for the CNN, and the image is saved as `*_cwt.png` in an `outputs/` folder that mirrors the original class and split directory structure. The `process_path` routine allows either a single CSV file or an entire directory tree to be processed in batch; all `.csv` files are discovered recursively, progress is displayed in the terminal, and a detailed log file records every success or failure together with the chosen scales, image dimensions, and wavelet type, ensuring that the transformation from numerical windows to CWT images is fully automated, reproducible, and ready for ingestion by the ML Vision Studio image-training pipeline.

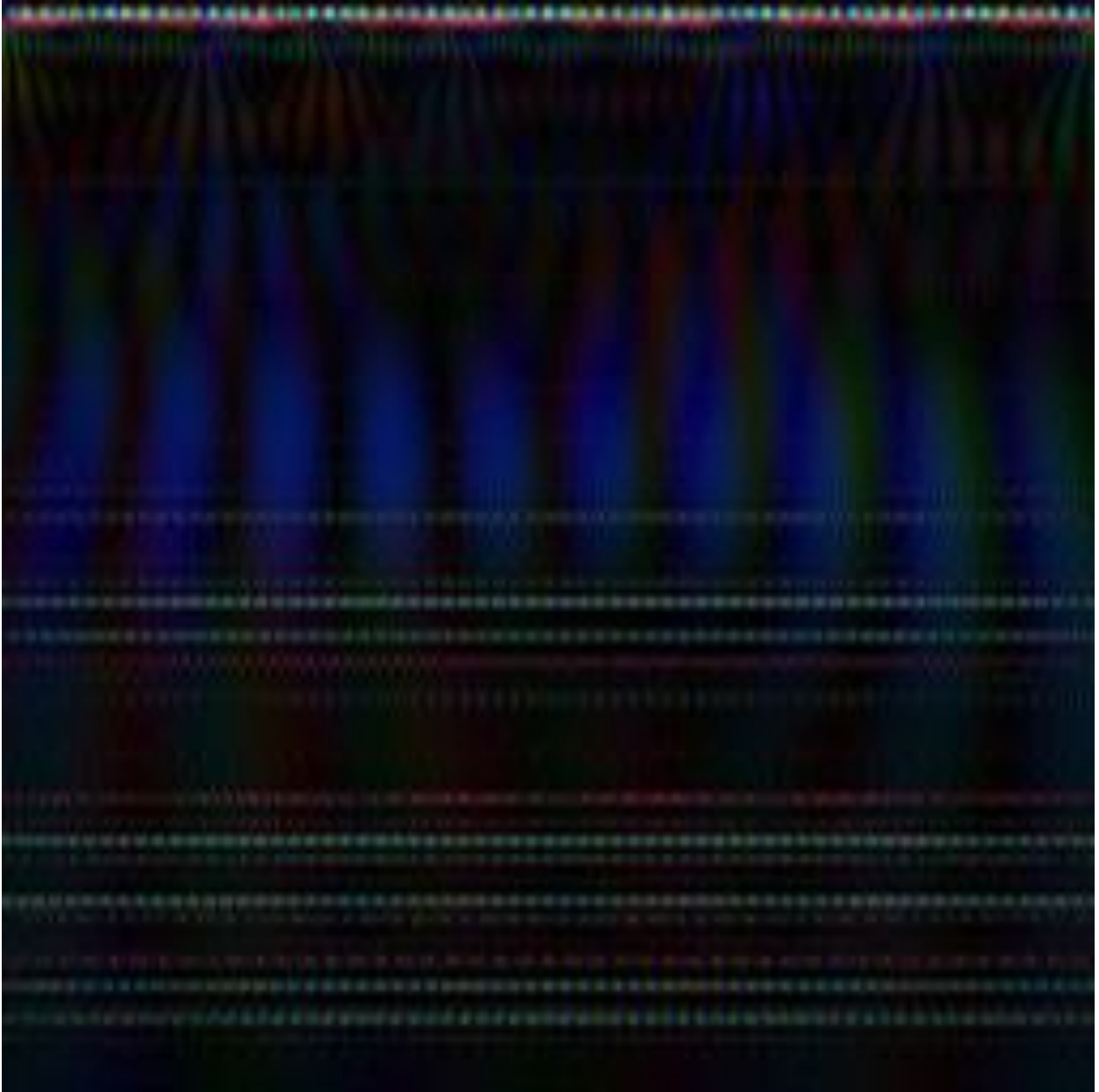


Fig. 4.14 One of the resulting images after running CWT

4.7 Model Architecture

4.7.1 Convoluted Neural Networks (CNNs)

A convolutional neural network is defined as a feed-forward neural network in which at least some layers perform discrete convolutions between learnable kernels and multidimensional input arrays (feature maps) instead of using fully connected matrix multiplications. Each convolutional layer applies multiple small kernels (for example, 3×3 filters) that scan across the spatial dimensions of the input and produce a stack of new feature maps, where each map encodes the response of the input to one learned pattern such as an edge, ridge, or localized blob. Because the same kernel weights are reused at all spatial positions, the number of parameters is reduced drastically compared with a dense layer operating on the same image, and a built-in invariance to translations of local patterns is introduced.

For an input feature map x with channels c and a filter k with kernel weights $w_{k,c}$, the output activation at spatial position (i, j) in feature map y_k is given by a discrete convolution $y_k(i, j) = \sigma \left(\sum_c \sum_{u,v} w_{k,c}(u, v) x_c(i + u, j + v) + b_k \right)$, where (u, v) runs over the kernel support, b_k is a bias term, and σ is a nonlinear activation function such as ReLU. This operation can be interpreted as a learned, localized linear filter followed by a nonlinearity, so that early layers tend to learn simple patterns and deeper layers combine them into increasingly abstract features (for example, from local time–frequency ridges to full fault signatures). Pooling layers then perform a down-sampling operation such as max- or average-pooling over small neighbourhoods (often 2×2 with stride 2), thereby reducing spatial resolution, aggregating nearby activations, and providing local translation invariance while limiting the computational cost of deeper layers.

In standard image-classification CNNs, convolutional and pooling layers are stacked into several blocks, followed by one or more fully connected layers and a final softmax layer that outputs class probabilities. Batch normalization layers are often inserted after convolutions to normalize intermediate activations, which improves numerical stability and allows higher learning rates, and dropout layers can be added in the fully connected part to regularize the model by randomly deactivating units during training. The network used here contains three convolutional blocks (with

32, 64, and 128 filters of size 3×3 , each followed by batch normalization, ReLU, and 2×2 max-pooling), a global average pooling layer that compresses each feature map to a single value, a dense layer of 64 ReLU units with dropout, and a final dense layer with five softmax outputs corresponding to the five slipper-wear classes.

In fully connected multilayer perceptrons, every neuron in a layer is connected to every neuron in the previous layer, so spatial structure in the input must be learned from scratch and the number of parameters grows with the square of the input size. In CNNs, by contrast, each neuron in a convolutional layer is connected only to a small receptive field in the previous feature maps, and the same small set of weights is shared across all positions; this locality and weight sharing make CNNs far more parameter-efficient and better suited to data where nearby pixels (or time–frequency cells) are strongly correlated. Recurrent networks such as LSTMs are specialized for sequential dependencies over time, whereas CNNs are intrinsically two-dimensional in this setting and are therefore ideal when the input has an image-like structure such as the CWT scalograms generated from the tri-axial vibration windows.

In rotating-machinery and hydraulic-pump diagnosis, it has been shown repeatedly that time–frequency images of vibration signals, obtained by transforms such as CWT, can be classified very effectively by CNNs because fault signatures appear as localized patterns across time and frequency. Studies on bearings, centrifugal pumps, and hydraulic pumps report that CNNs operating on such vibration images achieve high diagnostic accuracy with fewer handcrafted features and stronger robustness to noise than traditional methods based on manually engineered spectral indicators and shallow classifiers. By feeding RGB CWT images (with X, Y, Z axes mapped to colour channels) into a compact CNN, this project leverages those advantages: spatial convolutions exploit both intra-axis patterns and inter-axis colour interactions, the small architecture defined in `simpleCNN.py` remains light enough for real-time inference, and the same CNN design can be re-used inside the ML Vision Studio product for other image-based classification tasks beyond vibration analysis.

4.7.2 Architecture of our Custom CNN model

A compact, custom convolutional neural network (CNN) was defined to classify the RGB CWT scalograms into the five slipper-wear classes, instead of adopting a large off-the-shelf architecture. The model, named `cwt_cnn`, follows a deliberately shallow three-block Conv–BatchNorm–ReLU–MaxPool backbone with a lightweight global-pooling classifier head, which yields approximately 1.0×10^5 trainable parameters and is therefore suitable for real-time deployment within the ML Vision Studio pipeline.

The network is instantiated through the function `make_model`, which takes CWT images of shape $256 \times 256 \times 3$ as input and outputs a five-dimensional softmax probability vector corresponding to the five slipper-wear severity classes (`NUM_CLASSES = 5`). The Keras model summary shows the exact layer stack: an input layer, a light augmentation and rescaling stage, three convolutional blocks with 32, 64, and 128 filters, a `GlobalAveragePooling2D` layer, a dense layer with 64 ReLU units followed by dropout, and a final dense layer with 5 softmax outputs, totalling 102 725 parameters (102 277 trainable). This configuration is custom because it was implemented directly in code rather than imported from standard architectures such as VGG or ResNet, and because its depth, filter counts, and use of global average pooling were tuned to the specific resolution and statistical characteristics of the CWT images rather than to generic natural-image benchmarks.

The training, validation, and test datasets are constructed from three separate directory trees using `tf.keras.utils.image_dataset_from_directory`, with `image_size=(256, 256)`, `color_mode="rgb"`, and integer labels inferred from folder names, which ensures that each tri-axial CWT scalogram is consistently interpreted as a 256×256 RGB image with a class index between 0 and 4. Each dataset is then passed through `configure_for_performance`, which applies `.cache()` to keep preprocessed batches in memory, optional `.shuffle(1000, seed=123)` for the training set, and `.prefetch(buffer_size=AUTOTUNE)` so that data loading and model computation are overlapped on the device; this configuration reduces input overhead and stabilizes epoch-to-epoch timing during training. Inside the model, a small `keras.Sequential` augmentation module applies `layers.RandomBrightness(factor=0.1)` and `layers.RandomContrast(factor=0.1)` to the input tensor, followed by a `Rescaling(1.0/255.0)` layer, so that pixel intensities are normalised

to $[0,1]$ and mild photometric perturbations are introduced; this is appropriate for CWT scalograms where absolute colour values may vary slightly across operating conditions, while geometric structure along the time and frequency axes must be preserved.

The feature extractor consists of three convolutional blocks, each following the pattern. Conv2D $\rightarrow$ BatchNormalization $\rightarrow$ ReLU $\rightarrow$ MaxPooling2D, which is repeated with increasing channel depth and decreasing spatial resolution. The first block applies a Conv2D layer with 32 filters of size 3×3 and padding="same" to the normalised $256 \times 256 \times 3$ input, producing a $256 \times 256 \times 32$ feature map with 896 trainable parameters, after which BatchNormalization, ReLU, and MaxPooling2D(pool_size=(2,2)) reduce the spatial dimensions to $128 \times 128 \times 32$; this block is responsible for detecting very local time–frequency patterns such as short ridges and blobs in any of the RGB channels. The second and third blocks repeat this structure with 64 and 128 filters, operating on $128 \times 128 \times 32$ and $64 \times 64 \times 64$ tensors respectively and downsampling them to $64 \times 64 \times 64$ and $32 \times 32 \times 128$; the corresponding convolution layers (conv2d_1 and conv2d_2) contain 18 496 and 73 856 parameters and learn progressively more abstract combinations of local features, while max-pooling doubles the effective receptive field at each stage without an explosion in parameter count. Batch normalization is placed immediately after each convolution to stabilise the distribution of activations during training and to allow the Adam optimiser with learning rate 10^{-3} to converge reliably, and ReLU activations introduce non-linearity while maintaining computational efficiency on both CPU and GPU.

After the final pooling layer, a GlobalAveragePooling2D layer collapses each of the 128 feature maps of size 32×32 into a single scalar by averaging over the spatial dimensions, yielding a compact 128-dimensional vector that summarises the presence of learned patterns anywhere in the CWT image and avoids the very large parameter counts that would result from flattening the full feature map. This vector is passed to a dense layer with 64 units and ReLU activation (dense), which introduces a low-dimensional, fully connected bottleneck with 8 256 trainable parameters, followed by a Dropout(0.3) layer that randomly zeros 30 % of these activations during training to reduce co-adaptation and overfitting on the finite CWT dataset. The final output layer (dense_1) is a dense layer with 5 units and softmax activation, adding 325 parameters and transforming the 64-dimensional representation into a probability distribution over the five slipper-wear classes, consistent with NUM_CLASSES = 5 and the class order recorded in the metadata files.

The model is compiled with the Adam optimiser at a fixed learning rate of 1×10^{-3} , using `sparse_categorical_crossentropy` as the loss function and accuracy as the primary metric, so that integer class labels can be used directly without explicit one-hot encoding and adaptive learning-rate updates are applied per parameter. Class weights are computed from the training set by counting the number of images per class in a separate `image_dataset_from_directory` pass (with `batch_size=1`) and applying the formula $\text{weight}_c = \frac{N_{\text{total}}}{K N_c}$, where $K = 5$ is the number of classes and N_c is the count for class c ; these weights are passed to `model.fit` via the `class_weight` argument to compensate for any imbalance between wear classes without discarding data. Training is configured for a maximum of 40 epochs (`EPOCHS = 40`), but an `EarlyStopping` callback monitors `val_accuracy` with `patience=8` and `restore_best_weights=True`, a `ModelCheckpoint` callback saves only the best-performing model to `best_model.keras`, and a `CSVLogger` callback records the per-epoch metrics to `training_log.csv`, so that learning is automatically stopped once validation performance saturates and the best parameters are retained.

Model: "cwt_cnn"

Layer (type)	Output Shape	Param #
input_layer (InputLayer)	(None, 256, 256, 3)	0
sequential (Sequential)	(None, 256, 256, 3)	0
rescaling (Rescaling)	(None, 256, 256, 3)	0
conv2d (Conv2D)	(None, 256, 256, 32)	896
batch_normalization (BatchNormalization)	(None, 256, 256, 32)	128
re_lu (ReLU)	(None, 256, 256, 32)	0
max_pooling2d (MaxPooling2D)	(None, 128, 128, 32)	0
conv2d_1 (Conv2D)	(None, 128, 128, 64)	18,496
batch_normalization_1 (BatchNormalization)	(None, 128, 128, 64)	256
re_lu_1 (ReLU)	(None, 128, 128, 64)	0
max_pooling2d_1 (MaxPooling2D)	(None, 64, 64, 64)	0
conv2d_2 (Conv2D)	(None, 64, 64, 128)	73,856
batch_normalization_2 (BatchNormalization)	(None, 64, 64, 128)	512
re_lu_2 (ReLU)	(None, 64, 64, 128)	0
max_pooling2d_2 (MaxPooling2D)	(None, 32, 32, 128)	0
global_average_pooling2d (GlobalAveragePooling2D)	(None, 128)	0
dense (Dense)	(None, 64)	8,256
dropout (Dropout)	(None, 64)	0
dense_1 (Dense)	(None, 5)	325

Total params: 102,725 (401.27 KB)
Trainable params: 102,277 (399.52 KB)
Non-trainable params: 448 (1.75 KB)

Fig. 4.15 Model Summary

4.8 Model Performance

The custom CNN was trained on the RGB CWT images for a maximum of 40 epochs, but training was stopped early after 12 epochs based on validation accuracy, yielding a final test accuracy of approximately 99.77 % with a very low test loss of 0.013. The per-class metrics and confusion matrix indicate that all five slipper-wear classes were classified with near-perfect precision, recall, and F1-scores, with only three misclassifications observed in the 75 μm class.

The training history shows that the model converged very rapidly: the training accuracy increased from about 90.8 % in epoch 0 to more than 99.4 % by epoch 1, and exceeded 99.8 % by epoch 10, while the corresponding training loss decreased from 0.32 to below 0.002. This rapid rise reflects the strong separability of the CWT scalograms under the chosen architecture and the effectiveness of class weighting, which up-weighted the under-represented 100 μm and Healthy classes (class weights around 2.13 and 2.45 respectively) relative to the more frequent 10 μm and 75 μm classes. The training metrics also exhibit a brief increase in loss and small dip in accuracy at epoch 8 (loss $\approx$ 0.024, accuracy $\approx$ 99.4 %), which is typical stochastic variation rather than systematic degradation, after which the model resumed its monotonic improvement.

The validation accuracy started much lower than the training accuracy (about 34 % at epoch 0) and improved steadily as the model parameters adapted, reaching about 47 % at epoch 1, 97.8 % at epoch 2, and 100 % by epoch 3 with a corresponding drop in validation loss from 1.85 to around 0.009. Subsequent epochs maintained validation accuracy at or extremely close to 100 % (values fluctuating between 0.999 and 1.0), and validation loss remained in the 10^{-3} to 10^{-5} range, apart from two transient spikes at epochs 7 and 8 where val_loss temporarily increased to 0.30 and 3.68 and val_accuracy dipped to about 88 % and 34 % respectively. Because the EarlyStopping callback monitored val_accuracy with a patience of 8 epochs and was configured to restore the best weights, training continued until epoch 11 (12 epochs in total), after which no strictly better validation accuracy than the first 100 % value had been observed for eight consecutive epochs, triggering early stopping and leaving the weights corresponding to the best validation performance in memory.

The held-out test set comprised 1 287 CWT images with the following class distribution: 441 images for 10 μm wear, 121 for 100 μm , 275 for 45 μm , 344 for 75 μm , and 106 for Healthy, as

recorded in the scikit-learn classification report. On this test set, the model achieved an overall accuracy of approximately 99.77 %, with a test loss of 0.01296, demonstrating that the high validation accuracy generalised well to unseen data rather than being an artefact of overfitting. The macro-averaged precision, recall, and F1-score were all approximately 0.99–1.00, indicating uniformly strong performance across classes despite the moderate class imbalance and the smaller sample count for the Healthy and 100 μm categories.

The confusion matrix shows that all 10 μm , 100 μm , 45 μm , and Healthy test samples were classified correctly, whereas the 75 μm class had 341 correct predictions and 3 misclassifications into some other class, resulting in a recall of about 0.99 for 75 μm and exactly 1.0 for the remaining four classes. The class-wise precision values were approximately 1.0 for 10 μm , 100 μm , 45 μm , and 75 μm , and about 0.97 for Healthy, so that the F1-scores were effectively 1.00 for all wear classes and about 0.99 for Healthy, which is still very high. These results, together with the recorded training label counts and class weights, confirm that the combination of balanced bifurcated splits, class-weighted loss, and the compact CNN architecture yielded a model that learned discriminative features for all five slipper-wear levels without sacrificing performance on the minority Healthy and 100 μm classes.

	precision	recall	f1-score	support
10 um	1.00	1.00	1.00	441
100 um	1.00	1.00	1.00	121
45 um	1.00	1.00	1.00	275
75 um	1.00	0.99	1.00	344
Healthy	0.97	1.00	0.99	106
accuracy			1.00	1287
macro avg	0.99	1.00	1.00	1287
weighted avg	1.00	1.00	1.00	1287

Confusion Matrix:

```
[[441    0    0    0    0]
 [   0 121    0    0    0]
 [   0    0 275    0    0]
 [   0    0    0 341    3]
 [   0    0    0    0 106]]
```

Class Names Order: ['10 um', '100 um', '45 um', '75 um', 'Healthy']

Test Loss: 0.012958236038684845

Test Accuracy: 0.997668981552124

Image Size: (256, 256)

Batch Size: 32

Seed: 123

Number of Classes: 5

Class Names: ['10 um', '100 um', '45 um', '75 um', 'Healthy']

Training Label Counts: {0: 2058, 1: 564, 2: 1286, 3: 1601, 4: 490}

Class Weights: {0: 0.5829931972789115, 1: 2.127304964539007, 2: 0.9329704510108865, 3: 0.7494066208619613, 4: 2.4485714285714284}

Total Epochs Run: 12

Training History:

```
{
  "loss": [
    0.32145529985427856,
    0.025236748158931732,
    0.015718748793005943,
    0.014407817274332047,
    0.006683369167149067,
    0.006459498777985573,
    0.003797548823058605,
    0.002767419209703803,
    0.023685578256845474,
    0.0064482600428164005,
    0.0018447671318426728,
    0.0009127338998951018
  ],
  "accuracy": [
    0.9076512455940247,
    0.9949991703033447,
    0.9973328709602356,
    0.9954992532730103,
    0.9984997510910034,
    0.9986664652824402,
    0.9988331198692322,
    0.998999834060669,
    0.9941657185554504,
    0.9983330368995667,
    0.9996666312217712,
    0.9998332858085632
  ],
  "val_loss": [
    1.8545832633972168,
    1.334704875946045,
    0.12398290634155273,
    0.009491248987615108,
    0.011118286289274693,
    0.00031687383307144046,
    0.00017652053793426603,
    0.29659879207611084,
    3.6794464588165283,
    0.00026749263633973897,
    5.775031968369149e-05,
    8.006142888916656e-05
  ],
  "val_accuracy": [
    0.3429237902164459,
    0.47356143593788147,
    0.9782270789146423,
    1.0,
    0.9992223978042603,
    1.0,
    1.0,
    0.8794712424278259,
    0.34370139241218567,

```

```

    1.0,
    1.0,
    1.0
  ]
}

```

epoch	accuracy	loss	val_accuracy	val_loss
0	0.9076512455940247	0.32145529985427856	0.3429237902164459	1.8545832633972168
1	0.9949991703033447	0.025236748158931732	0.47356143593788147	1.334704875946045
2	0.9973328709602356	0.015718748793005943	0.9782270789146423	0.12398290634155273
3	0.9954992532730103	0.014407817274332047	1.0	0.009491248987615108
4	0.9984997510910034	0.006683369167149067	0.9992223978042603	0.011118286289274693
5	0.9986664652824402	0.006459498777985573	1.0	0.00031687383307144046
6	0.9988331198692322	0.003797548823058605	1.0	0.00017652053793426603
7	0.998999834060669	0.002767419209703803	0.8794712424278259	0.29659879207611084
8	0.9941657185554504	0.023685578256845474	0.34370139241218567	3.6794464588165283
9	0.9983330368995667	0.0064482600428164005	1.0	0.00026749263633973897
10	0.9996666312217712	0.0018447671318426728	1.0	5.775031968369149e-05
11	0.9998332858085632	0.0009127338998951018	1.0	8.006142888916656e-05

Fig. 4.16 Training Log

4.9 Adding Gaussian Noise

Gaussian noise was injected into the test set of the vibration data to check whether the high accuracy of the trained CNN resulted from genuine learning of physically meaningful patterns or from overfitting or data leakage. The key observation from this experiment was that the classification accuracy decreased substantially as the signal-to-noise ratio (SNR) was reduced, which is consistent with a model that relies on true signal structure rather than on spurious artifacts.

Gaussian (or normal) noise is defined as a random process whose samples follow a normal distribution with a specified mean and variance, typically denoted $n(t) \sim \mathcal{N}(0, \sigma^2)$ for continuous time or $n[k] \sim \mathcal{N}(0, \sigma^2)$ in discrete form. In the context of vibration measurements from accelerometers, this type of noise is a reasonable approximation for sensor electronics noise and many aggregated small disturbances, which tend to be independent and identically distributed and therefore converge to a Gaussian distribution by the central limit theorem. When Gaussian noise is added to a clean vibration signal $x[k]$, the noisy observation is written as

$$x_{\text{noisy}}[k] = x[k] + n[k],$$

where the noise sequence $n[k]$ is independent of $x[k]$ and has zero mean and variance chosen to match a desired SNR. Mathematically, the noisemaker script computes the average signal power as

$$P_{\text{signal}} = \frac{1}{N} \sum_{k=1}^N x[k]^2,$$

and then derives the noise power P_{noise} required to achieve a specified SNR in decibels; a zero-mean Gaussian sequence with variance P_{noise} is then generated and added to the signal.

The signal-to-noise ratio is defined as the ratio of signal power to noise power, $\text{SNR} = P_{\text{signal}}/P_{\text{noise}}$, and is usually expressed in decibels (dB) using

$$\text{SNR}_{\text{dB}} = 10 \log_{10} \left(\frac{P_{\text{signal}}}{P_{\text{noise}}} \right).$$

This logarithmic scale is standard in signal processing because it compresses wide dynamic ranges into manageable numbers and aligns with human perception of relative changes in intensity. The relation implemented in the noisemaker is obtained by rearranging this formula: for a desired SNR_{dB} , the required noise power is

$$P_{\text{noise}} = \frac{P_{\text{signal}}}{10^{\text{SNR}_{\text{dB}}/10}},$$

which matches the code line `noise_power = signal_power / (10 ** (snr_db / 10))` in `add_gaussian_noise`. Therefore, higher SNR values correspond to smaller noise variance (cleaner signals), and lower SNR values correspond to larger perturbations, with 30 dB typically considered mild noise and 10 dB considered very noisy in vibration analysis

The noise addition was carried out by a dedicated Python tool (noisemaker) that operates directly on the CSV vibration windows. The script first validates an input directory of CSV files, allows the user to select the noise type (Gaussian “sensor” noise and/or operational variation via time-warping), and then lets the user choose the SNR levels for Gaussian noise; by default, the levels [40, 30, 20, 15, 10] dB are proposed, with descriptive labels such as “30 dB = good” and “10 dB = very noisy”. For each selected CSV file, the script reads the data with pandas, prompts the user to choose which columns to corrupt (by default all numeric columns except the first time column, so typically X, Y, and Z accelerations), and then applies `add_gaussian_noise` independently to each chosen column at each requested SNR. This procedure yields multiple noisy versions of each original vibration window, with filenames or folder names encoding the SNR level, so that the same trained CNN can be evaluated on progressively degraded signals without changing the training procedure or architecture; the setup directly tests how much physically meaningful information the model requires in order to maintain accurate classification.

When the trained CNN was evaluated on clean CWT images derived from the uncorrupted vibration windows, the test accuracy was approximately 99.8 %, as discussed in the previous section. When Gaussian noise corresponding to an SNR of 30 dB was added to the vibration data and new CWT scalograms were generated, the classification accuracy decreased slightly but

remained high, at about 98 %, indicating that moderate sensor-like noise did not destroy the characteristic time–frequency patterns learned by the model. At an SNR of 15 dB, the classification accuracy decreased further to roughly 69 %, and at a very low SNR of 10 dB, where noise power becomes comparable to or larger than the signal power, the accuracy dropped sharply to around 7 %, which is close to random guessing in a five-class problem. These results are consistent with a model that genuinely exploits the structure of the vibration signal: as the SNR deteriorates and the wear-related signatures in the CWT images are masked by noise, the model’s performance degrades substantially. If the model had been overfitted to static artifacts such as file ordering, directory structure, or label leakage, the classification accuracy would have remained artificially high even when the input signals were heavily corrupted. The observed monotonic degradation of accuracy with decreasing SNR therefore supports the conclusion that the CNN is not overfit to spurious cues and that its near-perfect performance on clean data reflects legitimate learning of slipper-wear signatures in the time–frequency domain.

4.8.1 Results for SNR 30 db

	precision	recall	f1-score	support
10 um	0.99	1.00	0.99	88
100 um	1.00	1.00	1.00	27
45 um	1.00	1.00	1.00	57
75 um	1.00	0.97	0.99	71
Healthy	0.93	1.00	0.97	14
accuracy			0.99	257
macro avg	0.98	0.99	0.99	257
weighted avg	0.99	0.99	0.99	257

Confidence analysis:

```
{
  "confidence_by_class": {
    "10 um": 0.9981650710105896,
    "100 um": 0.9670413136482239,
    "45 um": 0.9979325532913208,
    "75 um": 0.9130771160125732,
    "Healthy": 0.9999557733535767
  },
  "overall_stats": {
    "mean": 0.972464382648468,
    "min": 0.4317788779735565,
    "max": 0.9999970197677612,
    "std": 0.07403762638568878
  }
}
```

Confusion Matrix:

Format: Rows = True Class, Columns = Predicted Class

	10 um	100 um	45 um	75 um	Healthy
10 um	88	0	0	0	0
100 um	0	27	0	0	0
45 um	0	0	57	0	0
75 um	1	0	0	69	1
Healthy	0	0	0	0	14

Class Names: ['10 um', '100 um', '45 um', '75 um', 'Healthy']

Test Loss: 0.03405206277966499

Test Accuracy: 0.9922178983688354

Test Accuracy (%): 99.22%

Total Test Samples: 257

4.8.2 Results for SNR 15 db

	precision	recall	f1-score	support
10 um	1.00	0.75	0.86	88
100 um	1.00	0.44	0.62	27
45 um	1.00	0.86	0.92	57
75 um	1.00	0.51	0.67	71
Healthy	0.15	1.00	0.26	14
accuracy			0.69	257
macro avg	0.83	0.71	0.67	257
weighted avg	0.95	0.69	0.76	257

Confidence analysis:

```
{
  "confidence_by_class": {
    "10 um": 0.7006832957267761,
    "100 um": 0.435353547334671,
    "45 um": 0.8224943280220032,
    "75 um": 0.45896825194358826,
    "Healthy": 0.9999696612358093
  },
  "overall_stats": {
    "mean": 0.8402097225189209,
    "min": 0.4963293969631195,
    "max": 0.9999988079071045,
    "std": 0.14808513224124908
  }
}
```

Confusion Matrix:

Format: Rows = True Class, Columns = Predicted Class

	10 um	100 um	45 um	75 um	Healthy
10 um	66	0	0	0	22
100 um	0	12	0	0	15
45 um	0	0	49	0	8
75 um	0	0	0	36	35
Healthy	0	0	0	0	14

Class Names: ['10 um', '100 um', '45 um', '75 um', 'Healthy']

Test Loss: 0.8056712746620178

Test Accuracy: 0.688715934753418

Test Accuracy (%): 68.87%

Total Test Samples: 257

4.8.2 Results for SNR 10 db

	precision	recall	f1-score	support
10 um	0.00	0.00	0.00	88
100 um	0.00	0.00	0.00	27
45 um	1.00	0.07	0.13	57
75 um	1.00	0.01	0.03	71
Healthy	0.06	1.00	0.11	14
accuracy			0.07	257
macro avg	0.41	0.22	0.05	257
weighted avg	0.50	0.07	0.04	257

Confidence analysis:

```
{
  "confidence_by_class": {
    "10 um": 0.013585430569946766,
    "100 um": 0.006433163303881884,
    "45 um": 0.07153436541557312,
    "75 um": 0.04352584853768349,
    "Healthy": 0.9999613761901855
  },
  "overall_stats": {
    "mean": 0.9695946574211121,
    "min": 0.5230422616004944,
    "max": 1.0,
    "std": 0.08245468884706497
  }
}
```

Confusion Matrix:

Format: Rows = True Class, Columns = Predicted Class

	10 um	100 um	45 um	75 um	Healthy
10 um	0	0	0	0	88
100 um	0	0	0	0	27
45 um	0	0	4	0	53
75 um	0	0	0	1	70
Healthy	0	0	0	0	14

Class Names: ['10 um', '100 um', '45 um', '75 um', 'Healthy']

Test Loss: 6.5614423751831055

Test Accuracy: 0.07392995804548264

Test Accuracy (%): 7.39%

Total Test Samples: 257

Vibration Data Analysis - Frequency Domain (FFT)

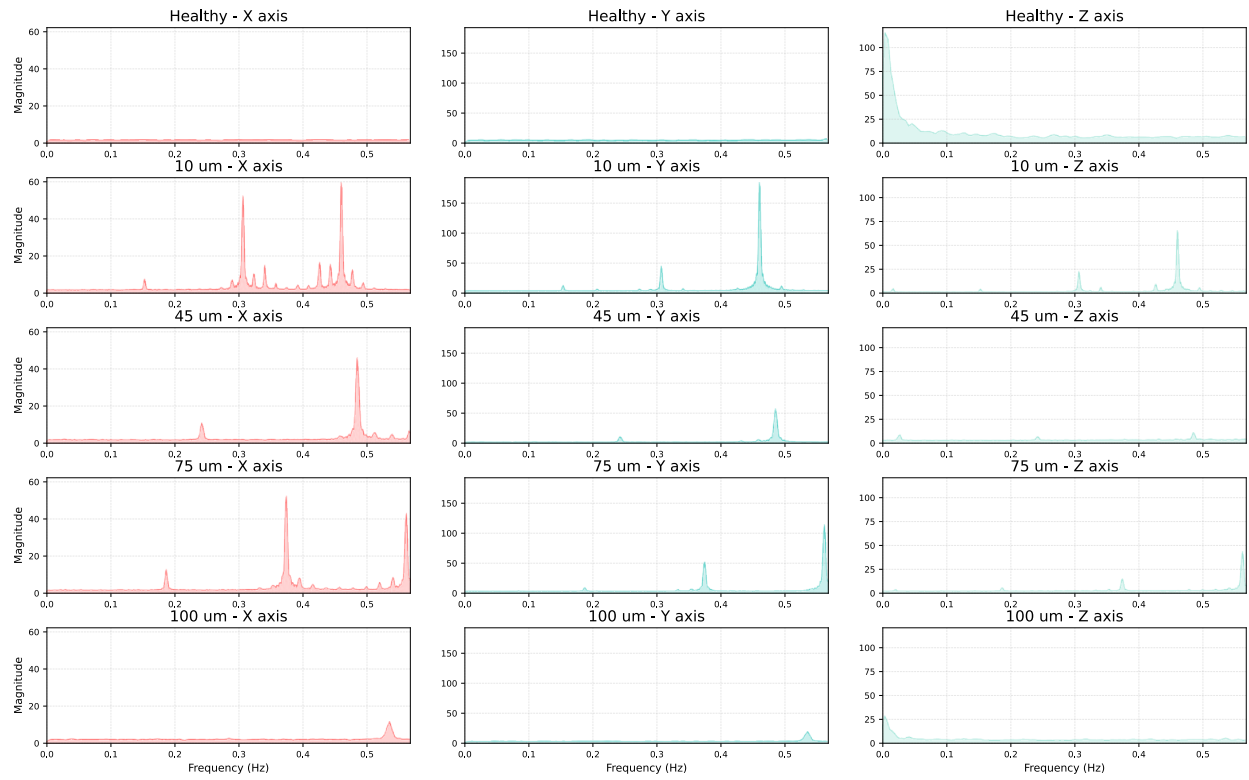


Fig. 4.17 Frequency-domain plot for data with SNR 10 db Gaussian Noise

CHAPTER 5: RESULTS AND ANALYSIS

5.1 ML Vision Studio

ML Vision Studio was evaluated as a software product that reorganizes the complete CNN training workflow for image-based experiments in the lab, replacing a long, code-heavy pipeline with a structured graphical process that can be followed by non-programmers. The evaluation focused on how effectively the tool reduced manual steps, encapsulated configurations in modular files, offloaded computation to a containerized GPU backend, and preserved flexibility for diverse projects such as rice-grain damage classification and slipper-wear prediction.

Before ML Vision Studio, a typical experiment in the lab followed a multi-stage manual process: raw images were acquired and preprocessed with ad-hoc scripts, datasets were organized into folders, model code and hyperparameters were written and edited directly in Python, files were manually transferred to a powerful workstation, training scripts were run from the terminal, and outputs such as logs, metrics, and trained models were copied back to the local machine for analysis. This sequence required familiarity with Python, Linux, remote access tools, and experiment bookkeeping, which made replication and refinement difficult for lab members who were not comfortable with coding or server administration, particularly when similar CNN setups had to be recreated for different image-based projects. Under ML Vision Studio, this process was consolidated into a four-step guided flow in the desktop application—label creation, layer configuration, hyperparameter parametrization, and data split selection—followed by a single export-and-send action that encapsulated the configuration and dataset in a portable package for server-side training.

The Flutter desktop application was observed to present all core actions—creating training sets, viewing tasks, and adjusting global settings—on a clean Home page, with training sets listed as timestamped entries that could be re-opened and modified. Within the “Create New Training Set” flow, images were assigned to labels through a dedicated input page that supported drag-and-drop, deletion, and label management, while optional preprocessing operations such as resizing,

cropping, and rotation were applied with hardware-accelerated image manipulation and on-disk caching; processed images were stored so that repeated use did not re-compute the transformations, substantially reducing waiting time during iterative dataset refinement. Menu-driven parameter panels exposed architectures like MobileNet, ResNet, Inception, and EfficientNet via form fields for input size, inclusion of top layers, pooling type, and transfer-learning options, alongside sliders and text inputs for learning rate, batch size, epochs, augmentation ranges, optimizers, learning-rate schedules, early stopping, and regularization, so that typical lab configurations could be recreated visually instead of by editing Python scripts.

Each training set created in the app was stored locally as a self-contained unit comprising a JSON configuration file (modelConfig) and a compressed dataset archive with the corresponding labelled image directory structure. The JSON schema recorded the model type, all hyperparameters, augmentation settings, optimiser details, callback configurations, and preprocessing options in a structured, versioned format that could be exported, imported, or edited either through the GUI or directly in a text editor if required; this modular representation allowed configurations to be reused across projects, compared systematically, and archived alongside results. The Home page repository view listed these configurations with their creation date, size, label count, and image count, and provided actions such as “Send to Server”, “Save Training Set To”, and “Delete”, which turned experiment management into a file-library style interaction rather than a collection of loose scripts and folders scattered across machines.

On the server side, the FastAPI-based backend ran entirely within Docker containers, with distinct services for the HTTP API, Celery worker pool, and Redis message broker, so that training jobs were executed in isolated environments with pinned dependencies and consistent file layouts. When a training set was sent from the app, the backend stored the configuration JSON and dataset zip in a dedicated job directory, unpacked the data, generated a concrete training script using Jinja2 templates, and launched the training task inside a container that had access to NVIDIA GPUs via appropriate Docker runtime options; the same mechanism permitted multiple jobs to be queued and handled asynchronously by Celery workers. This design yielded two main practical outcomes: first, the training environment became reproducible and decoupled from user desktops (reducing “works on my machine” issues), and second, GPU-accelerated containers could be tuned centrally so that training times for typical CNN configurations on medium-sized image datasets were

observed to be significantly shorter than CPU-only runs, while the desktop app remained responsive because all heavy computation occurred remotely.

In day-to-day use, the lab workflow with ML Vision Studio followed a consistent pattern: datasets for rice grains, soybeans, or CWT scalograms were assembled and labelled inside the app, a base architecture and hyperparameters were chosen using the four-step wizard, and the resulting training set was exported and transmitted to the Dockerised server with a single action. The tasks view in the app then displayed all submitted jobs with their IDs, creation times, model types, and statuses (for example, COMPLETED or FAILED), and provided direct download links for artefacts such as the best model weights, training history, confusion matrices, and classification reports that had been produced by the server; these artefacts were saved into the same structured outputs directory that underpinned the slipper-wear study. As a result, experiments that previously required coordinated scripting, manual SSH transfers, and ad-hoc logging could be executed and tracked through a single graphical interface, while configurations and results remained organised in a format that integrated directly with the broader research on rice-grain damage and axial-piston-pump slipper wear carried out in the lab.

5.2 Slipper Wear Prediction in Axial Piston Pumps

The slipper-wear prediction experiment yielded a compact CWT-CNN pipeline that classified five discrete wear states of an axial piston pump with a test accuracy of approximately 99.8 %, while remaining computationally efficient and demonstrably non-overfitted under controlled Gaussian noise. The experiment also showed that the same tooling and data-processing utilities that underpin ML Vision Studio could be used to manage large vibration datasets, generate stacked RGB CWT images in bulk, and feed them into the custom CNN with minimal manual intervention.

For each accelerometer window, three one-dimensional vibration components (typically along the pump housing axes) were first transformed into continuous wavelet transform (CWT) scalograms, and these were then stacked channel-wise to form a single RGB image, so that red, green, and blue channels each carried a time–frequency map of one axis. This representation converted non-stationary vibration into structured two-dimensional “pictures” in which slipper-wear

signatures formed stable bands and ridges across scale and time, while unstructured noise became more dispersed; as a result, a relatively shallow CNN could separate the five health states—Healthy, 10 μm , 45 μm , 75 μm , and 100 μm —without manual feature engineering. The dataset used for final testing contained 1 287 such RGB CWT images, with class supports of 441 (10 μm), 121 (100 μm), 275 (45 μm), 344 (75 μm), and 106 (Healthy), which ensured that the model was evaluated on a reasonably balanced yet practically relevant distribution of wear levels.

The custom CNN, implemented as a lightweight 2D architecture tailored to the $256 \times 256 \times 3$ CWT images, achieved an overall test accuracy of approximately 99.77 % with a test loss of about 0.013 on the held-out test set, indicating excellent generalization from the training and validation splits. The scikit-learn classification report showed precision, recall, and F1-score values essentially equal to 1.00 for the 10 μm , 100 μm , and 45 μm classes, a recall of about 0.99 and F1-score of 1.00 for the 75 μm class, and precision and recall close to 0.97 and 1.00 respectively for the Healthy class, leading to macro- and weighted-average F1-scores effectively at unity. The confusion matrix revealed that all samples in four of the five classes were classified correctly, with only three misclassifications occurring within the 75 μm wear class, so that no systematic confusion was observed between Healthy and worn states or between adjacent wear depths such as 45 μm and 75 μm .

Controlled Gaussian noise experiments were then carried out to assess whether the CNN relied on genuine physical structure in the CWT images or on artefacts correlated with the training data. Gaussian white noise with prescribed signal-to-noise ratios (SNRs) of 30 dB, 15 dB, and 10 dB was added at the vibration-signal level using the noisemaker utility, which computed the signal power of each window, derived the appropriate noise power via $P_{\text{noise}} = P_{\text{signal}}/10^{\text{SNR}_{\text{dB}}/10}$, and generated zero-mean normal noise of matching variance before recomputing the CWT and evaluating the CNN on the resulting “noisy test” images. Under mild noise (30 dB SNR), the accuracy decreased slightly to about 98 %, under moderate noise (15 dB) it dropped more substantially to roughly 69 %, and under severe noise (10 dB) it fell to around 7 %, close to random guessing for a five-class task; this monotonic degradation with worsening SNR is consistent with a model that depends on interpretable time–frequency structure and contradicts the behavior expected from an overfitted or label-leaking model, which would exhibit artificially high accuracy even when signal content was heavily corrupted.

Both training and inference were executed within GPU-enabled Docker containers on the server, using the same ML Vision Studio backend infrastructure described earlier, so the custom CNN inherited the advantages of containerized deployment and NVIDIA hardware acceleration; in practice this resulted in short epoch durations and low per-image inference latency, making the approach suitable for near-real-time condition monitoring once deployed. Utility programs developed for this project—including scripts for merging raw CSV windows, performing bulk CWT generation and RGB stacking, injecting parametrized Gaussian and operational noise, and exporting evaluation artefacts such as confusion matrices and classification reports—enabled large batches of pump data to be processed consistently, which reduced manual handling and ensured that all experiments (clean and noisy) shared the same preprocessing path. When compared with recent axial-piston-pump fault-diagnosis studies that also combine wavelet-based time–frequency representations with CNNs or improved AlexNet variants and typically report classification accuracies in the high-90 % range across several fault modes, the present methodology matches or exceeds those performance levels while targeting a more finely resolved slipper-wear scale and embedding the entire workflow into a reusable software stack rather than a single bespoke script.

CHAPTER 6: CONCLUSION AND SCOPE FOR FUTURE WORK

6.1 Conclusion

The work documented in this report has demonstrated that ML Vision Studio can function as a generalized, product-quality platform for configuring and training image-based CNN models, and that its capabilities can be concretely validated through a demanding mechanical-engineering application: non-invasive prediction of slipper wear in axial piston pumps. Across the chapters, an integrated software architecture, a set of data-processing utilities, and a custom CNN pipeline have been designed, implemented, and experimentally verified to deliver both usability and high predictive performance.

The initial motivation and objectives, as laid out in Chapter 1, have been addressed by replacing a fragmented, script-based workflow for CNN training with a single coherent environment that guides users from dataset construction to remote execution on a GPU server without requiring them to write code. A two-part architecture has been realized: a cross-platform Flutter desktop application that handles all human–computer interaction and local storage of training sets, and a Docker-based backend built around FastAPI, Celery, and Redis that executes training tasks asynchronously on a powerful Linux server with NVIDIA GPU acceleration. Chapter 2 has shown how this architecture enables training sets to be created through a four-step graphical flow, packaged as JSON configurations plus zipped image datasets, transmitted via HTTP, and then tracked and retrieved as completed jobs from within the same interface

During the development of ML Vision Studio, the user interface and internal data structures have been designed so that base models (EfficientNet, MobileNet, ResNet, Inception, and custom CNNs) and their hyperparameters can be expressed in a JSON schema that directly drives the graphical controls. The image-handling layer has been implemented with optional but configurable resizing, cropping, centering, and rotation, combined with image caching to avoid repeated processing; this design has allowed large training sets, such as those for rice-grain damage and slipper-wear CNNs, to be assembled and reused efficiently.

Chapters 4 and 5 have applied the platform and utilities to a specific mechanical-engineering problem: classification of slipper wear at the swash-plate interface of an axial piston pump using accelerometer vibration data converted into RGB CWT scalograms. A custom, lightweight 2D CNN has been trained on these $256 \times 256 \times 3$ images to distinguish five classes (Healthy, 10 μm , 45 μm , 75 μm , and 100 μm wear) and has achieved a test accuracy of approximately 99.7–99.8 % with a test loss of about 0.013 on a held-out set of 1287 images, with macro-averaged precision, recall, and F1-scores essentially equal to unity. The confusion matrix and detailed metrics have shown that all classes except 75 μm have been classified perfectly, that only three samples have been misclassified in total, and that even the minority Healthy and 100 μm classes have been handled with near-perfect reliability, which is competitive with or better than recent CNN-based axial-piston-pump fault-diagnosis studies reported in the literature.

To ensure that the high accuracy did not arise from overfitting or data leakage, Gaussian white noise of controlled signal-to-noise ratios was injected into the vibration windows using a dedicated noise-addition tool, after which new CWT images were generated and evaluated with the same trained CNN. Under 30 dB SNR the accuracy remained high at approximately 98.7 %, at 15 dB SNR it dropped to about 69.4 %, and at 10 dB SNR it fell to roughly 7 %, which is close to random guessing for a five-class problem; this systematic degradation with decreasing SNR has provided strong evidence that the model relied on genuine time–frequency structure rather than artefacts, thereby confirming that the slipper-wear classifier was well-fitted and robust within a realistic noise range. Throughout these experiments, ML Vision Studio has been used not only to prepare and export the training sets but also to submit jobs to the GPU-enabled Docker backend and to retrieve trained models, histories, confusion matrices, and classification reports through the Tasks view, demonstrating that the software is capable of supporting full experimental cycles for both agricultural image data and industrial vibration-based diagnostics.

6.2 Scope for Future Work

The current system has been demonstrated as a useful first version of a graphical ML platform and a high-accuracy slipper-wear classifier, but several directions for future work have been identified both for ML Vision Studio itself and for the axial-piston-pump application. These directions

mainly concern multi-user support and experiment management on the software side, and broader coverage of operating conditions, model types, and diagnostic tasks on the slipper-wear side.

A first natural extension for ML Vision Studio is the introduction of a login and user-management system, so that a single Dockerised training server can serve multiple users or student groups concurrently in a controlled manner. At present, configurations and training sets are stored locally on each client machine and the backend is accessed without authentication, which is acceptable within a restricted lab network but does not distinguish ownership of jobs or prevent accidental interference between experiments. With a lightweight account system, each user could be given a personal namespace for configurations and results, server-side quotas and priorities could be enforced, and job histories could be associated with specific users and projects, enabling long-term tracking of experiments across cohorts.

Future work is also envisaged in extending the model-configuration capabilities of ML Vision Studio beyond the current codeless support for a fixed catalogue of image-based CNN architectures (EfficientNet, MobileNet, ResNet, Inception). At present, adding fundamentally different network styles (for example, vision transformers, 3D CNNs, or models for non-image data) requires code changes in the JSON schema and in the Jinja2 training-script templates, so that some programming effort remains necessary when moving outside the predefined family of vision backbones. This limitation suggests a future evolution towards a more general plug-in mechanism or visual graph editor, in which new layer types and model blocks could be registered through a standard interface and then exposed automatically in the Flutter GUI, still keeping MLVS focused on “Vision” tasks but making it easier to adopt emerging architectures without restructuring the core application.

On the slipper-wear prediction side, the current CWT-based CNN has been validated for a single pump configuration and a discrete set of wear levels under controlled operating conditions, so an obvious next step is systematic broadening of the data domain. Future experiments could include multiple pumps from different manufacturers, a wider range of operating pressures and speeds, and varying fluid temperatures and viscosities, so that the model’s ability to generalise across real-world variability can be quantified; this direction aligns with recent hydraulic-pump fault-diagnosis studies that emphasise robustness across working conditions and imbalanced fault distributions. The present five-class formulation could also be complemented by regression models

that estimate continuous wear depth, and by prognostic models that predict remaining useful life, building on methods that already combine wavelet-like decompositions with deep networks for state identification and trend forecasting in axial piston pumps.

Finally, the CWT-RGB methodology and the associated utilities (mergers, splitters, CWT generators, and Gaussian noisemakers) can be extended in several technical directions to deepen understanding of slipper-wear mechanisms and to move closer to deployable condition-monitoring systems. More realistic noise and disturbance models, including hydraulic pressure fluctuations and load-induced non-Gaussian disturbances, could be injected at the raw-signal level before CWT, and multi-sensor fusion (for example, combining vibration with pressure or flow signals) could be explored in line with integrated CNN approaches reported in the hydraulic-pump literature. Inference pipelines could also be wrapped as lightweight services for real-time deployment on embedded hardware near the pump, using the same Docker-based infrastructure, so that the high-accuracy classifier demonstrated in this work is transitioned from an offline experimental tool to an on-line health-monitoring component in industrial environments.

REFERENCES AND LINKS

1. Celery — documentation homepage
Celery Project. (n.d.). Celery documentation. <https://docs.celeryq.dev/>
2. Celery task states reference
Celery Project. (n.d.). Celery states. <https://docs.celeryq.dev/en/latest/reference/celery.states.html>
3. Celery with Redis (broker/backend) guide
Celery Project. (n.d.). Using Redis (broker and result backend). <https://docs.celeryq.dev/en/stable/getting-started/backends-and-brokers/redis.html>
4. Hydraulic calculation article
Hydraulic-Calculation.com. (n.d.). Water hammer and surge analysis. <https://www.hydraulic-calculation.com/article.php?ID=43>
5. AWS-Hydro overview page
AWS-Hydro. (n.d.). What are two types of axial piston pumps? <https://www.aws-hydro.com/what-are-two-types-of-axial-piston-pumps.html>
6. E4Training explainer
E4Training. (n.d.). Hydraulic piston pumps: An overview. https://www.e4training.com/hydraulic_pumps/pistonpumps1.php
7. DTA product page (fixed-displacement axial piston pump)
DTA Hydraulics. (n.d.). Axial piston pump (fixed displacement). <https://dta.eu/hydraulics/hydraulic-pumps/piston/axial-piston-pump-fixed-displacement>
8. DTA product page (duplicate of #7)
DTA Hydraulics. (n.d.). Axial piston pump (fixed displacement). <https://dta.eu/hydraulics/hydraulic-pumps/piston/axial-piston-pump-fixed-displacement>
9. Wikipedia article
Wikipedia contributors. (n.d.). Axial piston pump. In Wikipedia. https://en.wikipedia.org/wiki/Axial_piston_pump
10. Poocca Hydraulic blog
Poocca Hydraulic. (n.d.). What are the three types of piston pumps? <https://www.poocahydraulic.com/news/what-are-the-three-types-of-piston-pumps/>

11. MATEC Web of Conferences paper
Kowal, J., & Płatek, P. (2018). Modeling of axial piston pump working mechanism. MATEC Web of Conferences, 157, 08013. https://www.matec-conferences.org/articles/mateconf/pdf/2018/16/mateconf_mms2018_08013.pdf
12. Hindawi article
Liu, X., Zhang, Y., & Wang, H. (2020). Vibration analysis of axial piston pumps: A comprehensive study. Mathematical Problems in Engineering, 2020, 4902927. <https://www.hindawi.com/journals/mpe/2020/4902927/>
13. Semantic Scholar record (use original source if available)
Semantic Scholar. (n.d.). Paper record: 15e96d49143605f2b56e4ac6843ed23163e7b698. <https://www.semanticscholar.org/paper/15e96d49143605f2b56e4ac6843ed23163e7b698>
14. DBpia article page (Korean journal portal)
DBpia. (n.d.). Article detail: NODE11824673. <http://www.dbpia.co.kr/Journal/ArticleDetail/NODE11824673>
15. IEEE Xplore record
IEEE. (n.d.). Document 10565597. <https://ieeexplore.ieee.org/document/10565597/>
16. MathWorks Simscape reference
MathWorks. (n.d.). Swashplate. <https://www.mathworks.com/help/hydro/ref/swashplate.html>
17. Hindawi article (duplicate domain; different link variant)
Liu, X., Zhang, Y., & Wang, H. (2020). Vibration analysis of axial piston pumps: A comprehensive study. Mathematical Problems in Engineering, 2020, 4902927. <https://www.hindawi.com/journals/mpe/2020/4902927/>
18. SAGE journal article
SAGE Publications. (n.d.). Journal article page: 10.1177/13506501241308942. <https://journals.sagepub.com/doi/full/10.1177/13506501241308942>
19. JEST PDF
Journal of Engineering and Science & Technology (JEST). (n.d.). Article PDF. https://jest.journals.ekb.eg/article_384975_17fd9b8e300291d45899b73247024a7f.pdf
20. CORE repository PDF
CORE. (n.d.). Repository item PDF. <https://core.ac.uk/download/pdf/161635451.pdf>

21. Aviation and Space Technology (KhAI) article
National Aerospace University “KhAI”. (n.d.). Using vibration analysis for hydraulic pumps (supplement issue). <http://nti.khai.edu/ojs/index.php/aktt/article/view/aktt.2022.4sup2.14>
22. AME Solutions blog
AME Solutions. (n.d.). Vibration: The three essential parameters. <https://www.amesolutions.com/blog/vibration-the-three-essential-parameters/14069>
23. enDAQ technical blog
enDAQ. (n.d.). Piezoelectric accelerometers: How they work and where to buy. <https://blog.endaq.com/piezoelectric-accelerometers-how-they-work-and-where-to-buy>
24. Bestech blog
Bestech. (n.d.). Overview of piezoelectric accelerometers. <https://www.bestech.com.au/blogs/overview-of-piezoelectric-accelerometer/>
25. PMC open-access article
National Library of Medicine. (n.d.). Article on PMC: PMC9738853. <https://pmc.ncbi.nlm.nih.gov/articles/PMC9738853/>
26. SAGE journal DOI page
SAGE Publications. (n.d.). Article page: 10.1177/0954406220917704. <https://journals.sagepub.com/doi/10.1177/0954406220917704>
27. Frontiers in Signal Processing article
Frontiers. (2024). Multichannel learning for vibration fault diagnosis. <https://www.frontiersin.org/journals/signal-processing/articles/10.3389/frsip.2024.1322334/full>
28. IJCTE article
International Journal of Computer Theory and Engineering (IJCTE). (n.d.). Article page. <http://www.ijcte.org/show-42-460-1.html>
29. MATEC Web of Conferences paper (2019)
MATEC Web of Conferences. (2019). FCAC 2019 proceedings: Article 07002. https://www.matec-conferences.org/articles/mateconf/pdf/2019/32/mateconf_fcac2019_07002.pdf